基于连续覆盖与费用驱动决策的无线电干扰源自动定位与清除

摘　要

无线电干扰源的自动清除同时受到测向误差、接收范围未知和定向遮蔽的影响。机器狗还需在清除已知目标后确认场内没有遗漏。为此，我们先用**连续覆盖**保证干扰源可被发现，再利用**有界误差集合**实现可靠定位，并根据各类动作的**虚拟时间**安排测量、移动和清除。

针对问题一，我们将多个示向**角楔统一取交**作为干扰源的可能位置区域，并分别计算**区域直径**和**最小包围圆**。模型能够处理空集、无界集以及点、线段等退化情形。进一步构造的三次合法示向可形成直径 40 米的等边三角形，而其最小包围圆半径约为 23.09 米，说明区域直径的一半**不能直接作为**可靠清除半径。

针对问题二，我们由首次成功接收推导接收半径的**隐含下界**，并将鲁棒第二测点的安全域化为四个圆盘的交集。随后利用连续角区间外包和可实现点对估计定位精度的上下界。在 1.005°工作误差下，精度型测点使最坏定位直径至少改善 14.30%，相应增加约 20.31 秒移动时间。

针对问题三，我们采用**原点加六个环点**的连续发现布局，并结合**正负反馈几何**、双侧测点选择和一次后验光学选点完成定位与清除。综合普通场景的**时间收益**和聚集场景的**退步风险**后，我们选择**稳定性更好的局部费用策略**。该策略在 84 个独立确认场景中全部完成清除，平均定位清除时间较共同基线降低 1.24%。

针对问题四，我们将未知朝向下的通用发现条件转化为近邻测点的**凸包覆盖条件**，并构造通过连续认证的 21 点布局。结合内圆周、边界覆盖和朝外接收的加权约束，可证明固定通用布局至少需要 19 点，因此该布局至多多用 2 点，点数比不超过 21/19。在此基础上，我们筛选已知频道的补充测量，并根据途中获得的真实观测重新安排任务。该策略在 84 个独立确认场景中**全部完成清除**，平均定位清除时间降低 6.73%。

问题三、四各三次正式测试累计清除全部 40、38 个源，平均定位清除时间分别为 225.84、442.91 秒/源。时间统计包含失败试清和无遗漏确认。我们通过同场对照、单项关闭实验和退步轨迹分析策略的收益与风险。

关键词：途中测向；连续覆盖；有界示向误差；最小包围圆；费用驱动决策

一、问题背景与重述

1.1 问题背景

无线电频谱是信息通信系统正常运行的重要基础。无线电干扰会降低通信质量并影响关键业务，因此需要尽快确定干扰源位置并实施清除。题目提出由机器狗搭载测向机、光学探测仪和清除装置完成精准定位阶段的检测、定位与清除。

无线电干扰源的自动清除需要同时解决搜索、定位和可靠退出问题。由于干扰源的位置、接收半径和发射方向未知，这使问题同时具有观测不确定性、搜索完整性和任务时间约束。机器狗只能通过逐次检测获得信息，并在有限时间内完成全部清除。测向误差和未知源数量进一步增加了任务难度。

1.2 问题重述

题目要求机器狗从原点出发，在半径 1800 米的圆形区域内定位并清除 10 至 16 个干扰源。20 个可选频道中，每个干扰源占用不同频道。源的位置、频道、发射方向和接收半径在一次测试中保持不变，不同干扰源之间互不影响。接收半径 R 满足 $1000 \le R \le 1500$ 米；全向源的接收区域为圆盘，定向源的接收区域为该圆盘与可见闭半平面的交集。[1]

正常示向的真实误差不超过$\pm 1°$，同一地点重复测量不能消除该固定误差。当机器狗距离干扰源不超过 5 米且处于辐射范围内时，系统返回近距响应；光学清除的有效范围为 20 米，并且不受射频发射方向影响。机器狗需要根据这些反馈完成定位、清除和无遗漏确认。[1]

围绕自动定位与清除，需要解决以下四个问题：

(1) 根据若干检测点的位置与有误差示向，求干扰源可能位置区域及其直径，并判断是否总能用同直径的圆覆盖该区域。

(2) 首次检测得到正常示向后，确定能够保证再次接收信号的第二检测位置，并分析该位置对定位精度的影响。

(3) 面向全向干扰源，设计可自动执行的搜索、定位、清除与退出算法，以实际清除数和平均定位清除时间评价表现。

(4) 将自动算法扩展至包含定向源的场景，在发射方向未知的情况下保证搜索的完整性，分析其时间代价并记录测试结果。

二、问题分析

四个问题由单源定位逐步扩展至多源搜索与清除，均需先保证任务可靠完成，再提高执行效率。为此，本文采用“可行性保障—费用优化法”的两层框架：前者利用有界误差集合、连续覆盖、安全测点和有限后备方案，保证干扰源能够被发现、定位并清除；后者**以完整任务的虚拟时间为费用**，将移动、检测、切频、光学尝试和激光清除等耗时统一到时间尺度，在满足安全性与完整性要求的候选

动作中估计当前及后续费用，并据此安排测点、访问顺序和补充测量时机。本文围绕题设接收条件，推导四圆盘安全域与发现覆盖界，并据此设计在线定位与清除策略。

2.1 问题一 有界示向下的位置集合与清除判据

每次带误差示向对应一个角楔，多次观测的交集构成干扰源的可能位置区域。我们分别计算该区域的最大点间距离和最小包围圆：前者描述候选位置间的最大差异，后者判断能否通过一次光学操作可靠清除。由于区域直径的一半不一定等于最小覆盖半径，还需构造反例说明二者关系，并处理空集、无界集以及点、线段等退化情形，从而建立后续算法统一使用的位置表示和清除判据。

2.2 问题二 安全第二测点与精度费用权衡

第二测点距离首测点过近时，两次示向的交会效果较差；移动过远时，又可能超出未知接收范围。为此，我们先由首次成功接收推导接收半径的隐含下界，确定能够保证再次接收的安全测点域，再在该区域内比较候选测点的最坏定位误差和移动时间，从而在不失联的前提下分析定位精度与行动费用之间的权衡。

2.3 问题三 全向源的连续发现与费用驱动调度

问题三需要在干扰源数量和频道未知的情况下，联合完成搜索、定位、清除和无遗漏确认。我们利用原点及六个环形测点构成连续覆盖骨架，保证任意合法全向源能够被发现，并通过安全双侧测点、可靠清除判据和有限后备维持任务完整性。在此基础上，将移动、逐频道检测、切频、失败尝试和无遗漏确认全部计入时间账本，只对通过几何检验的候选测点和任务顺序进行费用排序，并根据真实反馈持续更新可行域和调度方案。本问由此形成费用驱动决策的主体框架。

2.4 问题四 定向源的完整覆盖与途中信息决策

加入发射方向未知的定向源后，无信号既可能源于距离过远，也可能源于测点位于发射背面，因此不能直接沿用第三问的距离排除方法。我们将任意朝向下的发现要求转化为近邻测点的凸包覆盖条件，并仅在满足附加几何条件时利用双侧阴性反馈缩小候选区域。随后继承第三问的费用定位框架，增加已知频道补测、途中测向、路线保留和反馈后重排。

三、模型假设

根据题设给出的静止源、固定接收半径和串行动作条件，我们对信息表示和数值计算作如下假设。

(1) 假设接口返回的机器狗坐标可作为检测点坐标直接使用，不另加题目未给定的位置误差；定位不确定性由有界示向误差表示。

(2) 假设两位小数示向采用最近舍入，显示舍入误差不超过 $0.005°$。因此程序使用 $1.005°$作为工作误差半宽，理论推导仍按题设 $1°$分别讨论。

(3) 为比较尚未执行的候选动作，我们采用位置面积和接收半径均匀的参考先验；在定向情形下，再假设两类干扰源等权且相容方向均匀。候选动作按先验预测费用排序，位置可行域与退出条件由题设约束和实际反馈确定。

四、符号说明

表 1 主要符号及其含义

符号	含义与适用范围	单位
G	待定位的实际源位置，二维坐标向量	m
x	与观测相容的候选源位置，二维坐标向量	m
s_i，q	第 i 个检测点坐标；下一检测或清除位置坐标	m
n；i，j	单源累计正常示向数；检测点索引、频道索引	—
θ_i	在 s_i处取得的示向读数	°
δ	示向误差半宽：理论值 1°，工作值 1.005°	°
e(φ)	方向角φ对应的二维单位向量	—
W_i	第 i 次正常示向确定的角楔区域	集合
P	全部已用观测共同约束的定位可行区域	集合
D(P)	区域 P 内任意两点的最大距离	m
c*	覆盖区域 P 的最小包围圆圆心坐标	m
r*	覆盖区域 P 的最小包围圆半径	m
R	同一源固定且未知的接收半径，1000≤R≤1500	m
H_j；U	频道 j 实际检测位置集；尚未发现源的频道集	集合
L	一场运行中已接受动作的累计移动距离	m
M；K	累计无线电检测次数；累计实际切频次数	次
C；S	累计光学尝试数（含失败）；成功清除源数	次；个
T；t	单场总虚拟时间；单场平均定位清除时间 T/S	s；s/源

除特别说明外，文中角度均以度为单位，计算三角函数时先乘 $\pi/180$ 转换为弧度。符号 $e(\varphi)$ 表示方向角 φ 对应的单位向量，图中位置以米为单位，横纵坐标采用相同比例。

五、模型建立与求解

5.1 问题一 多测点交会区域与可靠清除

5.1.1 模型准备

给定 n 个检测点及其示向，我们先根据题设误差建立交会区域，再分别计算区域直径和光学圆覆盖范围。求解时同时处理空集、无界集以及点和线段等退化情形。

编写程序输出区域状态、边界顶点、直径和最小包围圆。为控制数值误差，我们在集合运算中保留向外容差，并以 19.99 米作为具有余量的清除判据。

5.1.2 模型建立

从角度误差得到位置集合：

在测点 s_i 取得示向 θ_i 后，干扰源的真实方向位于 $\theta_i-\delta$ 与 $\theta_i+\delta$ 之间。我们用两条边界射线形成的角楔表示本次观测，并将全部角楔取交集，得到与所有观测相容的位置区域。该表示保留了最不利角度误差，这类有界误差集合的表示方法可参见相关文献[2]。

$$u_i^{\pm} = (cos(\theta_i \pm \delta), sin(\theta_i \pm \delta)) \tag{1}$$

$$W_i = \{x : u_i^- \times (x - s_i) \geq 0,\ u_i^+ \times (x - s_i) \leq 0\} \tag{2}$$

$$P = \bigcap_{i=1}^{n} W_i = \{x : Ax \leq b\} \tag{3}$$

其中，u_i^-和 u_i^+分别表示第 i 次角楔的下、上边界方向，A 和 b 为汇总全部半平面约束后得到的系数矩阵和常数向量。

二维叉积 $a \times b = a_x b_y - a_y b_x$用于判断点位于边界直线的哪一侧。下边界保留左侧区域，上边界保留右侧区域，两者构成前向角楔，并可处理 359°与 1°之间的角度接缝。每次正常示向增加两个线性约束，故可行域随观测增加而保持不变或继续缩小。

在问题一中我们用原始示向角楔的交集定义 P，并据此判断区域是否无界。在线策略进一步将 P 与题设源分布圆域的保守外包相交。

区域直径与一次清除判据：

$$D(P) = sup_{x,y \in P} \|x - y\|, \quad r^* = min_c max_{x \in P} \|x - c\| \tag{4}$$

D(P)表示候选区域内两点之间可能达到的最大距离；$r*$表示覆盖整个候选区域所需的最小圆半径。只有当 $r* \leq 20$ 米时，机器狗才能在同一位置保证清除干扰源。对于给定清除点 c，还需验证 P 中的所有点均位于其 20 米作用范围内。

$$P \subseteq B(c, 20) \Leftrightarrow max_{x \in P} \|x - c\| \leq 20 \tag{5}$$

其中，B(c,20)表示以 c 为圆心、20 米为半径的闭圆盘。

对于非空有界凸多边形以及点、线段等退化情形，只需检查各顶点到 c 的距离。程序在 $r* \leq 19.99$ 米时执行保证清除；否则可尝试启发式清除并在失败后继续定位。

5.1.3 模型求解

具体求解过程如下。根据 n 个测点及其示向构造 m=2n 个半平面，再计算区域状态、边界顶点、直径、最远点对和最小包围圆。全部观测均参与可行域计算。

步骤 1，对 $Ax \leq b$ 进行线性可行性检验。若约束不可行，则返回空集并记录对应的原始观测，以便识别观测矛盾。

步骤 2，分别最大化 x、$-x$、y 和 $-y$。若任一目标无界，则将区域判定为无界并令直径为无穷；四个目标均有界时，再进行有限多边形计算。

步骤 3，枚举非平行边界直线的交点，并用全部 m 条不等式检验每个交点。去重

后构造凸包，同时区分点、线段和非退化多边形。

步骤 4，枚举凸包顶点 $v_1,\ldots,v_k$的全部点对并取最大距离。单点的直径为 0，线段的直径为两端点距离。

$$D(P) = max_{1\le i,j\le k}\|v_i - v_j\| \tag{6}$$

该步骤可由凸性严格说明。任取 $x = \sum\alpha_i v_i$ 、$y = \sum\beta_j v_j$，其中系数非负且各自和为 1，得到 $x - y = \sum\alpha_i\beta_j(v_i - v_j)$。由三角不等式，任意内点对距离不超过最大顶点对距离；最远顶点对本身属于 P，因此最大值可以达到。

$$\|x - y\| \le \sum_{i,j}\alpha_i\beta_j\|v_i - v_j\| \le max_{i,j}\|v_i - v_j\| \tag{7}$$

步骤 5，利用最小包围圆至多由三个支持点确定的性质，依次检查单点圆、两点直径圆和非共线三点外接圆。在能够覆盖全部顶点的候选圆中，选择半径最小者。本题单源观测数量较少，因此直接枚举即可满足计算要求。

设半平面约束数为 m，枚举交点并检查全部约束的时间复杂度为 $O(m^3)$，线性规划的判空和有界性检验另行计算。当凸包含 k 个顶点时直径计算的复杂度为 $O(k^2)$，最小包围圆的直接枚举复杂度为 $O(k^4)$。因单源观测规模较小，该方法能够满足本题要求。

5.1.4 模型分析与检验

为直观展示多次观测对候选区域的收缩作用，我们绘制了图 1。

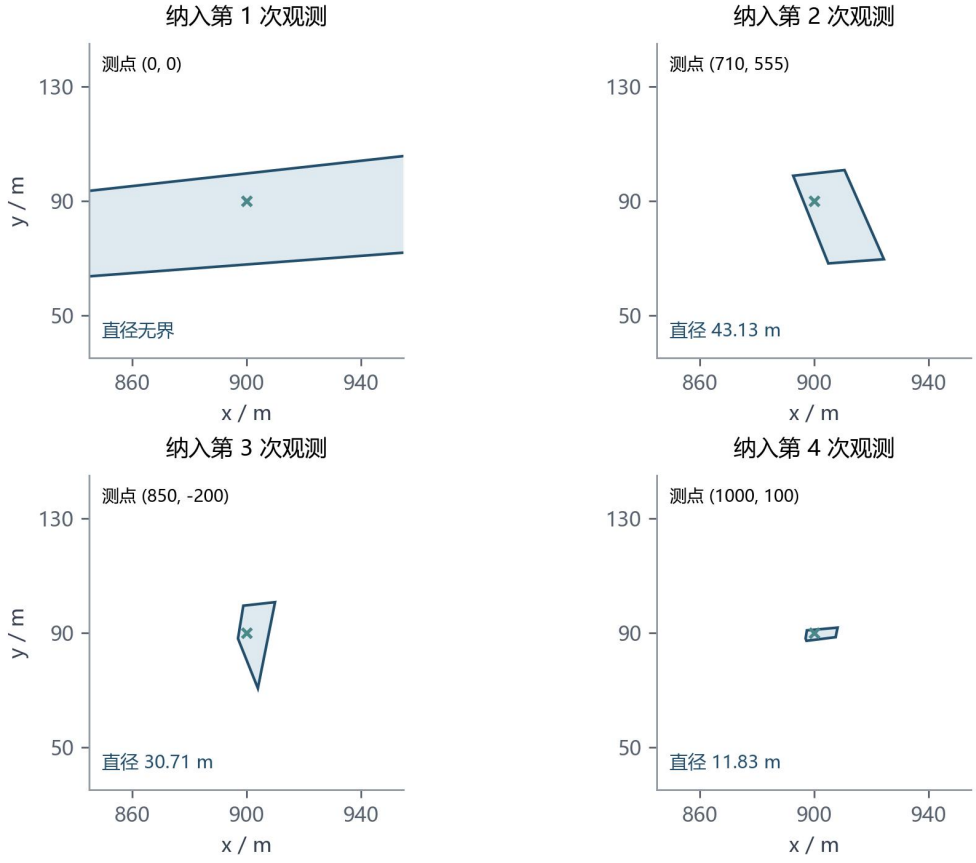


图 1 多次带误差示向逐步缩小可行域的几何示意

从图 1 可以看出，第一次示向只形成无界角楔；加入后续示向后，可行域逐步收缩并最终变为有界区域。区域直径随观测增加而单调不增，但不同测点带来的收缩程度并不相同。交会方向差异较大的测点通常能够更有效地削减长轴方向的不确定性。

四个测点加入后的累积交会结果见表 2。

表 2 四个检测点的累积交会结果 工作误差为 1.005°

累积观测数	新增测站 / m	示向 / °	直径 / m	包围圆半径 / m
1	**(0.0, 0.0)**	**5.31**	**无界**	**无有限包围圆**
2	**(710.0, 555.0)**	**292.83**	**43.1323**	**21.5662**
3	**(850.0, -200.0)**	**79.72**	**30.7107**	**15.3870**
4	**(1000.0, 100.0)**	**186.11**	**11.8315**	**5.9170**

表 2 所示观测可由位置为 (900,90)、接收半径为 1000 米的干扰源产生，四次示向误差均满足题设要求。第三次观测后，最小包围圆半径约为 15.39 米，已经达到可靠清除条件。第四次观测用于检验多次约束的累积处理；在线执行可在第三次观测后进入清除。

为分析区域直径与圆覆盖半径的关系，我们构造了表 3 所示的三次合法示向，理论误差均为$\pm 1°$。六个半平面的交集恰为顶点 $(0,0)$、$(40,0)$ 和 $(20,20\sqrt{3})$ 构成的等边三角形。

表 3 可产生等边三角形交会区域的三次示向

检测点	坐标 / m	示向 / °
s_1	$(-1000, 0)$	1
s_2	$(540, -500\sqrt{3})$	121
s_3	$(520, 520\sqrt{3})$	241

$$D(P) = 40, \quad r^* = \frac{40}{\sqrt{3}} \approx 23.0940 > \frac{D(P)}{2} = 20 \tag{8}$$

等边三角形反例的几何关系和最小包围圆如图 2 所示。

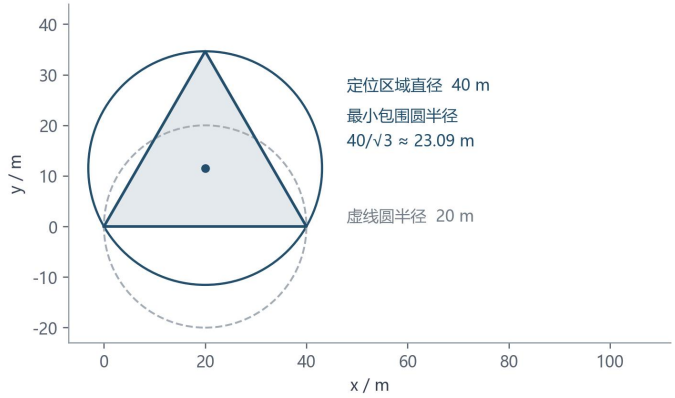


图 2 三测站可实现的反例及最小包围圆

由图 2 可知，该三角形的直径为 40 米，但最小包围圆半径约为 23.09 米。因此，不存在直径为 D(P)的圆覆盖该区域，第一问的相关命题不成立。对于线段，端点直径圆能够完成覆盖；对于一般多边形，则需要实际计算最小包围圆。

因此，后续在线策略以最小包围圆半径不超过 20 米作为可靠清除条件。

5.2 问题二 鲁棒接收域与第二测点选择

5.2.1 模型准备

将首次检测点平移到原点、示向中线旋转为局部 x 轴，令 $e(\varphi) = (\cos\varphi, \sin\varphi)$，可能源为 $G = r\,e(\varphi)$。正常示向给出 $5 < r \le 1500$、$|\varphi| \le \delta$。若同源接收半径记为 R，首测成功意味着 $R \ge r$，结合题设得到 $R \ge \max(1000, r)$。

文献[2]基于夹角交会研究有界测向误差下的静态测点选择。本文利用首次成功接收蕴含的距离约束 r≤R，并结合 1000≤R≤1500，推导全向干扰源第二测点的鲁棒接收安全域；随后分别在理论误差 1°与工作误差 1.005°下比较候选测点的定位精度和移动时间。

5.2.2 模型建立

利用首测接收隐含的信息：

$$K = \{r\,e(\varphi) : 5 < r \le 1500,\ |\varphi| \le \delta\} \tag{9}$$

$$q \in \Omega \iff \|q - r\,e(\varphi)\| \le max(1000, r),\quad \forall r\,e(\varphi) \in K \tag{10}$$

集合Ω要求第二测点对所有与首次观测相容的干扰源均保持可接收。首次成功接收说明 $R \ge \max(1000, r)$，因此该条件比统一取 $R = 1000$ 米更充分。**命题 1（四圆盘等价安全域）。以式（9）的 K 为全部位置约束，在全向接收且 0≤δ<90°时，式（10）的鲁棒接收安全域等价于：**

$$\Omega = \bigcap_{\sigma\in\{-1,1\}}[\,B(5e(\sigma\delta), 1000) \cap B(1000e(\sigma\delta), 1000)\,] \tag{11}$$

证明。B(c, a)表示以 c 为圆心、a 为半径的闭圆盘，σ =− 1 和 1 分别对应角楔的两条边界。取 φ =± δ、$r = 1000$ 可得到两个远端圆盘；令正常示向距离 r 趋近于 5 米，可得到两个近端圆盘。四个圆盘共同给出安全域的必要条件。

再证充分性。记 $q = (a, b)$，远端两个圆盘约束推出 $\|q\|^2 \le 2000\,q\cdot e(\pm\delta)$；两式相加可得 $a \ge 0$。

对$|\varphi| \le \delta < 90°$，有 $\cos\varphi \ge \cos\delta$、$|\sin\varphi| \le \sin\delta$，

故 $q\cdot e(\varphi) \ge a\cos\delta - |b|\sin\delta = \min\{q\cdot e(-\delta), q\cdot e(\delta)\}$。

于是距离平方关于角度的最坏情形在某个端点取得，只检验两条角边界即可。

$$\|q - r\,e(\varphi)\|^2 = \|q\|^2 - 2r\,q\cdot e(\varphi) + r^2 \tag{12}$$

当 $5 \le r \le 1000$ 时，上式是关于 r 的凸函数；在闭区间上的最大值位于两端，所以近端与远端圆盘控制所有中间距离。当 $r \ge 1000$ 时，需要距离不超过 r，将两边平

方后消去 r^2，只需满足下式。由于 $q\cdot e(\varphi)\geq 0$，右边随 r 增大而不减，$r=1000$ 的条件便覆盖所有更远源。至此得到四圆盘表述与连续鲁棒接收约束的等价性。

$$\|q\|^2 \leq 2r\, q\cdot e(\varphi), \quad r \geq 1000 \tag{13}$$

根据四圆盘约束，可得第二测点的安全域及三类候选点，如图 3 所示。

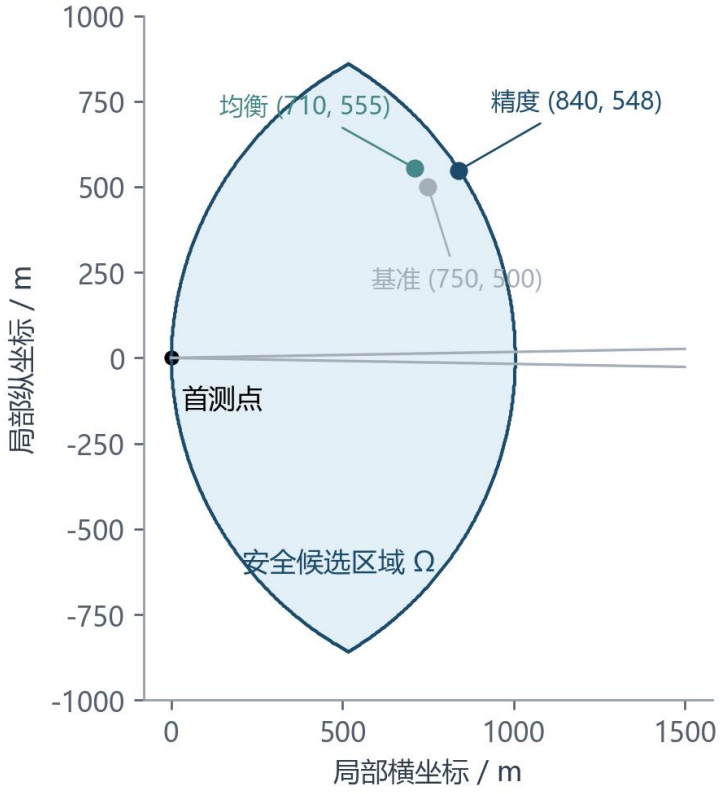


图 3 四圆盘安全域及基准 均衡 精度三类候选点；浅蓝区域为四圆盘的公共部分

在接收约束内权衡定位与移动

$$D(q) = sup_{\eta} diam(K \cap W(q,\eta,\delta)), \quad t_{move}(q) = \frac{\|q\|}{5} \tag{14}$$

其中，η为第二测点可能得到的示向角，D(q)表示测点 q 对应的最坏剩余直径，t_move(q)表示移动到 q 所需的时间。上确界只考虑 $K\cap W(q,\eta,\delta)$非空的正常示向。我们分别报告定位直径和移动时间：均衡点控制移动代价，精度点则通过增加路程进一步缩小候选区域。

局部坐标中的测点 q 通过式(15)转换到全局坐标，其中 $\mathrm{Rot}(\theta_0)$为二维旋转矩阵。两侧镜像点在局部几何和移动距离上等价，后续调度可再根据其他任务选择具体一侧。

$$q_{global} = s_0 + Rot(\theta_0)q \tag{15}$$

5.2.3 模型求解

为覆盖连续变化的第二示向角，我们先用凸多边形 P^+外包 K，再将全部可能示向

划分为闭区间 I_j。设区间中心为η_j、宽度为 h_j，在中心角处将误差半宽扩大 $h_j/2$，即可同时覆盖该区间内的所有角楔。

$$P_j^{+} \;=\; P^{+} \;\cap\; W\!\left(q,\eta_j,\delta+\frac{h_j}{2}\right) \tag{16}$$

$$K\cap W(q,\eta,\delta)\;\subseteq\; P_j^{+}\;\;(\eta\in I_j),\qquad D(q)\le U(q)=max_j diam(P_j^{+}) \tag{17}$$

我们用相邻切线的交点外包 1500 米圆弧，以保证外包区域包含全部合法位置。当工作误差为 1.005°且圆弧划分为 16 段时，最大径向外扩量不超过 0.000902 米。外包同时保留 5 米近距区域，因此所得直径仍是保守上界。三个固定候选均位于首测扇形外侧，可根据支撑方向确定连续示向范围。

$$\varepsilon_{arc} \;=\; 1500[sec(\delta/16)-1] \tag{18}$$

其中，ε_arc 表示圆弧外包产生的最大径向误差，$\sec x = 1/\cos x$。

为得到下界，我们在真实扇形内构造两个源位置 g_1和 g_2，使它们相对于第二测点 q 的方向差不超过 2δ。两点可以产生同一个第二示向，因此其距离构成最坏直径的可实现下界。程序只在真实圆弧端点和相反角边界射线上寻找下界见证。

$$L(q)=\|g_1-g_2\|\;\le\; D(q)\;\le\; U(q) \tag{19}$$

计算中取 $h_j\le 0.0005°$，并用 16 段外切线逼近圆弧。半平面向外扩展 10^{-7}米，最终上界再增加 10^{-5}米裕量。上下界之差用于衡量最坏直径的数值夹定精度。计算采用双精度，同时报告外包参数、下界见证和复算结果。

候选开发先在四圆盘域内沿给定移动半径搜索方向，以可行点对下界辅助搜点，再用连续上界独立筛选；最终取整数坐标并保留接收余量。路程受限时选(710,555)，最坏精度优先且能多支付约 20.31 秒移动时选(840,548)。

5.2.4 模型分析与检验

我们对三类候选测点的定位精度和移动时间进行比较，结果见表 4。

表 4 第二测点在工作误差 1.005° 下的连续界

候选	局部坐标 / m	最坏直径区间 / m	移动时间 / s
基准	(750.0, 500.0)	[130.3223, 130.3323]	180.2776
均衡	(710.0, 555.0)	[129.5087, 129.5173]	180.2360
精度	(840.0, 548.0)	[111.6704, 111.6773]	200.5895

表 4 显示，基准点(750,500)的最坏直径约为 130.33 米，精度点(840,548)的上界为 111.6773 米，低于基准点的下界。在相同工作误差下，精度点的保守改善超过 14.30%，但移动距离由 901.39 米增加到 1002.95 米，相应增加约 20.31 秒移动时间。

均衡点(710,555)的移动距离约为 901.18 米，与基准点接近，适合优先控制路程的情形；精度点则适合优先降低最坏定位直径。图 4 直观展示了三类测点之间的精度与时间权衡。

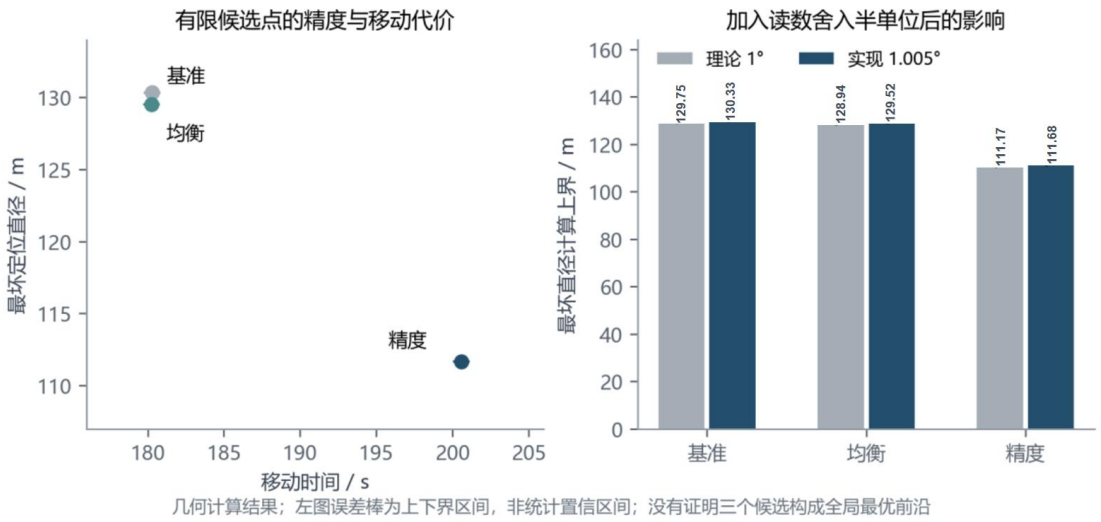


图 4 第二测点的移动时间与最坏直径权衡

在题设理论误差 1°下，三类测点的最坏直径上界依次为 129.7497 米、128.9375 米和 111.1694 米，排序与工作误差保持一致。如表 4 中三个候选测点在两次示向后的最坏定位直径均超过 100 米，尚不能保证一次可靠清除。

5.3 问题三 连续覆盖下的费用驱动定位与清除

5.3.1 模型准备 状态与完整计费

我们将完整任务分为未知频道搜索、已知源定位、可靠清除和无遗漏确认四个阶段，整体流程如图 5 所示。

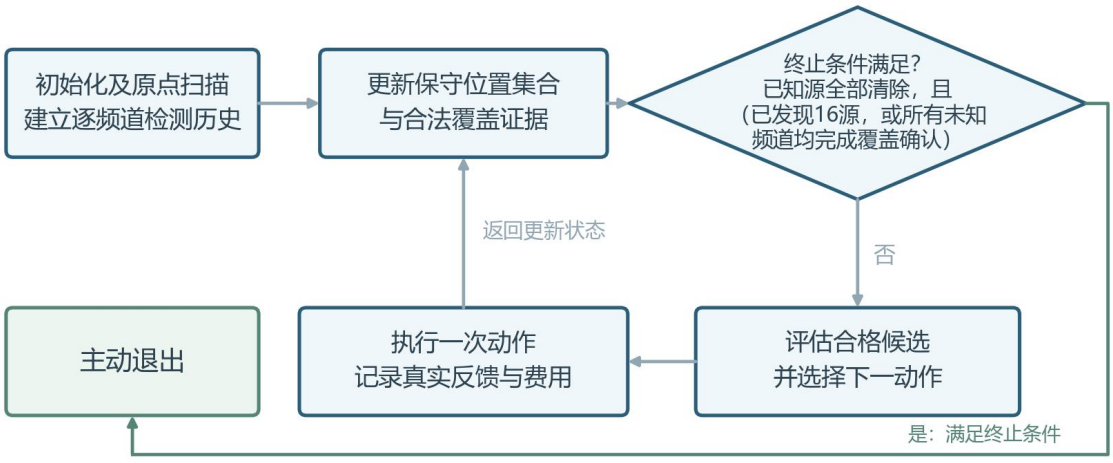


图 5 单次动作反馈循环与终止判断

算法以机器狗的当前位置、当前频道和逐次接口反馈为输入。我们为每个频道保存实际检测位置、正负响应、候选多边形 P、最小包围圆和清除状态，同时记录尚未访问的搜索站。以实际反馈更新位置集合，以预测费用选择后续动作。算法最终输出动作序列、时间账本和完整退出依据。

以本场实际总虚拟时间 T 除以成功清除数 S 作为评价值。检测和清除均在停止移动后串行执行，移动速度为 5 米/秒；M、K、C 分别表示实际检测、实际切频和全部光学尝试次数，C 包含失败尝试。光学成功另付 2 秒激光费用，光学操作不改变测向机当前频道。

$$T = \frac{L}{5} + 5M + K + 3C + 2S, \quad t = \frac{T}{S} \tag{20}$$

式中五项依次为移动、检测、切频、光学尝试和激光清除时间。

总任务时间累计至完整退出，带帽符号表示候选动作的预测费用。在线输入为机器狗状态与接口反馈，合成场景真值用于事后核验。

5.3.2 模型建立 连续发现与费用选点

(1) 七点发现覆盖。取原点及半径 1125 米圆周上六个等角测点，形成全向搜索骨架。因为 $R \geq 1000$ 米，只要每个未知频道在这七点完成实际检测，就能在连续圆域内发现其可能存在的源。

$$Q_{3} = \{(0,0)\} \cup \{1125e(60°k) : k = 0, \ldots, 5\} \tag{21}$$

$\rho \leq 900$ 米时到原点距离不超过 900 米；$900 \leq \rho \leq 1800$ 米时，总有环点与源的极角差不超过 30°。下式右侧关于ρ为凸函数，最大值在两端取得，且 1800 米端较大，从而给出整个圆域的严格上界。

$$d^2 \leq \rho^2 + 1125^2 - 1125\rho\sqrt{3}, \quad d \leq 999.1107 < 1000 \tag{22}$$

其中，ρ表示干扰源到原点的距离，d 表示该源到最近环形测点的距离。

原点在该布局中承担实际扫描任务。若只保留外圈六点，圆心附近会出现半径 125 米的覆盖缺口。对于后续产生的替代测点，我们重新进行连续覆盖认证，并仅将完成实际检测的位置计入相应频道的测点集合。

三种单环布局及其覆盖关系如图 6 所示。

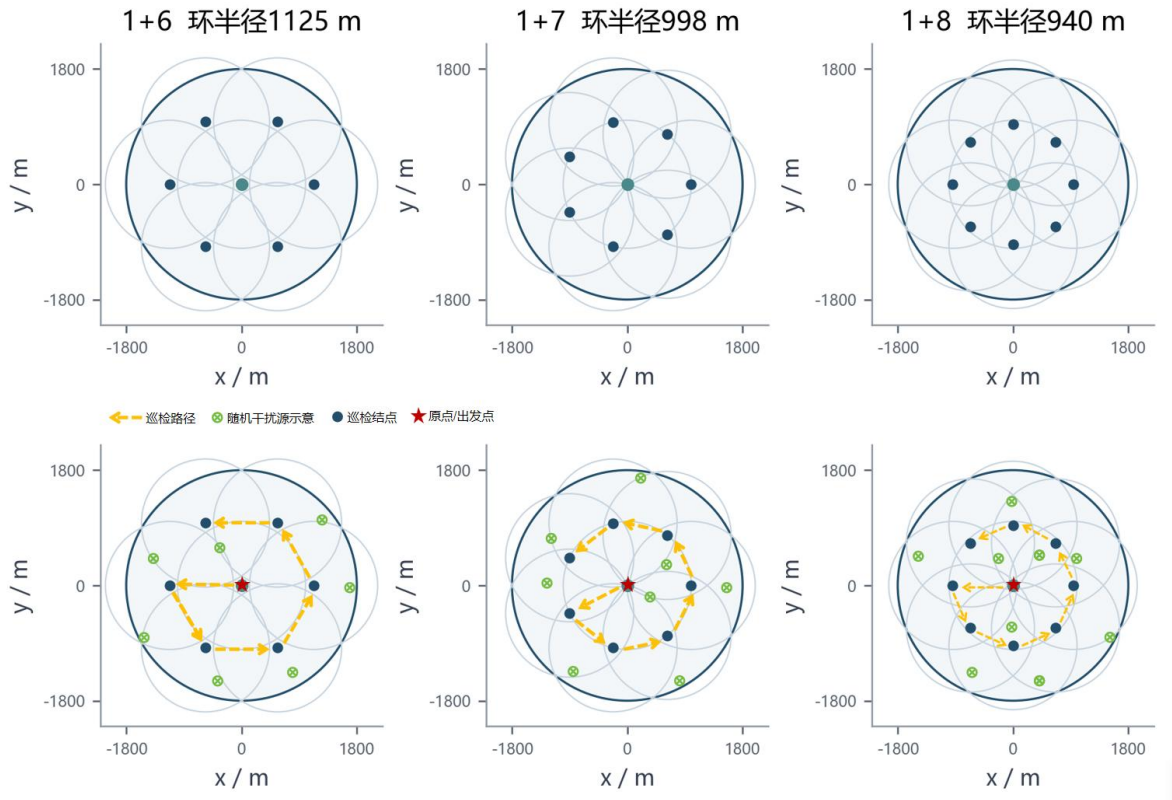


图 6 Q3 三种单环巡检布局 原点初扫加环点补全 薄圆为 1000 米接收范围；下排为固定巡检路径示意，随机源仅作示意

为检验“增加环点能否以更短的巡检路线抵消额外检测费用”，我们将 1+7 和 1+8 布局作为 1+6 布局的邻近对照。设外圈包含 k 个等角测点、环半径为 r_k，则任意干扰源与最近环点的夹角不超过 π/k。对最不利的边界源 $\rho = 1800$，连续覆盖要求（ π /k 以弧度计）：

$$1800^2 + r_k^2 - 3600 r_k \cos\frac{\pi}{k} \le 1000^2 \tag{23}$$

当 k=1 时，源圆边界上与环点方向相反的位置距该点为 $1800 + r_1 > 1000$ 米，无法覆盖。对 $k \ge 2$，式(23)有正半径解的必要条件为 $1000^2 - 1800^2 \sin^2(\pi/k) \ge 0$，因而 $k \ge 6$。故 1+6 是这类等角单环布局中环点数最少的可行方案，进一步比较 1+7、1+8 布局，检验增加测点后的整场时间是否下降。

(2) 全向阴性信息。设同源在 p^+检测成功、在 p^-返回无信号。接收半径在整场中固定，因而真实源 x 满足$\|x - p^+\| \le R < \|x - p^-\|$，平方展开得到一个保守闭半平面。与此同时，阴性检测排除以 p^-为圆心的 1000 米闭圆盘。

$$2(p^- - p^+)\cdot x \le \|p^-\|^2 - \|p^+\|^2 \tag{24}$$

数值实现中，我们将阴性圆盘半径取为 999.9 米，并由多边形与圆周交点及保留顶点构造外凸包，再向外增加微小余量。该处理允许保留少量已排除区域，从而保证合法位置不会因数值误差被删除。由于定向源的无信号还可能由背向造成，该距离排除只用于第三问。

(3) 在可靠候选内按费用排序。以首测示向为纵轴，在多边形投影范围的 25%、40%、55%、70%、85%处取前向位置，侧向距离取 30、60、100、180 米，并保留原候选。每个双侧候选均先验证到 P 全部顶点的最远距离不超过 999.9999 米；未通过者不进入评分。由凸性，该顶点检验保证 P 中全部位置满足接收距离约束。

同时扩大侧向距离，使两探针夹住远侧多边形通向首测点的全部射线。第三问两点均保证接收，可先测较近点；第四问利用此构造推导双阴性约束。

我们用面积求积点 x_i表示 P 中的候选位置。其权重由面积权重与符合正负观测历史的接收半径区间长度共同确定，归一化后记为 w_i。当上述权重无效时，算法退回面积权重排序。根据 q 处指向 x_i的名义无误差示向，可得到预测包围圆 (c_i, r_i)。

$$\hat{C}_+(x_i,q) = \frac{max\{0,\|q-c_i\|-max(0,19.99-r_i)\}}{5} + 5 + 0.25max(0,r_i-19.99) \tag{25}$$

$$\hat{J}(q) = \frac{\|s-q\|}{5} + 5 + \sum_i w_i\hat{C}_+(x_i,q), \quad \sum_i w_i = 1 \tag{26}$$

其中，s 为当前位置。式 (25) 和式 (26) 依次计入当前移动、一次检测、后续移动、5 秒成功清除时间以及 0.25 秒/米的剩余半径惩罚。几何检验用于筛除不可靠测点，费用评分用于排列其余候选。实验费用按式（20）统计。

5.3.3 模型求解 滚动调度与有限清除

(1) 滚动安排任务。完成原点扫描后，我们将剩余搜索站和已知待清源共同加入开放路线，并利用近邻构造和换边改进[3]确定下一任务。到达搜索站后逐频道检测；处理已知源时，先判断当前位置是否适合补测，再进入局部定位和清除。完成一次局部服务后，算法返回全局层重新调度。

前置补测要求与旧测点至少相隔 60 米，且交会角与距离满足筛选条件。局部服务依次检查近距清除、整域可靠清除和继续定位；需要测向时，先验证安全双侧候选，再按费用选对并先测较近点，取得真实反馈后更新 P。

(2) 一次后验光学替代。当局部中心尚不满足可靠清除条件时，我们利用 512 个确定性面积求积点估计候选位置的后验质量。候选点包括原中心、加权几何中位点、加权均值、长轴分位点附近 18 米带内的条件均值以及当前位置。以 19.8 米为估计命中半径，得到命中概率 $h(q)$，并按下式排序：

$$\hat{J}_o(q) = \frac{\|s-q\|}{5} + 3 + 2h(q) + \sum_{\|q-x_i\|>19.8} w_i\left[\frac{\|q-c\|}{5} + 10 + \frac{\|c-x_i\|}{5}\right] \tag{27}$$

其中，c 为原包围圆中心。若在 q 处命中，则光学定位和激光清除共耗时 5 秒；若未命中，则继续估计返回 c 测量、再次移动并完成清除的时间。每个频道最多使用一次替代位置，且只有评分不高于原中心时才采用。失败后返回原中心继续定位，真实清除半径仍按题设的 20 米计算。

双侧测点和后验光学替代分别用于可靠定位和费用优化，两者的决策关系如图 7 所示，点大小示意权重。

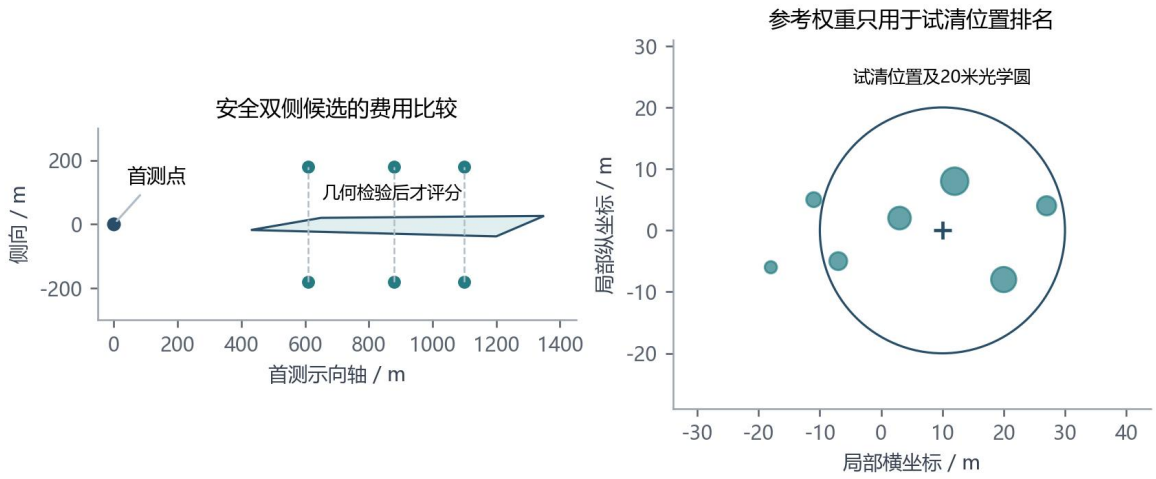


图 7 几何验证与费用排名分工：双侧探针选择及一次光学位置替代

(3) 有限后备与完整退出。若三轮局部试探后仍未清除，我们用 P 的旋转外接矩形生成光学覆盖网格，使每个网格单元的半对角线不超过 19.99 米，并逐点尝试。网格覆盖整个外接矩形，预计费用只用于选择行列划分和访问方向。

若局部集合运算异常或上述覆盖仍不足，算法启用首测条带内的 153 个试清点组成的最终后备。首个正常示向将纵向范围限制在 [0,1500] 米，横向绝对值不超过 26.31 米。纵向按 30 米间隔取点，横向取−20、0 和 20 米，使任意候选位置到最近网格点的距离不超过 18.03 米。因而在反馈合法且执行资源充足时，该有限后备能够完成清除；几何矛盾和外部中断则单独记录为异常。

第三问还允许机器狗在已清除源的位置扫描全部未知频道，并在连续覆盖认证通过后删除一个未来搜索站。每次删去尚未访问的测站前，均检验已扫描位置与其余计划测站是否仍能共同覆盖整个源域；检验通过后，才取消该站的访问。搜索完成时，再按各未知频道的实际检测记录确认覆盖。发现 16 个不同频道后可停止未知搜索；若源数不足 16，则需为每个未知频道保留连续覆盖证据。清除、搜索站和补测次数均有有限上界，因此算法能够持续推进。

5.3.4 模型分析

问题三最终采用局部费用策略，按预计动作耗时选择测点和试清位置。该策略在普通场景中的平均收益略低，但能明显控制聚集场景的退步。选择性信息策略用于考察更积极的信息规划。

实验分为开发组和独立确认组，每组包含 84 个场景，源数 10 至 16 各 12 场。三种策略使用相同的源位置、频道、半径和固定位置误差。我们先计算每场总时间与清除数之比，再对各场等权平均。开发组用于选择策略，参数固定后再在新的确认组上运行；压力场景单独统计。问题四采用相同口径。

固定采用配置后，复用确认组 84 个场景，对三种预设布局进行 252 次配对运行，以分析环点数量与整场费用的关系。

比较原点加 k 个等角环点的三个预设布局。按前述距离凸性将最大夹角换为 $180°/k$ 并调整环半径 r_k，保证连续覆盖。固定点集首段至少 r_k，其余段至少为相邻弦长，顺次访问达到以下最短开放路长；式中三角函数以弧度计，路长不含定位清除绕行。

$$L_k = r_k + 2(k-1)r_k \sin(\frac{\pi}{k}) \tag{28}$$

增加环点可以缩小环半径和巡检路长，但会增加逐频道检测次数，比较时只改变环点数量和半径，其余策略机制保持一致。三种布局共运行 252 次，均完成全部清除，每种布局均清除 1092 个干扰源。三种固定巡检布局的同场比较结果见表 5。

表 5 局部费用策略在同一 84 场中的巡检布局比较

布局	环半径 / m	巡检路长 / km	整场均费 / s/源	P90 / s/源	较 1+6 变慢
1+6	**1125**	**6.75**	**240.46**	**280.59**	**0/84**
1+7	998	6.19	242.83	287.83	53/84
1+8	940	5.98	247.33	296.72	62/84

由表 5 可知，1+7 和 1+8 布局相对 1+6 布局分别节省 1.98 秒/源和 2.65 秒/源的移动时间，但检测与切频时间分别增加 4.37 秒/源和 9.41 秒/源。增加的操作时间抵消了路程收益，最大单场退步分别达到 25.76%和 14.39%。这一补充结果支持保留原冻结的 1+6 布局：多设环点虽能缩短路程，额外检测却使整场时间增加。

表 6 比较本文构建的三种策略。它们共用七点发现骨架、正负反馈几何、联合调度及可靠清除条件。共同基线已含双侧定位、前置补测和经连续认证的替站；局部费用策略按预计动作耗时选择双侧测点，并至多进行一次后验光学位置替代。选择性信息策略进一步强化当前任务的路线排序、筛选已知频道补测，并在单站扫描中发现 16 源后跳过剩余未知频道。

表 6 Q3 开发对照的同场比较

策略	清除/总数	均值	P90	上尾均值	样本最大
共同基线	1092/1092	239.41	289.94	307.30	335.45
局部费用策略	**1092/1092**	**236.02**	**284.03**	**302.50**	**331.47**
选择性信息策略	1092/1092	232.82	279.59	296.08	307.37

单位：秒/源；每组 84 场。P90 为线性插值分位数，上尾均值取费用最高的 9 场；加粗行为该问采用方案。

表 6 显示，局部费用策略较共同基线平均节省 3.39 秒/源，降幅为 1.42%。84 个场景中有 16 场变慢，最大相对退步为 1.75%。选择性信息策略虽然进一步降低了普通场景均值，但在聚集反例中比基线慢 115.53%；局部费用策略在压力组中的最大退步约为 5.15%。综合平均表现和困难场景稳定性，我们选择局部费用策略。均值只能反映总体水平，因此我们进一步进行逐场配对，结果如图 8 所示。

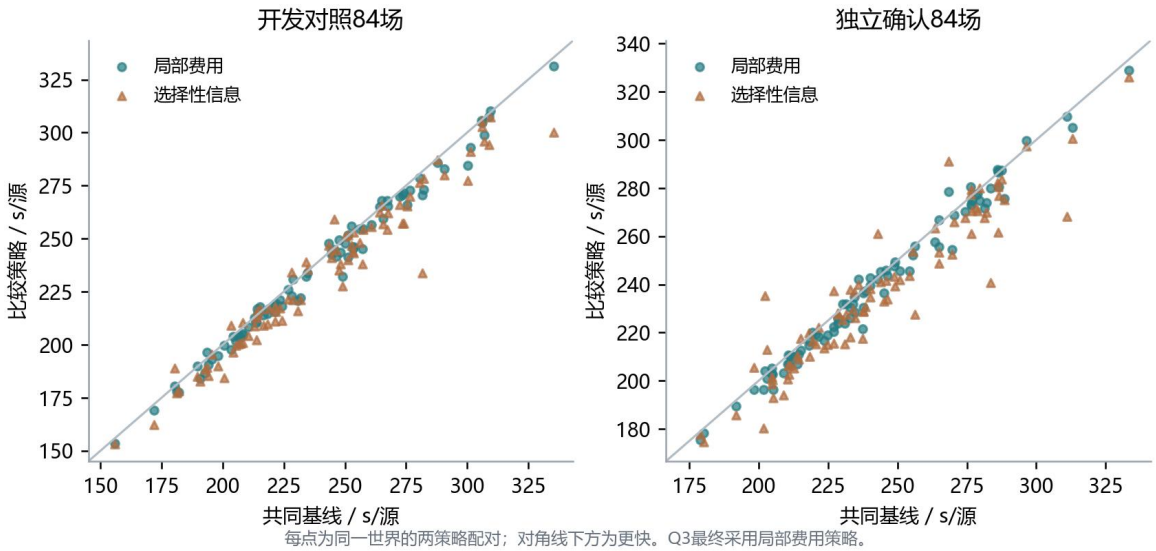


图 8 Q3 开发与确认组逐场费用配对 各 84 场 对角线下方为改善

为分析策略在不同困难情景下的表现，我们按场景机制进行分组，结果见表 7。

表 7 Q3 按困难机制分组的逐场等权均值

机制	场数	共同基线	局部费用	选择性信息
圆周边界	14	302.25	303.11	304.02
空间聚集	14	186.23	179.60	187.86
固定−1°	14	244.04	241.86	237.49
固定+1°	14	247.92	241.30	239.56
最大接收半径	14	229.45	227.39	223.61
最小接收半径	14	263.90	257.44	251.66

单位：秒/源；各困难机制分别统计。

图 9 左侧为各场相对变化，正值表示退步；右侧按式（20）分解聚集案例 m021-stress-q3-cluster-000027 的实际费用。

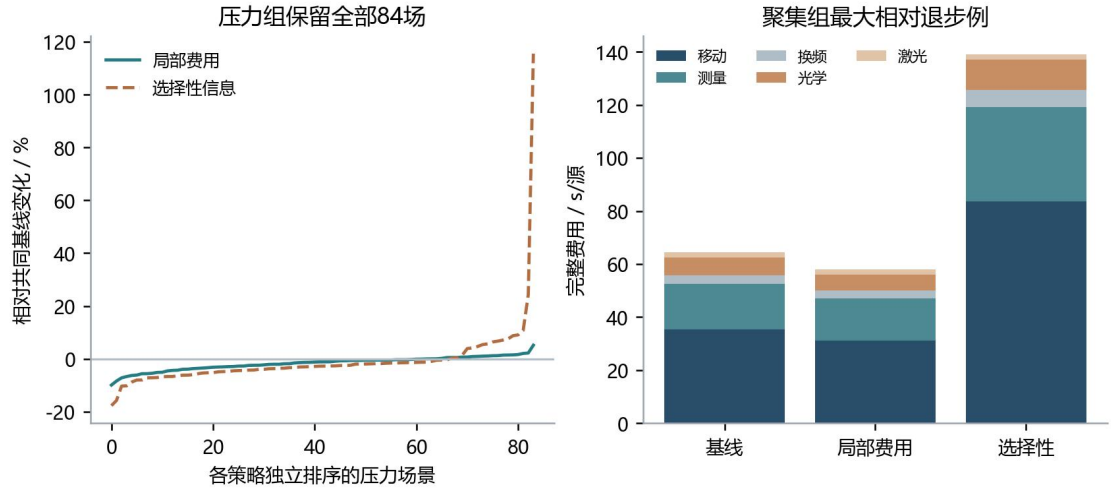


图 9 Q3 逐场相对变化及聚集反例的时间构成

图 9 中的聚集反例表明，较短的名义路线可能推迟最后一个未知频道的首次有效测量，从而延后数量上限停止条件的触发。该现象说明局部路线长度不能完整表示发现任务结束后所减少的工作量。

5.3.5 模型检验

固定策略在新确认组中全部清除。成功数、实际动作账本和完整退出条件同时核对。冻结参数后，采用策略在新确认组中的同场比较结果见表 8。

表 8 Q3 独立冻结确认的同场比较

策略	清除/总数	均值	P90	上尾均值	样本最大
共同基线	1092/1092	243.47	286.10	298.73	333.22
局部费用策略	**1092/1092**	**240.46**	**280.59**	**296.98**	**328.96**
选择性信息策略	1092/1092	237.09	278.84	291.14	325.88

单位：秒/源；统计口径同前，加粗行为采用方案。

由表 8 可知，采用方案在确认组中的平均定位清除时间为 240.46 秒/源，较共同基线降低 1.24%。84 场中 18 场有退步，最大退步为 3.73%。

为识别各项机制的贡献，我们在固定场景中分别关闭单项功能，结果见表 9。

表 9 Q3 固定场景单项关闭对照

配置	场数	均值 / (s/源)	费用差额 / (s/源)
完整策略	28	237.38	+0.00
关闭后验光学选点	28	238.91	+1.53
关闭费用探针对选	28	239.25	+1.87

使用确认组每个源数层前 4 例，共 28 场；只关闭指定机制，其余输入与参数保持。差额为关闭版本减完整策略，表示其余机制启用时，单独关闭相应机制产生的边际变化；正值表示该机制在本子集降低平均费用。

表 9 显示，关闭后验光学选点和费用探针对选后，平均时间分别增加 1.53 秒/源和 1.87 秒/源，且分别有 17 场和 20 场慢于完整策略。两类机制分别改善清除位置和测向位置的选择。

连续解析上界验证七点发现；动态替点、全向阴性收缩及可靠清除按每步真实历史另行核验，保持发现与清除的几何条件。

问题三共完成六次官方演练，API 记录显示成功清除 86 个干扰源。按每场平均定位清除时间等权计算，结果为 223.45 秒/源；按总时间除以总清除数计算，结果为 221.89 秒/源。本文采用前一种统计结果作为主指标。

问题三的六次官方演练结果见表 10。

表 10 Q3 六次官方演练的归档结果

案例编码	清除数	平均时间 / s/源	运行时间 / s	清除比例
78JR-RBZF-XBRR-DPXX	16	198.59	2.27	100%
89Y6-5Q9J-3AXY-472D	13	214.99	1.60	100%
SQDS-KFK9-TX88-D263	15	211.06	1.60	100%
2ZPG-KPEB-9RDV-86WU	15	244.32	2.25	100%
SAGQ-TC3E-NQJP-DAFQ	12	265.45	2.16	100%
QE4D-F7CJ-2KGB-HAXX	15	206.29	2.38	100%

程序运行时间取官方响应中成功 enter 至成功 exit 的实时时间戳差，不含本地启动准备和退出后的打包耗时。

成功清除数与费用由 API 响应重算，清除比例依据模拟器结果页核对。

问题三的正式测试结果按表 11 所示口径记录。

表 11 问题三正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
RKMV-HG3J-9ZHA-WYCT	14	223.79	2.21
2JVQ-KT9B-D8Z7-4TMP	14	229.40	1.64
QWGB-TSJX-EJFR-2TGH	12	224.33	1.70

采用算法：局部费用策略

5.4 问题四 定向覆盖与途中信息决策

5.4.1 模型准备 定向失联与信息决策

问题四同时包含全向源和定向源。定向源只在发射方向两侧各 90°的闭半平面内辐射，并要求测点与源的距离不超过 R。因此，无信号既可能表示距离过远，也可能表示测点位于背面。我们保留第三问的逐频道状态和完整计时方法，并重新构造适用于任意发射方向的发现覆盖。

本问沿用第三问的费用定位与有限后备，增加已知源补测决策：每轮检查原地信息，选择搜索或清除任务，再考虑途中停车；取得真实反馈后重排。

5.4.2 模型建立 发现保障与代价边界

(1) 局部凸包覆盖

对候选源 x，令 $H_j(x)$为频道 j 已检测且距 x 不超过 1000 米的站点。要对全部未知发射方向保证至少一次接收，等价于 x 位于这些近邻站点的凸包中。

$$H_j(x) = \{s \in H_j : \|s - x\| \le 1000\}, \quad x \in conv\, H_j(x) \tag{29}$$

其中，conv 表示点集的凸包。

该条件的充分性可由凸组合证明。若全部近邻测点都位于某一朝向的背面，则它们相对于 x 的投影均为负，其凸组合不可能得到零向量。由分离直线定理得到必要性：当 x 位于近邻凸包外时，可以选择一个方向使所有近邻测点均落在背面，并令 $R = 1000$ 米排除其余远点。理论允许 x 位于凸包边界，数值认证保留严格内含余量。

距离覆盖与未知朝向下的定向接收条件如图 10 所示，x 为源位置，圆为最小接收距离，三角形为邻近搜索点凸包。

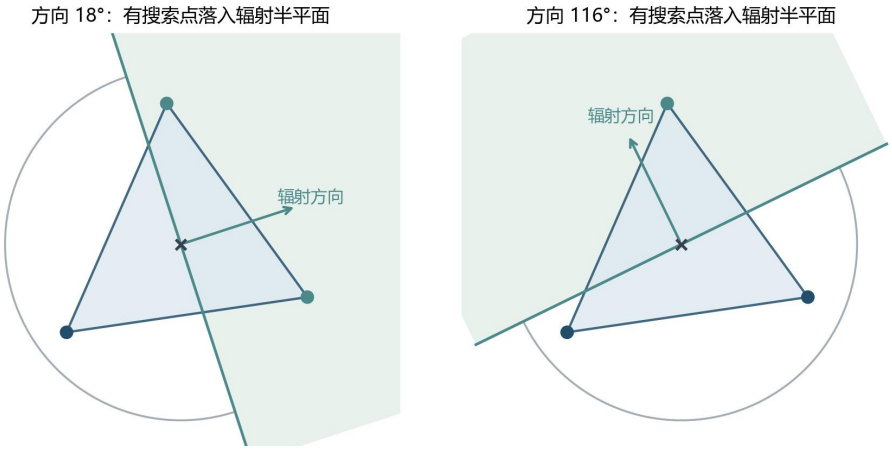


图 10 近邻凸包与未知朝向：距离覆盖不能替代定向接收保障

构造先从边长 900 米的三角格出发，将一个基本三角形的重心置于圆心；保留与源圆域相交的全部三角形顶点，得到 27 点。任一源位于相应三角形内，到三顶点均不超过 900 米，因此满足近邻凸包条件。该构造容易证明。中心附近存在较多重复覆盖；同时，部分边界测点伸出场地较远，使巡检路线偏长。因此，不能只在原三角格中简单删点，而需要重新设计测点之间的几何支撑关系。

根据式（29）的近邻凸包条件，利用圆域的对称性，我们进一步构造分层圆环布局。首先将测点按功能分层：原点和内环负责中心及中间区域，外环从场地外侧包围边界，使朝外发射的定向源仍能被多个近邻测点夹持。随后，我们枚举内外环点数、半径和相位差，并对每组参数进行连续区域递归认证。最终得到“原点+8 个内环点+12 个外环点”的 21 点布局。两环采用交错相位，既减少中心区域的冗余测点，又保留边界区域所需的凸包支撑。与 27 点三角格相比，该方案减少 6 个测点，开放巡检路线由 23.92 千米缩短至 17.85 千米，降幅为 25.38%。

两环半径分别为 999 米和 1867 米，外环相位为 15°，坐标写为：

$$Q_4 = \{(0,0)\} \cup \{999e(45°k)\}_{k=0}^{7} \cup \{1867e(15° + 30°k)\}_{k=0}^{11} \tag{30}$$

为验证布局对连续区域的覆盖，我们将包围源域的方形递归四分。对于每个叶格，先找出到四个角点的距离均不超过 999.99 米的测点，再检验四个角点是否严格位于这些测点的同一凸包内。距离平方和凸包的凸性保证该结论覆盖整个叶格。无法直接认证的网格继续细分，整数毫米坐标和支撑边余量用于控制舍入误差。

21 点布局共生成 6485 个递归节点和 4864 个叶格，独立检查程序逐格验证分区、距离和凸包条件，全部通过。实际执行时，我们还需根据每个频道的真实检测记录确认覆盖完成。

我们还用独立的圆周判据复核同一 21 点布局：除源域边界外，逐一检查各测站接收圆周在源域内的部分，并在该圆周的凸包判断中删去其圆心测站。这样可以同时检查外边界与接收范围切换处的内部漏区。采用有理参数化得到 720 个闭区间叶，全部通过独立核验；判据证明和证书见问题四_圆周覆盖证书核验.py 及数据/。

两种固定布局及其巡检路线的对比如图 11 所示。

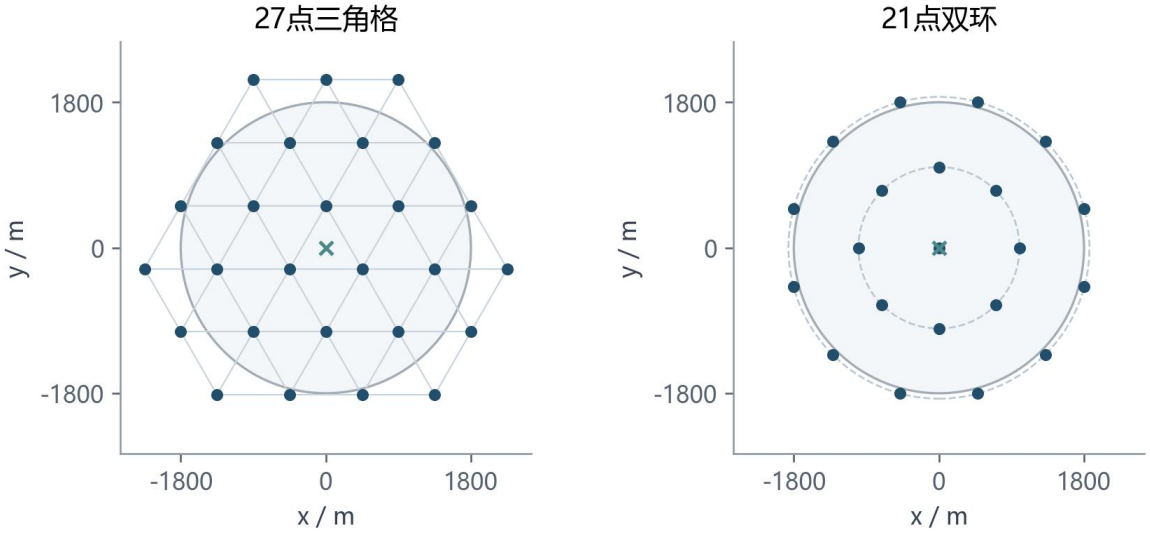


图 11 Q4 由 27 点三角格到 21 点双环构造 两图同尺度 原点为行走起点

图中开放巡检路线由确定性构造与换边改进得到。

(2) 命题 2：固定通用布局的 19—21 点界

结论为 19≤N*≤21。N ∗表示对源圆域内所有位置、任意发射方向及 1000 至 1500 米接收半径都保证发现的固定布局最少点数，测点允许在场地外。21 点构造给出上界；下界由总加权需求至少 18.141 与单站贡献不超过 1 推出。

证明。取最小接收半径 1000 米。记 $b_t(\rho)$为距圆心ρ的测点在半径 t 源圆上覆盖的半弧角，角度以弧度计。先判断ρ+t≤1000 时取π，否则在|ρ−t|≥1000 时取 0；其余为部分相交，由余弦定理得

$$b_t(\rho) = \arccos\left(\frac{\rho^2 + t^2 - 1000^2}{2t\rho}\right) \tag{31}$$

再记 $e_t(\rho)$为该测点对径向朝外发射源的接收半弧角。$\rho > t$ 时，$e_t = \min\{\arccos(t/\rho), b_t\}$；$\rho \le t$ 时取 0。$\rho = 0$ 时，b_t按整圆是否可接收取π或 0。对任一正半径源圆，有限个例外位置之外至少需要三个接收近邻，否则源点只能落在有限条测点连线上。因此 $\Sigma b_t \ge 3\pi$；1800 米边界朝外还要求$\Sigma e_{1800} \ge \pi$。

把 50、200、1800 米源圆的三重接收要求与边界朝外要求加权相加，定义单点贡献 F。权重先由数值搜索寻找，再取成可精确核验的有理数；结论只依赖这些固定权重通过连续区间核验。

$$F(\rho) = \frac{85b_{50}(\rho)+393b_{200}(\rho)+5333b_{1800}(\rho)+708e_{1800}(\rho)}{1000\pi} \tag{32}$$

$$\sum_i F(\rho_i) \geq \frac{3(85+393+5333)+708}{1000} = 18.141 \tag{33}$$

关键是证明任意站位都有 $F \leq 1$。$t \leq 1000$ 时，b_t随ρ单调不增；$t>1000$ 时，仅有内部极大值$\rho = \sqrt{(t^2 - 1000^2)}$。$e_{1800}$仅在$\rho = \sqrt{(1800^2 + 1000^2)}$处取内部极大值。把[0,2800]米分成 560 个闭区间，在每段端点和可能包含的极大值处取有理上界，得到所有区间 $F < 1$；最紧上界约为 0.999907534486。$\rho > 2800$ 时各项均为 0。由式（33）的总量 18.141 和点数整数性，结合 21 点构造，有

$$19 \leq N* \leq 21, \quad 21 - N* \leq 2, \quad \frac{21}{N*} \leq \frac{21}{19} \tag{34}$$

在固定通用布局的站点数指标下，21 点构造至多比最少点数多 2 点，相对差不超过 2/19，约 10.53%。进一步可证：若 19 点布局存在，恰有一站距圆心小于 660 米，其余 18 站都在 800 米之外。该条件能缩小 19 点布局的搜索范围，一般 19、20 点是否可行仍未确定。18 点初步证明和内站条件推导见支撑材料“数据/问题四覆盖证明/”，完整证书与核验程序一并提供。

(3) 无遗漏确认的必要行走代价。

除测点数量外我们还根据缺失频道的不可区分观测历史推导无遗漏确认所需的行走距离下界。

考虑对题面全部合法物理场景保证安全的策略，在实际源数 $N < 16$ 的场景终止时，完整路径记为Γ。若某单位方向 u 满足 $\sup_p \in \Gamma p \cdot u < 1780$，则可在一个未出现频道的 1800u 处增加朝向 u 的定向源。新增源与所有原路径点均相距超过 20 米且位于射频背面，原动作反馈完全不变，而总数仍不超过 16。原退出因而遗漏新增源，矛盾。

$$conv\,\Gamma \supseteq B(0,1780), \quad N < 16 \tag{35}$$

由式 (35) 可知，路径从圆心出发且必须与半径 1780 米圆的每条切线相交。将 Joris 关于此类连续曲线的定理[4]作尺度变换，可得到式(36)所示的长度下界。

$$L \geq 1780\left(1 + \sqrt{3} + \frac{7\pi}{6}\right) \approx 11387.0912\,m, \quad N < 16 \tag{36}$$

式（36）给出 N<16 时完成无遗漏确认所必需的行走距离。

(4) 定向局部探针。继续使用第三问中通过 1000 米距离检验的双侧候选。以首测点为原点、示向为 x 轴，若真实源横坐标 $x \geq a$，则从源指向原接收点的射线穿过前向截面 x=a，两探针夹住交点。原点在可见半平面中，故两探针不可能同时严格位于背

面；又因两点均在接收距离内，至少一个可接收。因此，仅在两次同频道实际反馈均为阴性时，才能以 $x \le a$ 保守裁掉远侧。

探针返回示向或近距响应时，按相应观测约束更新位置集合。几何评分还对阴性分支计入移至第二探针、再检测与后续清除的费用。候选评分采用两类源各占 1/2、相容方向均匀及相容半径均匀的参考先验。

(5) 已知频道的信息价值。以当前包围圆中心 c 构造名义示向，计算预测收缩后的半径 r'，用 $r - r'$作为几何收益，再乘以方向可见性近似权重。方向按 5°离散，保留在名义中心处与已有正测点方向差均不超过 90°的朝向；其中也能看到新点 p 的方向所占比例记为 $a(p)$。

$$\hat{g}(p) = max(0, r - r') \cdot [0.5 + 0.5a(p)] \tag{37}$$

式 (37) 中的 r 和 r'分别表示补测前后的预测包围圆半径，$a(p)$表示新测点对相容方向的可见比例，因此$\hat{g}(p)$的单位为米。若测点间距不足 60 米、当前不确定半径小于 40 米，或 p 到名义中心的距离不在 25 至 1450 米之间，则令预测收益为 0。

5.4.3 模型求解 选择性扫描与途中测向

(1) 选择性扫描与原地信息。在搜索站，对仍未知频道照常实测；对已知待清频道，仅当满足原有几何与间距条件、且$\hat{g} \ge 30$ 米时追加检测。每轮安排主要任务前，对当前位置收益$\hat{g}$≥120 米的已知待清源按收益排序补测。发现 16 个不同源频道后，停止未知频道扫描并继续处理已知待清源。

(2) 途中停车。设当前实际位置为 s，计划下一个目标点为 z，线段长度为ℓ。只有ℓ≥150 米才考虑中途检测；候选比例α取 0.15、0.30、0.50、0.70、0.85 及源名义中心在线段上的投影比例（截到[0.1,0.9]），并要求距起点至少 60 米、距终点至少 30 米。

$$p(\alpha) = (1 - \alpha)s + \alpha z, \quad score(p) = \frac{\hat{g}(p)}{30 + \alpha\ell} \tag{38}$$

途中候选只服务于当前目标之外的已知待清源，并要求其不确定半径至少为 80 米、$\hat{g} \ge 120$ 米。每个频道最多进行两次途中测量，阴性反馈同样占用测量预算。停车后，算法根据真实反馈重新排序后续任务。式 (38)中的 30 米是信息收益的固定归一化尺度，$score(p)$只用于候选点排序。

途中测向点的生成和反馈更新过程如图 12 所示。

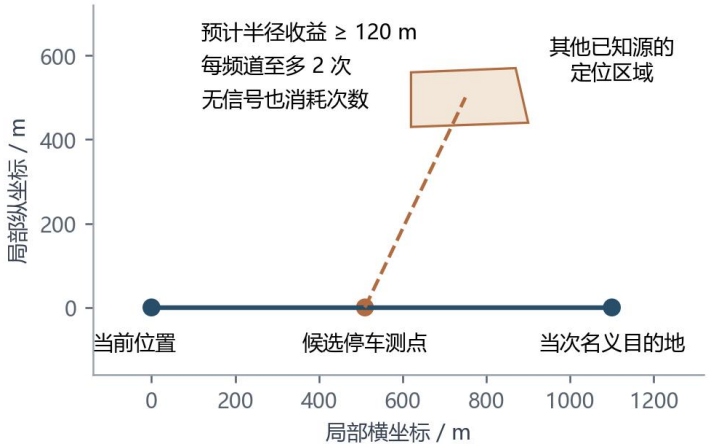


图 12 当前路段内的途中信息决策：停车前作预测，停车后仅用实际反馈更新

途中检测沿当前计划路段选点，取得观测后重排后续任务。

(3) 保留剩余路线与可靠执行。搜索站与已知源共同构成任务集合；尚有搜索站时，包围圆半径不超过 400 米的已知源可进入联合队列。保留上一轮尚有效的路线后缀，把新增任务插入，再与重新构造的路线比较名义长度并作换边改进；每轮仍只执行首项。

局部处理使用费用加权双侧探针和一次后验光学替代。在 Q4 光学失败后的预测评分中加入 $40[1-\mathrm{p}(\mathrm{c})]$秒的未接收风险惩罚，其中 $\mathrm{p}(\mathrm{c})$由上述参考先验计算。三轮局部试探后仍未清除时，先用外接矩形网格覆盖；必要时再启用最终试清网格。

固定 21 点骨架、每频道途中预算和原地补测间距限制保证有限任务推进。退出仍须已知源全清，并完成未知频道覆盖或已发现 16 源；有限后备的合法反馈与资源条件同第三问。

5.4.4 模型分析

三种策略在同时包含全向源和定向源的场景中配对，共用 21 点发现骨架、几何定位、联合调度和可靠清除条件。共同基线已在单站扫描结束后检查 16 源上限并停止后续搜索；局部费用策略增加费用选点、一次后验光学位置替代、剩余路线保留与原地补测。选择性信息策略进一步筛选已知频道补测、插入途中测向，并在单站扫描中达到 16 源上限后立即跳过剩余未知频道。

问题四三种策略在开发组中的同场比较结果见表 12。

表 12 Q4 开发对照的同场比较

策略	清除/总数	均值	P90	上尾均值	样本最大
共同基线	1092/1092	476.01	605.44	624.27	649.64
局部费用策略	1092/1092	466.71	584.44	604.87	623.17
选择性信息策略	**1092/1092**	**443.35**	**571.63**	**589.11**	**611.47**

单位：秒/源；统计口径同前，加粗行为采用方案。

由表 12 可知，采用策略较共同基线平均减少 32.67 秒/源，降幅为 6.86%，样本最大值由 649.64 秒/源降至 611.47 秒/源。84 个场景中有 10 场变慢，最大相对退步为 20.43%。

为比较策略在不同困难情景中的表现，我们进一步进行分组统计，结果见表 13。

表 13 Q4 按困难机制分组的逐场等权均值

机制	场数	共同基线	局部费用	选择性信息
圆周边界	14	549.22	514.02	468.04
空间聚集	14	427.63	420.14	422.39
固定−1°	14	487.57	478.42	457.80
固定+1°	14	487.15	479.87	453.57
最大接收半径	14	474.68	460.08	444.86
最小接收半径	14	532.28	495.87	471.27
朝外定向	14	548.88	532.37	483.59

单位：秒/源；各困难机制分别统计。

逐场变化和最不利案例的费用分解如图 13 所示。

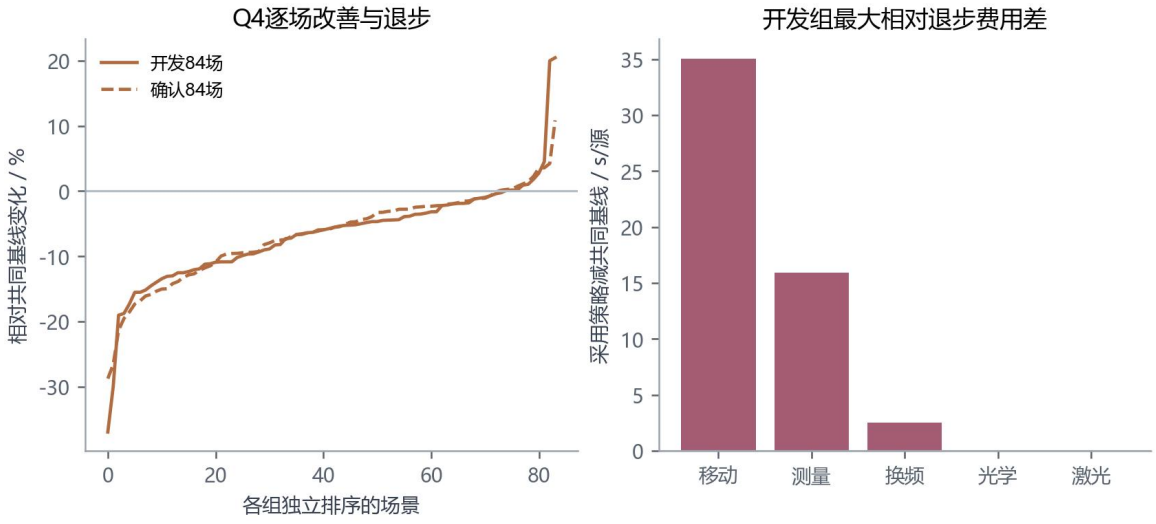


图 13 Q4 逐场相对变化与开发组最不利案例的费用差额

左图实线为开发组、虚线为确认组；右图取开发组最大相对退步例，正值为额外费用，负值为节省。案例编号及原始账本见支撑数据。

图 13 所示不利案例中，扫描顺序的变化推迟了未知定向源的有效检测，额外获得的已知源信息不足以抵消后续大范围移动。

补充外缘切向 10 源反例：同场旧 V12 与当前策略均全清，但总费由 5632.98 增至 6866.33 秒，增加 21.90%，其中 1132.36 秒来自移动。两者 21 站固定巡检折线同为 17.85 千米；当前策略穿插清除后实际行走 23.77 千米，旧策略为 18.10 千米。第 11 个巡检站起外环访问方向不同，随后三段穿插清除造成约 4.71 千米额外路程；频道 9 的首次发现由 3064.52 延迟至 6277.45 秒。

该案例说明，路线、发现时刻和清除顺序之间的耦合可能抵消局部信息收益。本例的对照对象为归档策略 V12，结果单列用于分析发现时序与清除顺序的耦合。原始动作和分段时间账本见支撑材料。

5.4.5 模型检验

参数固定后，我们在新的确认场景中再次进行同场比较，结果见表 14。

表 14 Q4 独立冻结确认的同场比较

策略	清除/总数	均值	P90	上尾均值	样本最大
共同基线	1092/1092	481.80	610.69	624.67	646.71
局部费用策略	1092/1092	470.30	589.75	604.53	632.12
选择性信息策略	**1092/1092**	**449.35**	**572.33**	**595.72**	**617.96**

单位：秒/源；统计口径同前，加粗行为采用方案。

表 14 显示，采用方案在独立确认组中的平均定位清除时间为 449.35 秒/源，较共同基线降低 6.73%。84 个场景中有 11 场变慢，最差相对变化为+10.84%。各源数层包含相同数量的场景，因此组间均值具有可比性。

随后通过单项关闭实验检验途中测向和已知频道筛选的作用，结果见表 15。

表 15 Q4 固定场景单项关闭对照

配置	场数	均值 / (s/源)	费用差额 / (s/源)
完整策略	28	453.06	+0.00
关闭途中测向	28	470.90	+17.84
关闭已知频道门控	28	457.14	+4.08

按 Q3 相同分层规则取 Q4 确认子集 28 场，差额口径同表 9。

由表 15 可知，关闭途中测向后，平均时间增加 17.84 秒/源，28 个场景中有 19 场变慢；关闭已知频道筛选后，平均时间增加 4.08 秒/源，其中 25 场变慢。结果表明，及时获取有效信息和减少低价值重复检测均能改善总体表现，其中途中测向的平均贡献更大。

问题四共完成六次官方演练，API 记录显示成功清除 78 个干扰源。按每场平均定位清除时间等权计算，结果为 476.89 秒/源；按总时间除以总清除数计算，结果为 460.28 秒/源。本文采用前一种统计结果作为主指标。

问题四的六次官方演练结果见表 16。

表 16 Q4 六次官方演练的归档结果

案例编码	清除数	平均时间 / s/源	运行时间 / s	清除比例
XD9J-6HEH-V246-4PGY	10	578.47	3.12	100%
XPPZ-6GNM-3WXN-7P76	16	328.29	3.17	100%
DGQV-55ES-YW63-X9C3	12	535.63	2.95	100%
XE6X-MQSB-WP8X-W9BA	16	410.46	3.58	100%
BN7C-7MVW-7XMM-HPFW	14	446.11	3.94	100%
HADD-VY97-VNKS-SRXX	10	562.38	2.94	100%

程序运行时间取官方响应中成功 enter 至成功 exit 的实时时间戳差，不含本地启动准备和退出后的打包耗时。

成功清除数与费用由 API 响应重算，清除比例依据模拟器结果页核对。

问题四的正式测试结果按表 17 所示口径记录。

表 17 问题四正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
Y65D-NFHG-HSA7-PTJ2	11	497.44	3.19
VWK5-49M4-GXCY-PD82	12	463.97	3.44
PEAD-ET89-3T4M-3RFR	15	367.34	3.44

采用算法：选择性信息策略

5.5 Q3Q4 综合检验

最后，我们按干扰源数量和动作类型分解完整任务时间，用于比较少源场景与满源场景的费用差异。由于两问的物理条件不同，策略比较均在同一问题的配对场景内进行。

分层结果和动作时间构成如图 14 所示。

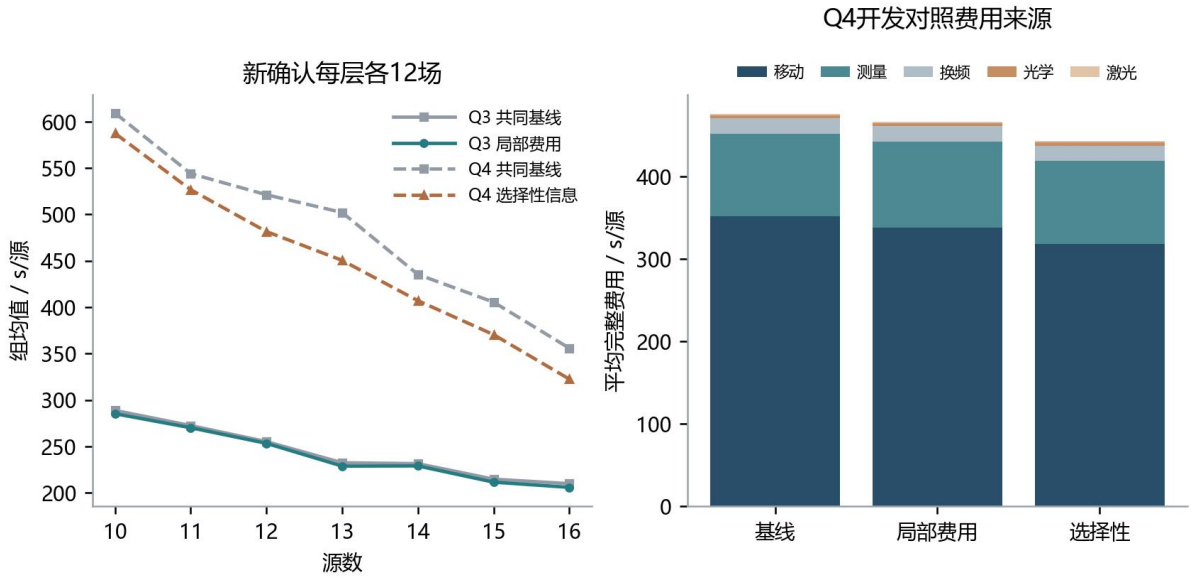


图 14 左为两问确认组按源数分层费用（Q3 实线、Q4 虚线）右为 Q4 开发组动作费用分解

由图 14 可知，干扰源较少时，无遗漏确认时间需要由较少的已清除源分摊；达到 16 个源的上限后，机器狗可以停止未知频道搜索。因此，平均定位清除时间随源数发生明显变化。

参数邻域检查固定每问 7 个普通场景（源数 10 至 16 各一场）及 2 个结构压力场景。Q3 仅将后验光学求积点数设为 256、512、1024，Q4 仅将搜索站已知频道的扫描门槛设为 20、30、40 米，其他设置不变；54 次运行均全清。默认参数固定后开展本检验，普通场景取自确认组，压力场景单列统计。

关键参数在邻域内变化时的整场费用见表 18。

表 18 关键参数邻域的整场费用变化

问题与参数	取值	普通均费	压力均费	普通最大退步 / %
Q3 求积点数	256	237.40	366.11	4.07
	512	235.29	366.19	0.00
	1024	236.83	366.20	2.64
Q4 站点已知频道门槛 / m	20	454.29	586.56	0.20
	30	454.41	586.38	0.00
	40	456.91	585.78	5.28

均费单位为 s/源；退步相对同问默认值（Q3 为 512 点，Q4 为 30 米）。原地与途中测向 120 米门槛保持不变。压力均值各取 2 场；原始费用与逐场差额见支撑数据。

表 18 显示，增加求积点数并未使问题三的任务时间单调下降，问题四的扫描门槛变化也可能改变后续路线。普通场景均值变化较小，但最大单场退步分别达到 4.07% 和 5.28%。该结果刻画了固定默认配置附近的局部敏感性。

图 15 汇总十二次官方演练的动作时间；横轴为案例码前四位，完整案例码见表 10 和表 16，虚线为逐场等权均值。

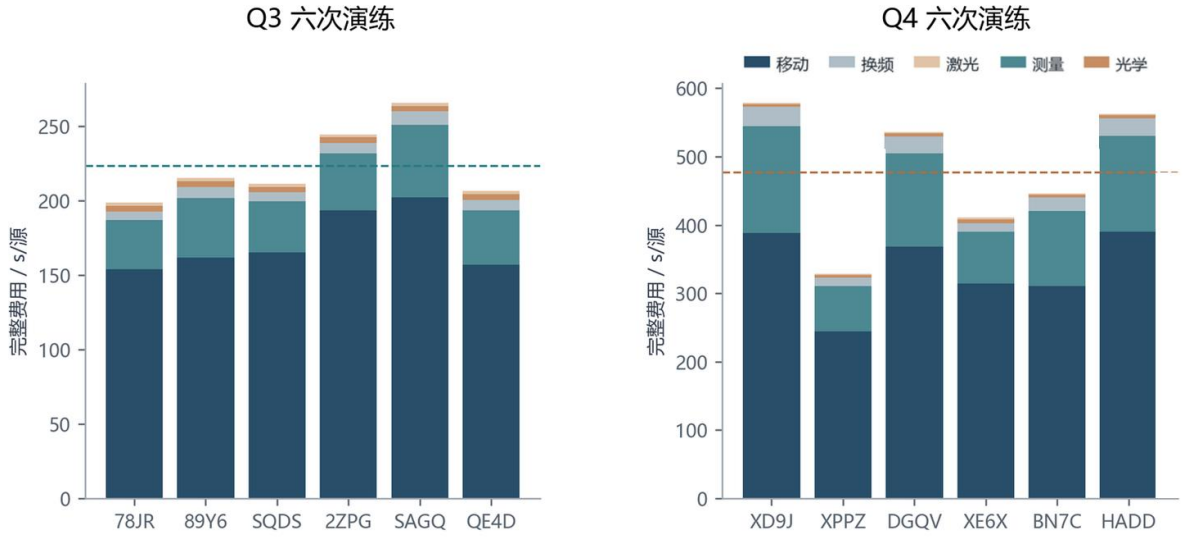


图 15 十二次官方演练归档的完整时间构成

各项时间先除以本场实际清除数。

为展示高耗时场景中的实际执行过程，我们绘制两问最慢演练的动作轨迹，如图 16 所示。

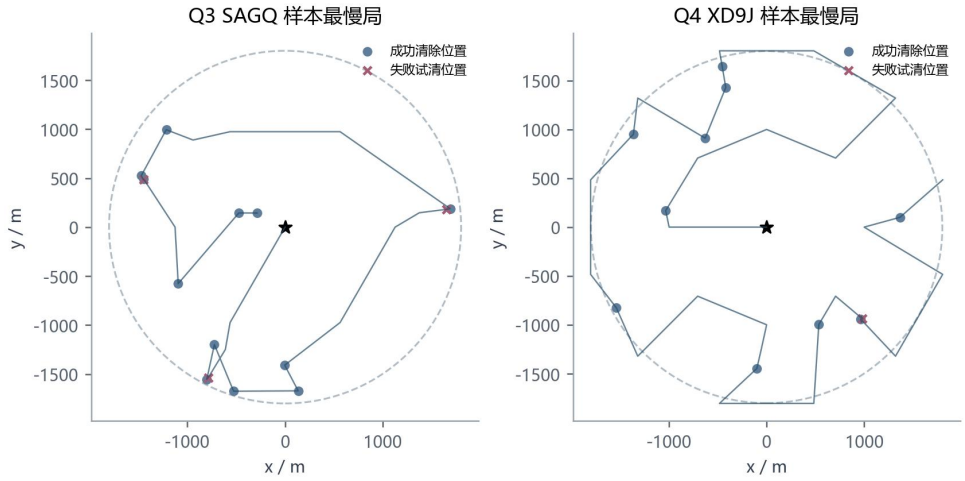


图 16 两问本批最慢演练的实际动作轨迹

图 16 展示了两问中平均定位清除时间最长的演练轨迹，包括实际行走路径以及成功和失败的光学尝试位置。图中标记成功清除位置，其 20 米作用范围内包含相应干扰源。

为便于运行与结果复核，本文提供统一图形界面，支持问题三、四的任务选择、离线模拟、官方接口连接及单局回放。界面与命令行共用模型及固定参数，操作方法见支撑材料。

六、模型评价与改进

6.1 模型优点

① 我们由首次接收信息推导接收半径约束，并建立四圆盘第二测点安全域。连续上下界能够同时评价定位精度和移动代价，最小包围圆则给出可靠清除判据。

② 我们利用近邻凸包刻画未知朝向下的发现条件，构造并连续认证了 21 点双环布局；同时利用多圆周加权约束将固定通用布局的点数界收紧为 19 至 21 点。

③ 我们在可靠几何约束内依据动作耗时估计选择测点和清除位置，并对已知频道筛选补测、安排途中测向；同场比较与单项关闭实验用于分析这些决策的收益。

6.2 模型局限

① 模型采用浮点运算和多边形外包处理连续几何约束。虽然设置了保守余量，但极端退化情形仍可能增加计算量，并影响有限时间内的执行效率。

② 费用评分含名义后续几何、惩罚和参考先验，只参与动作排序。固定候选测点后的访问排序具有旅行商路径问题的组合结构，在线任务还会随观测更新。本文采用插入、换边改进和滚动重规划控制计算量，未给出完整任务的全局最优保证。

③ 演练样本数量有限，且官方场景生成分布未知。因此，离线均值和压力场景结果仅反映相应实验范围内的表现。

6.3 改进方向

Vander Hook 等[5]研究了高斯误差条件下主动定位的费用，可为后续设计提供参考。本题采用有界误差，并包含未知发射半平面、失败试清和无遗漏确认，因此还需针对这些条件重新推导局部收缩和费用界。

后续可继续判定一般 19 点和 20 点布局的可行性，并利用唯一内站条件缩小 19 点搜索范围；在安全测点域内，进一步研究最坏后续费用与发现时序。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于【思路核查，论文排版矫正，批量数值试验，模拟仿真器制作】，详细使用情况见支撑材料。

参考文献

[1] 2026 年高教社杯全国大学生数学建模竞赛组委会. B 题 无线电干扰源的快速自动定位与清除及附件 1—2[S]. 2026.

[2] TOKEKAR P, ISLER V. Sensor Placement and Selection for Bearing Sensors with Bounded Uncertainty[C]//2013 IEEE International Conference on Robotics and Automation. 2013: 2515–2520. DOI: 10.1109/ICRA.2013.6630920.

[3] CROES G A. A Method for Solving Traveling-Salesman Problems[J]. Operations Research, 1958, 6(6): 791–812. DOI: 10.1287/opre.6.6.791.

[4] JORIS H. Le chasseur perdu dans la forêt (Un problème de géométrie plane)[J]. Elemente der Mathematik, 1980, 35(1): 1–14. 英译本见 FINCH S, 2019, arXiv:1910.00615.

[5] VANDER HOOK J, TOKEKAR P, ISLER V. Cautious Greedy Strategy for Bearing-only Active Localization: Analysis and Field Experiments[J]. Journal of Field Robotics, 2014, 31(2): 296–318. DOI: 10.1002/rob.21499.

附录

第一部分 支撑材料

以下路径均相对支撑材料根目录。58 个程序按题号逐项列出文件名与用途；数据、图件按目录列出。第二部分列出 24 个完整建模程序及 2 份参数。

编号	源程序文件及用途
01	问题一_多测点示例与退化检验.py：复核多点交集与退化情形
02	问题一_定位区域与包围圆.py：计算可行域、直径和包围圆
03	问题二_安全第二测点.py：求解鲁棒接收区域
04	问题二_定位直径上下界.py：计算定位精度界
05	问题三_七八九点巡检对比.py：比较三套巡检布局
06	问题三_主动测点选择.py：全向源主动测点备用精化规则
07	问题三_全向扇形约束.py：建立全向源几何约束
08	问题三_阴性观测几何核验.py：复核阴性更新规则
09	问题三_阴性观测收缩.py：利用无信号结果缩小范围
10	问题四_三角网格覆盖.py：生成网格覆盖基线
11	问题四_二十七点与二十一点对比.py：核验布局与路线差异
12	问题四_切向退步案例分析.py：解释边界场景费用增加
13	问题四_十九点下界与内站核验.py：检验点数及内站必要条件
14	问题四_圆周覆盖证书核验.py：核验圆周连续覆盖
15	问题四_基础点数界核验.py：复核初步点数界
16	问题四_途中信息收益求解.py：运行问题四选择性信息策略
17	问题四_阴影区域包含核验.py：定向阴性观测阴影的包含性检查
18	问题三四_交互模拟终端.py：启动图形操作界面
19	问题三四_信息调度基线.py：提供原地测量与延后服务组件
20	问题三四_信息费用调度.py：组织选择性扫描与信息动作
21	问题三四_候选区域概率计算.py：计算候选区域权重
22	问题三四_光学清除覆盖.py：生成保守光学清除网格
23	问题三四_光学试清位置选择.py：选择下一次试清位置
24	问题三四_全范围几何核验.py：检查几何约束与边界
25	问题三四_关键机制消融.py：比较移除机制后的结果
26	问题三四_几何求解基线.py：运行共同基线的几何定位框架
27	问题三四_动作接口协议.py：统一观测与动作数据
28	问题三四_动作费用核算.py：按题目规则累计费用
29	问题三四_动作轨迹独立核验.py：检查动作合法性与费用
30	问题三四_参数敏感性检验.py：比较邻近参数设置
31	问题三四_同场策略比较.py：在相同场景比较算法
32	问题三四_困难场景严格重放.py：复核困难场景动作记录
33	问题三四_场景求解与核验.py：执行场景并检查结果
34	问题三四_基础参数读取.py：读取模型公共参数
35	问题三四_安全定位候选.py：筛选满足接收条件的测点
36	问题三四_完整费用选点.py：按动作耗时估计比较局部测点
37	问题三四_官方模拟器连接.py：连接用户已开启的场景
38	问题三四_局部候选测点评估.py：评估邻近测点收益
39	问题三四_巡检点与路线生成.py：生成巡检站及初始路线
40	问题三四_开放巡检路线优化.py：优化无需返回起点的路线
41	问题三四_搜索定位清除框架.py：组织搜索、定位与清除
42	问题三四_模型加载.py：加载采用算法及依赖
43	问题三四_模拟与单局回放进程.py：执行本局并生成回放数据
44	问题三四_模拟终端界面.html：交互地图、状态与当前单局回放界面

编号	源程序文件及用途
45	问题三四_正式算法与参数入口.py：统一加载问题三四配置
46	问题三四_测向动作选择.py：选择测向或后续动作
47	问题三四_测点与清除规则.py：处理候选点及清除规则
48	问题三四_消融策略.py：提供机制移除的对照策略
49	问题三四_消融记录独立核验.py：核对消融汇总与原始记录
50	问题三四_物理约束调度.py：按 90°半角修正候选信息评分
51	问题三四_离线物理模拟.py：生成本地观测和动作反馈
52	问题三四_论文统计汇总.py：汇总论文引用指标
53	问题三四_费用驱动候选求解.py：运行局部费用候选及离线对照
54	问题三四_边界场景构造.py：构造针对性边界案例
55	问题三四_连续覆盖构造.py：构造可认证的覆盖布局
56	问题一至四_几何示意图绘制.py：绘制模型几何插图
57	问题一至四_实验结果图绘制.py：绘制结果与对比图
58	问题一至四_统一复现.py：统一运行四问复现任务
数据	数据/：问题一至四的参数、输入场景、统计结果和证明证书；official_logs/与 practice/保存 12 局演练，正式测试/保存 6 局正式日志、API 记录与结果。
图件	图件：正文 16 幅插图的 PNG 与 SVG 文件，文件编号与正文图号一致。
说明	复现说明.md：运行步骤与统计口径；环境依赖.txt：依赖版本；AI 工具使用详情.docx：AI 使用记录。

第二部分 建模求解源程序

文件 1—2 对应集合几何与第二问，文件 3—4 对应第三问费用选点，文件 5—6 对应第四问途中测向与调度，随后为共同数学组件及参数。以下模型文件完整列出，与支撑材料同名文件逐行对应。界面、绘图、独立核验和运行工具代码不在此展开；进入支撑材料文件夹，按复现说明运行。

1 问题一_定位区域与包围圆.py 半平面交与最小包围圆

```
"""有界示向误差的几何运算；坐标与距离容差均以米计。"""
import itertools
import math

EPS = 1e-7


def unit(deg):
    a = math.radians(deg)
    return math.cos(a), math.sin(a)


def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]


def sub(a, b):
    return a[0] - b[0], a[1] - b[1]


def distance(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])


def bearing_planes(position, bearing, delta=1.005, max_range=None):
    """返回两条角边界对应的 a*x+b*y≤c 半平面。"""
    low, high = unit(bearing - delta), unit(bearing + delta)
    normals = [(low[1], -low[0]), (-high[1], high[0])]
    planes = [(a, b, a * position[0] + b * position[1]) for a, b in normals]
    if max_range is not None:
        a, b = unit(bearing)
```

1 问题一_定位区域与包围圆.py 半平面交与最小包围圆

```
        planes.append((a, b, a * position[0] + b * position[1] + max_range))
    return planes


def convex_hull(points):
    pts = sorted(set(tuple(p) for p in points))
    if len(pts) <= 1:
        return pts
    def half(seq):
        result = []
        for p in seq:
            while len(result) >= 2 and cross(sub(result[-1], result[-2]), sub(p, result[-1])) <= 0:
                result.pop()
            result.append(p)
        return result
    return half(pts)[:-1] + half(reversed(pts))[:-1]


def clip(poly, plane):
    """按略向外放宽的半平面裁剪，避免数值误差收缩真实边界。"""
    if not poly:
        return []
    a, b, c = plane
    c += EPS * math.hypot(a, b)
    out = []
    for p, q in zip(poly, poly[1:] + poly[:1]):
        fp, fq = a * p[0] + b * p[1] - c, a * q[0] + b * q[1] - c
        if fp <= 0:
            out.append(p)
        if (fp <= 0) != (fq <= 0):
            t = fp / (fp - fq)
            out.append((p[0] + t * (q[0] - p[0]), p[1] + t * (q[1] - p[1])))
    return convex_hull(out)


def diameter(poly):
    """枚举全部顶点对计算直径，复杂度 O(k²)，作为小区域的参考实现。"""
    if not poly:
        return None, None
    pair = max(itertools.combinations_with_replacement(poly, 2), key=lambda pq: distance(*pq))
    return distance(*pair), pair


def circle_two(a, b):
    c = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
    return c, distance(c, a)


def circle_three(a, b, c):
    """平移坐标后求三个点的最小包围圆。"""
    candidates = [circle_two(p, q) for p, q in ((a, b), (a, c), (b, c))]
    enclosing = [cr for cr in candidates if all(distance(cr[0], p) <= cr[1] + EPS for p in (a, b, c))]
    if enclosing:
        return min(enclosing, key=lambda cr: cr[1])
    u, v = sub(b, a), sub(c, a)
    d = 2 * cross(u, v)
    if abs(d) < 1e-18:
        # 机器精度下共线时，以最远点对圆为基础保守扩张。
        center, _ = max(candidates, key=lambda cr: cr[1])
        return center, max(distance(center, p) for p in (a, b, c))
    u2, v2 = sum(x*x for x in u), sum(x*x for x in v)
    center = (a[0] + (u2*v[1] - v2*u[1])/d,
              a[1] + (u[0]*v2 - v[0]*u2)/d)
    return center, max(distance(center, p) for p in (a, b, c))


def enclosing_circle(poly):
    """枚举 1、2、3 个边界支撑点，复杂度 O(k⁴)，用于小多边形。
    候选圆若遗漏顶点就不接受；只有选出包住全部点的圆后才附加数值余量。"""
    if not poly:
        return None
    if len(poly) == 1:
        return poly[0], 0.0
```

1 问题一_定位区域与包围圆.py 半平面交与最小包围圆

```
    best = None
    candidates = (circle_two(*pair) for pair in itertools.combinations(poly, 2))
    for center, radius in itertools.chain(candidates,
            (circle_three(*triple) for triple in itertools.combinations(poly, 3))):
        if best is not None and radius >= best[1]:
            continue
        if all(distance(center, p) <= radius + EPS for p in poly):
            best = center, max(distance(center, p) for p in poly)
    if best is None:
        raise ArithmeticError('No enclosing circle found')
    return best[0], best[1] + EPS


def point_region_distance(point, poly):
    if not poly:
        return math.inf
    if len(poly) == 1:
        return distance(point, poly[0])
    if len(poly) >= 3 and all(cross(sub(q, p), sub(point, p)) >= -EPS
                              for p, q in zip(poly, poly[1:] + poly[:1])):
        return 0.0
    best = math.inf
    for p, q in zip(poly, poly[1:] + poly[:1]):
        v = sub(q, p)
        norm2 = v[0]*v[0] + v[1]*v[1]
        t = max(0, min(1, sum(a*b for a, b in zip(sub(point, p), v))/norm2)) if norm2 else 0
        best = min(best, distance(point, (p[0] + t*v[0], p[1] + t*v[1])))
    return best


def halfplane_region(planes):
    """Q1 先用线性规划判可行性和有界性，再枚举合法边界交点。
    不添加人工外接框；独立分析需要 SciPy。一般复杂度为 O(m³)加线性规划，是便于核查的参考实现。"""
    from scipy.optimize import linprog
    normalized = []
    for a, b, c in planes:
        norm = math.hypot(a, b)
        if norm == 0:
            if c < 0:
                return {'status': 'empty', 'vertices': [], 'diameter_m': None}
        else:
            normalized.append((a/norm, b/norm, c/norm))
    if not normalized:
        return {'status': 'unbounded', 'vertices': [], 'diameter_m': None}
    kwargs = dict(A_ub=[p[:2] for p in normalized], b_ub=[p[2] for p in normalized],
                  bounds=[(None, None), (None, None)], method='highs')
    feasible = linprog([0, 0], **kwargs)
    if feasible.status == 2:
        return {'status': 'empty', 'vertices': [], 'diameter_m': None}
    if not feasible.success:
        raise ArithmeticError(feasible.message)
    for objective in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        result = linprog(objective, **kwargs)
        if result.status == 3:
            return {'status': 'unbounded', 'vertices': [], 'diameter_m': None}
        if not result.success:
            raise ArithmeticError(result.message)
    vertices = []
    for (a, b, c), (d, e, f) in itertools.combinations(normalized, 2):
        det = a*e - b*d
        if abs(det) < 1e-14:
            continue
        point = ((c*e - b*f)/det, (a*f - c*d)/det)
        if all(x*point[0] + y*point[1] <= z + EPS for x, y, z in normalized):
            if not any(distance(point, p) < EPS for p in vertices):
                vertices.append(point)
    hull = convex_hull(vertices)
    if not hull:
        raise ArithmeticError('Bounded feasible set without recoverable vertices')
    value, pair = diameter(hull)
    center, radius = enclosing_circle(hull)
    return {'status': 'point' if len(hull) == 1 else 'segment' if len(hull) == 2 else 'polygon',
            'vertices': hull, 'diameter_m': value, 'diameter_pair': pair,
```

1 问题一_定位区域与包围圆.py 半平面交与最小包围圆

```
            'enclosing_center': center, 'enclosing_radius_m': radius}


def second_point(position, bearing, lateral=500.0):
    u, v = unit(bearing), unit(bearing + 90)
    return (position[0] + 750*u[0] + lateral*v[0],
            position[1] + 750*u[1] + lateral*v[1])


def second_point_bounds(lateral=500.0, delta=1.005):
    d = math.radians(delta)
    farthest = max(math.hypot(750, lateral), math.hypot(1500*math.cos(d)-750,
                                                        1500*math.sin(d)+abs(lateral)))
    return {'max_source_distance_m': farthest, 'guaranteed_omni_reception': farthest <= 1000,
            'move_distance_m': math.hypot(750, lateral)}


def second_candidate_region(position, bearing, delta=1.005):
    """三个圆盘不等式给出连续接收安全域的充分条件。
    首测 1500 米扇区包含于顶点为原点及(1500,±1500tanδ)的三角形。距离为凸函数，三个顶点到候选均不超过 1000 米即可保证整
域。"""
    u, v = unit(bearing), unit(bearing+90)
    lateral = 1500*math.tan(math.radians(delta))
    centers = [tuple(position)] + [(position[0]+1500*u[0]+sign*lateral*v[0],
                                    position[1]+1500*u[1]+sign*lateral*v[1]) for sign in (-1, 1)]
    return {'description': 'intersection of three closed disks; sufficient for omni reception',
            'disk_centers': centers, 'disk_radius_m': 1000.0,
            'chosen_point': second_point(position, bearing)}
```

2 问题二_定位直径上下界.py 第二问连续上下界

```
"""用外包多边形向量化评价 Q2 不确定性，不调用模拟器。"""
import math
import numpy as np

def outer_prior(delta=1.005, arc_segments=16):
    # 用相邻切线交点外包半径 1500 米的首测扇区。
    d=math.radians(delta)
    angles=np.linspace(-d,d,arc_segments+1)
    points=[(0.,0.),(1500*math.cos(d),-1500*math.sin(d))]
    for a,b in zip(angles[:-1],angles[1:]):
        r=1500/math.cos((b-a)/2)
        points.append((r*math.cos((a+b)/2),r*math.sin((a+b)/2)))
    points.append((1500*math.cos(d),1500*math.sin(d)))
    return np.asarray(points)

def clip_batch(vertices,counts,normals,cs):
    n,k,_=vertices.shape
    idx=np.arange(k)[None,:]
    nxt=(idx+1)%np.maximum(counts[:,None],1)
    following=vertices[np.arange(n)[:,None],nxt]
    fp=np.einsum('nki,ni->nk',vertices,normals)-cs[:,None]
    fq=np.einsum('nki,ni->nk',following,normals)-cs[:,None]
    valid=idx<counts[:,None]
    keep=(fp<=0)&valid
    cut=((fp<=0)!=(fq<=0))&valid
    denom=fp-fq
    ratio=np.divide(fp,denom,out=np.zeros_like(fp),where=denom!=0)
    crossing=vertices+ratio[:,:,None]*(following-vertices)
    stream=np.stack([vertices,crossing],axis=2).reshape(n,2*k,2)
    flags=np.stack([keep,cut],axis=2).reshape(n,2*k)
    rank=np.cumsum(flags,axis=1)-1
    newcounts=flags.sum(axis=1)
    output=np.zeros((n,k+1,2))
    ni,ki=np.where(flags)
    output[ni,rank[ni,ki]]=stream[ni,ki]
    return output,newcounts

def posterior_batch(point,readings,delta=1.005,prior=None):
    prior=outer_prior(delta) if prior is None else np.asarray(prior)
    n=len(readings)
    vertices=np.broadcast_to(prior,(n,len(prior),2)).copy()
    counts=np.full(n,len(prior),dtype=int)
    for sign in (-1,1):
        angle=np.radians(readings+sign*delta)
        normal=sign*np.stack([-np.sin(angle),np.cos(angle)],axis=1)
        c=normal@np.asarray(point)+1e-7
        vertices,counts=clip_batch(vertices,counts,normal,c)
    # 枚举全部保留顶点对，以最大距离作为直径。
    best=np.zeros(n)
    pairs=np.zeros((n,2,2))
    k=vertices.shape[1]
    for i in range(k):
        for j in range(i):
            d=np.linalg.norm(vertices[:,i]-vertices[:,j],axis=1)
            valid=(i<counts)&(j<counts)&(d>best)
            best[valid]=d[valid]
            pairs[valid,0]=vertices[valid,i]
            pairs[valid,1]=vertices[valid,j]
    return best,pairs

def bound(point,step=.01,delta=1.005,arc_segments=16):
    prior=outer_prior(delta,arc_segments)
    angles=np.unwrap(np.arctan2(prior[:,1]-point[1],prior[:,0]-point[0]))
    lo,hi=np.degrees([min(angles),max(angles)])
    n=max(2,math.ceil((hi-lo+2*delta)/step))
    edges=np.linspace(lo-delta,hi+delta,n+1)
    half=(edges[1]-edges[0])/2
    readings=(edges[:-1]+edges[1:])/2
    value=-1
    witness=None
    for start in range(0,n,1000):
        b=readings[start:start+1000]
        ds,pairs=posterior_batch(point,b,delta+half,prior)
```

2 问题二_定位直径上下界.py 第二问连续上下界

```
        i=int(np.argmax(ds))
        if ds[i]>value:
            value=float(ds[i]); witness={'reading_bin_center_deg':float(b[i]),
                'outer_pair_m':pairs[i].tolist()}
    return {'upper_m':value+1e-5,'angle_step_deg':2*half,'bins':n,

'arc_segments':arc_segments,'outer_arc_excess_m':1500*(1/math.cos(math.radians(delta)/arc_segments)-1),
            'witness_bin':witness,'numerics':'outward 1e-7 m halfplanes, 1e-5 m reported guard; floating-point
geometry'}

def sector_witness(point,delta=1.005):
    # 在两条角边界上构造可行点对，其中 A 是真实圆弧端点。
    d=math.radians(delta)
    best={'lower_m':0}
    for sign in (-1,1):
        A=np.array([1500*math.cos(d),sign*1500*math.sin(d)])
        beta=math.atan2(A[1]-point[1],A[0]-point[0])
        ray=np.array([math.cos(-sign*d),math.sin(-sign*d)])
        for turn in (-1,1):
            angle=beta+turn*2*d
            v=np.array([math.cos(angle),math.sin(angle)])
            try:r,t=np.linalg.solve(np.column_stack([ray,-v]),np.asarray(point))
            except np.linalg.LinAlgError:continue
            B=r*ray
            if 5<r<=1500 and t>5 and math.dist(A,point)>5:
                dist=float(np.linalg.norm(A-B))
                if dist>best['lower_m']:
                    best={'lower_m':dist,'points_m':[A.tolist(),B.tolist()],
                          'first_reading_deg':0.,'second_reading_deg':math.degrees(beta+turn*d),
                          'reception_radius_m':1500.}
    return best
```

3 问题三四_完整费用选点.py 按动作耗时估计比较局部测点

```
"""按检测 5 秒、失败清除 3 秒、成功清除 5 秒修正费用排名。
这些量是启发式费用估计，并非精确随机价值函数。候选几何和接收检查保持不变。"""
import math
from 问题三四_光学试清位置选择 import OpticalPosteriorQ3, OpticalPosteriorQ4, weighted_median_point
from 问题三四_候选区域概率计算 import hypothesis_mass, halfplane_mass
from 问题三四_局部候选测点评估 import sample_polygon, receive_probability, distance, unit, clip, bearing_planes,
enclosing_circle
import 问题三四_模型加载 as bootstrap

MEASURE_S=5.
FAILED_CLEAR_S=3.
SUCCESSFUL_CLEAR_S=5.
SPEED_M_S=5.


class Cost5Mixin:
    def choose_pair(self,original,track,**kwargs):
        """同时替换后验排名与面积权重回退排名。"""
        baseline=original(track,**kwargs)
        if self.cfg['posterior_mode']=='nearest':
            if baseline and
distance(self.s.position,baseline['points'][1])<distance(self.s.position,baseline['points'][0])-1e-9:
                return dict(baseline,points=list(reversed(baseline['points'])),
                    max_source_distance_m=list(reversed(baseline['max_source_distance_m'])))
            return baseline
        if not track.poly or track.inconsistent or track.cleared or not track.first:return baseline
        origin,bearing=track.first;u=unit(bearing)
        xs=[(p[0]-origin[0])*u[0]+(p[1]-origin[1])*u[1] for p in track.poly]
        lo,hi=min(xs),max(xs);span=hi-lo
        candidates=[]
        if baseline:candidates.append(baseline)
        for fraction in (.25,.4,.55,.7,.85):
            for lateral in (30.,60.,100.,180.):
                pair=original(track,forward=max(.001,lo+fraction*span),lateral=lateral)
                if pair:candidates.append(pair)
        if not candidates:return baseline
        positives=[p for p,r in self.local_history[track.channel] if r!='no_signal']
        negatives=[p for p,r in self.local_history[track.channel] if r=='no_signal']
        samples=[]
        for source,area_weight in sample_polygon(track.poly):

mass,omni,components=hypothesis_mass(source,positives,negatives,self.problem,self.cfg['posterior_mode'])
            if mass>0:samples.append((source,area_weight*mass,mass,omni,components))
        area_fallback=not samples
        if area_fallback:
            samples=[(p,w,1.,1.,[]) for p,w in sample_polygon(track.poly)]
        total=sum(x[1] for x in samples);best=None
        for pair in candidates:
            points=sorted(pair['points'],key=lambda p:distance(self.s.position,p))
            if points!=pair['points']:

pair=dict(pair,points=points,max_source_distance_m=list(reversed(pair['max_source_distance_m'])))
            q=points[0];estimate=distance(self.s.position,q)/SPEED_M_S+MEASURE_S
            for source,weight,mass,omni,components in samples:
                poly=list(track.poly)
                angle=math.degrees(math.atan2(source[1]-q[1],source[0]-q[0]))
                for plane in bearing_planes(q,angle,1.005,1500):poly=clip(poly,plane)
                circle=enclosing_circle(poly) if poly else track.circle;r=circle[1]
                positive=(max(0.,distance(q,circle[0])-max(0.,19.99-r))/SPEED_M_S+
                          SUCCESSFUL_CLEAR_S+self.cfg['pair_cost_weight']*max(0.,r-19.99)*.25)
                if self.problem=='q3':probability=1.
                elif area_fallback or
self.cfg['posterior_mode']=='radius':probability=receive_probability(origin,q,source)
                else:probability=(omni+sum(w*halfplane_mass(intervals,source,q) for w,intervals in
components))/mass
                negative=(distance(q,points[1])/SPEED_M_S+MEASURE_S+
                          distance(points[1],source)/SPEED_M_S+SUCCESSFUL_CLEAR_S)
                estimate+=weight/total*(probability*positive+(1-probability)*negative)
            key=(estimate,tuple(points))
            if best is None or key<best[0]:best=(key,pair)
        self.events.append(dict(method='m020_cost5_pair_ranking',channel=track.channel,
            evaluated_candidates=len(candidates),selected_surrogate_cost_s=best[0][0],
            ranking_only=True,measurement_cost_s=MEASURE_S,successful_clear_cost_s=SUCCESSFUL_CLEAR_S,
```

3 问题三四_完整费用选点.py 按动作耗时估计比较局部测点

```
            synthetic_prior='uniform_area_radius_50pct_directional',area_fallback=area_fallback,
            positive_points=positives,negative_points=negatives,source_polygon=track.poly,
            selected_points=best[1]['points'],posterior_mode=self.cfg['posterior_mode']))
        return best[1]

    def select_optical(self,track,center):
        """只替代一次清除位置，失败后执行原中心 5 秒测量。"""
        samples=self.posterior_profile(track)
        if not samples:return None
        start=self.s.position
        candidates=[tuple(center),weighted_median_point(samples)]
        candidates.append((sum(p[0]*w for p,w,*_ in samples),sum(p[1]*w for p,w,*_ in samples)))
        u=unit(track.first[1]);origin=track.first[0]
        projected=sorted(((p[0]-origin[0])*u[0]+(p[1]-origin[1])*u[1],p,w)
                         for p,w,*_ in samples)
        quantiles=[i/20 for i in range(1,20)];cumulative=0.;j=0
        for x,p,w in projected:
            cumulative+=w
            while j<len(quantiles) and cumulative>=quantiles[j]:
                nearby=[(q,v) for y,q,v in projected if abs(y-x)<=18]
                total=sum(v for q,v in nearby)
                candidates.append((sum(q[0]*v for q,v in nearby)/total,sum(q[1]*v for q,v in nearby)/total))
                j+=1
        candidates.append(tuple(start))
        continuation=[]
        for source,w,mass,omni,components in samples:
            reception=1. if self.problem=='q3' else (omni+sum(v*halfplane_mass(parts,source,center) for
v,parts in components))/mass
            remaining=MEASURE_S+SUCCESSFUL_CLEAR_S+distance(center,source)/SPEED_M_S+40*(1-reception)
            continuation.append((source,w,remaining))
        def score(q):
            value=distance(start,q)/SPEED_M_S+FAILED_CLEAR_S
            back=distance(q,center)/SPEED_M_S;hit=0.
            for source,w,remaining in continuation:
                if distance(q,source)<=19.8:
                    hit+=w;value+=(SUCCESSFUL_CLEAR_S-FAILED_CLEAR_S)*w
                else:value+=w*(back+remaining)
            return value,hit
        original=score(center)
        best=min(((score(q),q) for q in candidates),key=lambda row:(row[0][0],row[1]))
        gain=original[0]-best[0][0]
        if gain<self.cfg['optical_gain_threshold'] or distance(best[1],center)<1e-5:return None
        return dict(point=best[1],gain_estimate_s=gain,old_score_s=original[0],new_score_s=best[0][0],
                    old_hit_probability_estimate=original[1],new_hit_probability_estimate=best[0][1],
                    samples=len(samples),candidates=len(candidates),measurement_cost_s=MEASURE_S,
                    failed_clear_cost_s=FAILED_CLEAR_S,successful_clear_cost_s=SUCCESSFUL_CLEAR_S,
                    interpretation='approximate ranking surrogate with physical action prices')


class Cost5Q3(Cost5Mixin,OpticalPosteriorQ3):pass
class Cost5Q4(Cost5Mixin,OpticalPosteriorQ4):pass


def solve(session,problem,settings=None):
    cls=Cost5Q3 if problem=='q3' else Cost5Q4
    return cls(session,problem,bootstrap.baseline_settings(problem) if settings is None else settings).run()
```

4 问题三四_光学试清位置选择.py 一次后验光学位置替代

```
"""一次有界的后验光学试探与后验路线预测。
后验只用于排名，抽样不排除几何可能性。替代试清失败后立即回原中心实测，并保留有限后备。"""
import math
from 问题三四_候选区域概率计算 import PosteriorQ3, PosteriorQ4, hypothesis_mass, halfplane_mass, distance, unit
import 问题三四_模型加载 as bootstrap


def radical_inverse(index,base):
    value=0.;scale=1./base
    while index:
        index,digit=divmod(index,base);value+=digit*scale;scale/=base
    return value


def polygon_quadrature(polygon,count=512):
    """按凸多边形面积作确定性取点，只用于动作排名。"""
    c=(sum(p[0] for p in polygon)/len(polygon),sum(p[1] for p in polygon)/len(polygon))
    triangles=[];total=0.
    for a,b in zip(polygon,polygon[1:]+polygon[:1]):
        area=abs((a[0]-c[0])*(b[1]-c[1])-(a[1]-c[1])*(b[0]-c[0]))/2
        if area>1e-12:
            triangles.append((total,total+area,a,b));total+=area
    if total<1e-10:return [(p,1/len(polygon)) for p in polygon]
    result=[]
    for i in range(1,count+1):
        x=radical_inverse(i,2)*total;y=radical_inverse(i,3)
        lo,hi,a,b=next(t for t in triangles if t[0]<=x<t[1])
        r=math.sqrt((x-lo)/(hi-lo));wa=r*(1-y);wb=r*y;wc=1-r
        result.append(((wc*c[0]+wa*a[0]+wb*b[0],wc*c[1]+wa*a[1]+wb*b[1]),1/count))
    return result


def weighted_median_point(samples):
    c=(sum(p[0]*w for p,w,*_ in samples),sum(p[1]*w for p,w,*_ in samples))
    for _ in range(30):
        weighted=[(p,w/max(1e-5,distance(c,p))) for p,w,*_ in samples]
        total=sum(w for p,w in weighted)
        q=(sum(p[0]*w for p,w in weighted)/total,sum(p[1]*w for p,w in weighted)/total)
        if distance(c,q)<1e-4:break
        c=q
    return c


class OpticalPosteriorMixin:
    def __init__(self,session,problem,settings=None):
        super().__init__(session,problem,settings)
        self.cfg.setdefault('posterior_target',False)
        self.cfg.setdefault('posterior_optical',True)
        self.cfg.setdefault('optical_gain_threshold',2.)
        self.cfg.setdefault('optical_samples',512)
        self._optical_used=set()
        self._profile_cache={}

    def posterior_profile(self,track):
        key=(track.channel,tuple(track.poly or ()),len(self.local_history[track.channel]))
        if key in self._profile_cache:return self._profile_cache[key]
        positives=[p for p,r in self.local_history[track.channel] if r!='no_signal']
        negatives=[p for p,r in self.local_history[track.channel] if r=='no_signal']
        result=[]
        for source,area_weight in polygon_quadrature(track.poly,self.cfg['optical_samples']):
            mass,omni,components=hypothesis_mass(source,positives,negatives,self.problem,'full')
            if mass>1e-16:result.append((source,area_weight*mass,mass,omni,components))
        if result:
            total=sum(x[1] for x in result)
            result=[(p,w/total,m,o,c) for p,w,m,o,c in result]
        self._profile_cache[key]=result
        return result

    def target_point(self,track):
        if (self.cfg.get('posterior_target',False) and track.poly and not track.inconsistent and
                track.near is None and track.circle and track.circle[1]>25):
            samples=self.posterior_profile(track)
            if samples:
```

4 问题三四_光学试清位置选择.py 一次后验光学位置替代

```
                c=weighted_median_point(samples)
                if len(track.observations)==1 and track.circle[1]>=200:
                    v=unit(track.first[1]+90);offset=self.cfg['lateral']
                    candidates=[(c[0]+sign*offset*v[0],c[1]+sign*offset*v[1]) for sign in (-1,1)]
                    c=min(candidates,key=lambda p:distance(self.s.position,p))
                return c
        return super().target_point(track)

    def select_optical(self,track,center):
        samples=self.posterior_profile(track)
        if not samples:return None
        start=self.s.position
        candidates=[tuple(center),weighted_median_point(samples)]
        candidates.append((sum(p[0]*w for p,w,*_ in samples),sum(p[1]*w for p,w,*_ in samples)))
        # 沿不确定性长轴寻找光学候选，以横向条件
        # 均值避免稀疏求积点造成任意离散位置。
        u=unit(track.first[1]);origin=track.first[0]
        projected=sorted(((p[0]-origin[0])*u[0]+(p[1]-origin[1])*u[1],p,w)
                         for p,w,*_ in samples)
        quantiles=[i/20 for i in range(1,20)];cumulative=0.;j=0
        for x,p,w in projected:
            cumulative+=w
            while j<len(quantiles) and cumulative>=quantiles[j]:
                nearby=[(q,v) for y,q,v in projected if abs(y-x)<=18]
                total=sum(v for q,v in nearby)
                candidates.append((sum(q[0]*v for q,v in nearby)/total,sum(q[1]*v for q,v in nearby)/total))
                j+=1
        # 尚无明确清除位置时，原地或附近补测也可能有价值。
        candidates.append(tuple(start))
        continuation=[]
        for source,w,mass,omni,components in samples:
            reception=1. if self.problem=='q3' else (omni+sum(v*halfplane_mass(parts,source,center) for
v,parts in components))/mass
            remaining=3+5+distance(center,source)/5+40*(1-reception)
            continuation.append((source,w,remaining))
        def score(q):
            value=distance(start,q)/5+3
            back=distance(q,center)/5;hit=0.
            for source,w,remaining in continuation:
                if distance(q,source)<=19.8:
                    hit+=w;value+=2*w
                else:value+=w*(back+remaining)
            return value,hit
        original=score(center)
        best=min(((score(q),q) for q in candidates),key=lambda row:(row[0][0],row[1]))
        gain=original[0]-best[0][0]
        if gain<self.cfg['optical_gain_threshold'] or distance(best[1],center)<1e-5:return None
        return dict(point=best[1],gain_estimate_s=gain,old_score_s=original[0],new_score_s=best[0][0],
                    old_hit_probability_estimate=original[1],new_hit_probability_estimate=best[0][1],
                    samples=len(samples),candidates=len(candidates))

    def attempt(self,track,p,method):
        if (method=='center_probe' and self.cfg['posterior_optical'] and track.channel not in
self._optical_used and
                track.poly and not track.inconsistent and track.circle and track.circle[1]>20):
            self._optical_used.add(track.channel)
            choice=self.select_optical(track,p)
            if choice:
                self.events.append(dict(method='m020_posterior_optical_choice',channel=track.channel,
                    original_center=p,source_polygon=track.poly,ranking_only=True,choice=choice,
                    failure_continuation='original center measurement and unchanged bounded fallback'))
                # 每频道仅替代一次；失败时由调用方测量原来的
                # 中心 p，然后继续原有有界循环。
                return super().attempt(track,choice['point'],'m020_posterior_optical_probe')
        return super().attempt(track,p,method)


class OpticalPosteriorQ3(OpticalPosteriorMixin,PosteriorQ3):pass
class OpticalPosteriorQ4(OpticalPosteriorMixin,PosteriorQ4):pass


def solve(session,problem,settings=None):
    cls=OpticalPosteriorQ3 if problem=='q3' else OpticalPosteriorQ4
```

4 问题三四_光学试清位置选择.py 一次后验光学位置替代

```
    return cls(session,problem,bootstrap.baseline_settings(problem) if settings is None else settings).run()
```

5 问题四_途中信息收益求解.py 有次数约束的途中测向

```
"""只插入有利用价值的信息动作，比较较早采样与最大几何收益。"""
import math
from 问题三四_模型加载 import m020_settings
from 问题三四_信息费用调度 import Q3 as ParentQ3, Q4 as ParentQ4
class InterceptMixin:
    def intercept(self,key,goal):
        threshold=self.cfg['intercept_gain_m']
        if not math.isfinite(threshold) or self.cfg['intercept_scans_only'] and key[0]!='scan':return False
        start=self.s.position;dx,dy=goal[0]-start[0],goal[1]-start[1];length=math.hypot(dx,dy)
        if length<150:return False
        choices=[]
        for t in self.tracks.values():
            if t.cleared or not t.circle or t.inconsistent or
self.intercepts[t.channel]>=self.cfg['intercept_cap']:continue
            if self.cfg.get('intercept_other_only',False) and key==('target',t.channel):continue
            center,radius=t.circle
            if radius<80:continue
            fraction=((center[0]-start[0])*dx+(center[1]-start[1])*dy)/(length*length)
            for alpha in sorted(set([.15,.3,.5,.7,.85,max(.1,min(.9,fraction))])):
                if alpha*length<60 or (1-alpha)*length<30:continue
                p=(start[0]+alpha*dx,start[1]+alpha*dy);score=self.predicted_gain(t,p)
                ranking=score/(30+alpha*length) if self.cfg.get('intercept_efficiency',False) else score
                if score>=threshold:choices.append((ranking,-alpha,-t.channel,p,t,score))
        if not choices:return False
        ranking,negative_alpha,_,point,t,score=max(choices,key=lambda row:row[:3])
        self.intercepts[t.channel]+=1
        self.events.append(dict(method='enroute_measure_choice',channel=t.channel,position=point,
            planned_next=key,planned_endpoint=goal,start=start,predicted_radius_gain_m=score,
            interpolation_fraction=-negative_alpha,attempt=self.intercepts[t.channel],ranking=ranking,
            note='Planning heuristic; no deletion of coverage or guarantee of lower total cost'))
        self.observe(t.channel,point);return True
class Q3(InterceptMixin,ParentQ3):pass
class Q4(InterceptMixin,ParentQ4):pass
def solve(session,problem,settings=None):return (Q3 if problem=='q3' else
Q4)(session,problem,m020_settings(problem) if settings is None else settings).run()
```

6 问题三四_信息费用调度.py 已知频道扫描与路线控制

```
"""途中测向与路线改进；完整退出的条件保持不变。"""
import math
from 问题三四_模型加载 import m020_settings
from 问题三四_物理约束调度 import CorrectedMixin
from 问题三四_完整费用选点 import Cost5Q3, Cost5Q4
from 问题一_定位区域与包围圆 import distance, point_region_distance
from 问题三四_开放巡检路线优化 import optimize

class ControlMixin:
    def __init__(self,session,problem,settings=None):
        super().__init__(session,problem,settings)
        self.cfg.setdefault('strong_route',False)
        self.cfg.setdefault('route_exact_limit',10)
        self.cfg.setdefault('intercept_gain_m',float('inf'))
        self.cfg.setdefault('intercept_cap',2)
        self.cfg.setdefault('intercept_scans_only',False)
        self.cfg.setdefault('scan_gain_m',None)
        self.cfg.setdefault('scan_count_stop',False)
        self.intercepts={ch:0 for ch in self.tracks}
    def select_route(self,nodes):
        route=super().select_route(nodes)
        if not self.cfg['strong_route']:return route
        lookup={key:i for i,(key,p) in enumerate(nodes)}
        indices,certificate=optimize([p for _,p in nodes],self.s.position,[lookup[k] for k,p in
route],self.cfg['route_exact_limit'])
        chosen=[nodes[i] for i in indices]
        self.route_suffix=[key for key,p in chosen]
        self.events.append(dict(method='predicted_route_optimization',start=self.s.position,
            nodes=nodes,order=indices,certificate=certificate))
        return chosen
    def scan(self,p,force=False):
        threshold=self.cfg['scan_gain_m']
        if threshold is None and not self.cfg['scan_count_stop']:return super().scan(p,force)
        if not force and self.scanned and min(distance(p,q) for q in
self.scanned)<self.cfg['scan_spacing']:return
        self.scanned.append(tuple(p))
        for ch,t in sorted(self.tracks.items(),key=lambda kv:(kv[0]!=self.s.channel,kv[0])):
            if t.cleared:continue
            if self.cfg['scan_count_stop'] and not t.observations and sum(bool(z.observations) for z in
self.tracks.values())==16:continue
            if self.cfg['skip_impossible'] and t.poly and not t.inconsistent and
point_region_distance(p,t.poly)>1500.00001:continue
            if t.circle and t.circle[1]<19.99:continue
            if t.observations and min(distance(p,o['position']) for o in t.observations)<60:continue
            if threshold is not None and t.observations and not t.inconsistent and
self.predicted_gain(t,p)<threshold:

self.events.append(dict(method='optional_known_scan_skipped',channel=ch,position=p,threshold_m=threshold))
                continue
            self.observe(ch,p)
        if sum(bool(t.observations) for t in self.tracks.values())==16:self.count_stop=True;self.todo=[]
        elif self.cfg['adaptive_cover']:self.prune()
    def intercept(self,key,goal):
        threshold=self.cfg['intercept_gain_m']
        if not math.isfinite(threshold) or self.cfg['intercept_scans_only'] and key[0]!='scan':return False
        start=self.s.position;dx,dy=goal[0]-start[0],goal[1]-start[1];length=math.hypot(dx,dy)
        if length<150:return False
        choices=[]
        for t in self.tracks.values():
            if t.cleared or not t.circle or t.inconsistent or
self.intercepts[t.channel]>=self.cfg['intercept_cap']:continue
            center,radius=t.circle
            if radius<80:continue
            fraction=((center[0]-start[0])*dx+(center[1]-start[1])*dy)/(length*length)
            fractions=sorted(set([.15,.3,.5,.7,.85,max(.1,min(.9,fraction))]))
            for alpha in fractions:
                if alpha*length<60 or (1-alpha)*length<30:continue
                p=(start[0]+alpha*dx,start[1]+alpha*dy)
                score=self.predicted_gain(t,p)
                if score>=threshold:choices.append((score,-alpha,-t.channel,p,t))
        if not choices:return False
        score,negative_alpha,_,point,t=max(choices,key=lambda row:row[:3])
        self.intercepts[t.channel]+=1
```

6 问题三四_信息费用调度.py 已知频道扫描与路线控制

```
        self.events.append(dict(method='enroute_measure_choice',channel=t.channel,position=point,
            planned_next=key,planned_endpoint=goal,start=start,predicted_radius_gain_m=score,
            interpolation_fraction=-negative_alpha,attempt=self.intercepts[t.channel],
            note='Zero geometric detour relative to this predicted segment, not a total-cost guarantee'))
        self.observe(t.channel,point)
        return True
    def run(self):
        self.scan((0.,0.),True)
        self.todo=[(i,p) for i,p in self.todo if distance(p,(0.,0.))>1e-7]
        while self.todo or any(t.observations and not t.cleared for t in self.tracks.values()):
            self.gather()
            if not self.todo and not any(t.observations and not t.cleared for t in self.tracks.values()):
                break
            pending=[t for t in self.tracks.values() if t.observations and not t.cleared]
            if self.todo:pending=[t for t in pending if (t.near is not None or t.circle and
t.circle[1]<=self.cfg['ready_radius']) and not self.defer(t)]
            nodes=[(('scan',i),p) for i,p in self.todo]
            if self.cfg['joint'] or not nodes:nodes += [(('target',t.channel),self.target_point(t)) for t in
pending]
            if not nodes:raise RuntimeError('Pending state has no legal macro action')
            key,p=self.select_route(nodes)[0]
            if self.intercept(key,p):continue
            if key[0]=='scan':self.todo=[(i,q) for i,q in self.todo if i!=key[1]];self.scan(p,True)
            else:self.resolve(self.tracks[key[1]])
        self.coverage_complete=not self.todo
        cleared=sorted(ch for ch,t in self.tracks.items() if t.cleared)
        absent=sorted(ch for ch,t in self.tracks.items() if not t.observations)
        complete=not self.todo and len(cleared)+len(absent)==20 and 10<=len(cleared)<=16 and not
any(t.inconsistent for t in self.tracks.values())
        return dict(termination='coverage_complete' if complete else
'incomplete',cleared_channels=cleared,absent_channels=absent,
            coverage_evidence=dict(kind='source_count_upper_bound' if self.count_stop else
'continuous_actual_sites' if self.cfg['adaptive_cover'] else 'fixed_sites',

problem=self.problem,design=self.cfg['cover'],sites=self.fixed,actual_scan_sites=self.scanned,measurements_per
_channel=self.measured),
            geometry_inconsistencies=[ch for ch,t in self.tracks.items() if
t.inconsistent],events=self.events,
            tracks=[t.snapshot() for t in self.tracks.values() if t.observations])
class Q3(ControlMixin,CorrectedMixin,Cost5Q3):pass
class Q4(ControlMixin,CorrectedMixin,Cost5Q4):pass
def solve(session,problem,settings=None):return (Q3 if problem=='q3' else
Q4)(session,problem,m020_settings(problem) if settings is None else settings).run()
```

7 问题三四_光学清除覆盖.py 有限光学覆盖后备

```
"""为观测产生的保守多边形建立有限光学覆盖。
覆盖整个旋转外接矩形，不凭抽样相交删除网格。几何不一致或覆盖未成功时，调用方仍保留原 153 点条带后备。"""
import math


def _finite_point(point):
    return (len(point) == 2 and all(isinstance(x, (int, float))
                                    and math.isfinite(x) for x in point))


def clear_cover(track, start=None, radius=19.99):
    """返回光学点集与覆盖证书；无效多边形返回空值。
    依据当前多边形及机器狗位置比较坐标轴、行列数和蛇形方向，以移动/5 加 3 倍尝试数估计全失败费用。每格半对角线不超过工作半径且小于 20 米，覆盖保证独立于费用排名。"""
    if not math.isfinite(radius) or not 0 < radius < 20:
        raise ValueError('Require a finite optical-cover radius strictly between 0 and 20')
    if (getattr(track, 'inconsistent', True) or getattr(track, 'cleared', True)
            or not getattr(track, 'poly', None) or not getattr(track, 'first', None)):
        return None
    polygon = [tuple(p) for p in track.poly]
    if not all(_finite_point(p) for p in polygon):
        return None
    origin, bearing = track.first
    if not _finite_point(origin) or not math.isfinite(bearing):
        return None
    if start is None:
        start = origin
    if not _finite_point(start):
        raise ValueError('Require a finite actual starting position')
    start = tuple(start)
    angle = math.radians(bearing)
    axes = [('first_bearing', (math.cos(angle), math.sin(angle)))]
    farthest = None
    maximum = 0.
    for i, p in enumerate(polygon):
        for q in polygon[i+1:]:
            length = math.hypot(p[0]-q[0], p[1]-q[1])
            if length > maximum:
                maximum = length
                farthest = (p, q)
    if farthest is not None and maximum > 1e-10:
        p, q = farthest
        axes.append(('longest_vertex_chord', ((q[0]-p[0])/maximum,
                                              (q[1]-p[1])/maximum)))
    padding = 1e-5
    working_radius = radius-1e-6
    if working_radius <= 0:
        raise ValueError('Radius is smaller than the numerical coverage margin')
    best = None
    for axis_name, u in axes:
        v = (-u[1], u[0])
        local = [((p[0]-origin[0])*u[0]+(p[1]-origin[1])*u[1],
                  (p[0]-origin[0])*v[0]+(p[1]-origin[1])*v[1]) for p in polygon]
        lo_x, hi_x = min(p[0] for p in local)-padding, max(p[0] for p in local)+padding
        lo_y, hi_y = min(p[1] for p in local)-padding, max(p[1] for p in local)+padding
        width, height = hi_x-lo_x, hi_y-lo_y
        # 每行高度不超过工作半径时，横向已有充分覆盖空间；
        # 为取得有效网格无需再增加超过两行。
        # 候选点数按启发式费用选择，完整性由覆盖条件单独保证：
        # 每个参与比较的网格都覆盖整个外接矩形。
        maximum_rows = max(1, math.ceil(height/working_radius)+2)
        for rows in range(1, maximum_rows+1):
            dy = height/rows
            remaining = working_radius**2-(dy*.5)**2
            if remaining <= 0:
                continue
            max_dx = 2*math.sqrt(remaining)
            columns = max(1, math.ceil(width/max_dx))
            dx = width/columns
            half_diagonal = math.hypot(dx, dy)*.5
            if half_diagonal > radius:
                continue
            count = rows*columns
```

7 问题三四_光学清除覆盖.py 有限光学覆盖后备

```
            internal_lengths = {
                'rows': rows*(columns-1)*dx+(rows-1)*dy,
                'columns': columns*(rows-1)*dy+(columns-1)*dx}
            for snake_axis, internal_length in internal_lengths.items():
                for flip_x in (False, True):
                    for flip_y in (False, True):
                        x = hi_x-dx*.5 if flip_x else lo_x+dx*.5
                        y = hi_y-dy*.5 if flip_y else lo_y+dy*.5
                        entry = (origin[0]+x*u[0]+y*v[0], origin[1]+x*u[1]+y*v[1])
                        length = math.hypot(entry[0]-start[0], entry[1]-start[1])+internal_length
                        cost = length/5+3*count
                        key = (cost, count, length, axis_name, rows, columns,
                               snake_axis, flip_x, flip_y)
                        if best is None or key < best['key']:
                            best = dict(key=key, axis_name=axis_name, u=u, v=v,
                                        bounds=(lo_x, hi_x, lo_y, hi_y), rows=rows,
                                        columns=columns, dx=dx, dy=dy,
                                        half_diagonal=half_diagonal,
                                        snake_axis=snake_axis,
                                        flip_x=flip_x, flip_y=flip_y,
                                        nominal_length=length)
    if best is None:
        return None
    nx, ny = best['columns'], best['rows']
    if best['snake_axis'] == 'rows':
        indices = [(j, k) for k in range(ny)
                   for j in (range(nx) if k % 2 == 0 else range(nx-1, -1, -1))]
    else:
        indices = [(j, k) for j in range(nx)
                   for k in (range(ny) if j % 2 == 0 else range(ny-1, -1, -1))]
    lo_x, hi_x, lo_y, hi_y = best['bounds']
    u, v = best['u'], best['v']
    points = []
    for j, k in indices:
        if best['flip_x']:
            j = nx-1-j
        if best['flip_y']:
            k = ny-1-k
        x, y = lo_x+(j+.5)*best['dx'], lo_y+(k+.5)*best['dy']
        points.append((origin[0]+x*u[0]+y*v[0], origin[1]+x*u[1]+y*v[1]))
    if not all(_finite_point(p) for p in points):
        return None
    length = sum(math.hypot(p[0]-q[0], p[1]-q[1])
                 for p, q in zip([start]+points[:-1], points))
    certificate = dict(kind='complete_rotated_rectangle_cells',
                       channel=track.channel, source_polygon_before=polygon,
                       origin=tuple(origin), axis_name=best['axis_name'],
                       axis_u=u, axis_v=v, bounds_local=best['bounds'],
                       rows=ny, columns=nx, point_count=len(points),
                       cell_width_m=best['dx'], cell_height_m=best['dy'],
                       cell_half_diagonal_m=best['half_diagonal'],
                       certified_radius_m=radius,
                       rectangle_padding_m=padding,
                       snake_axis=best['snake_axis'], flip_x=best['flip_x'],
                       flip_y=best['flip_y'], start=start,
                       complete_visit_length_m=length,
                       complete_failure_cost_s=length/5+3*len(points),
                       removed_cells=0)
    return dict(points=points, certificate=certificate)
```

8 问题三四_连续覆盖构造.py 连续覆盖构造和证书

```
"""不读取真值的有限定向覆盖构造及连续证书。
证书覆盖完整源圆域；坐标以整数毫米记录，检查只用整数平方距离和叉积，不导入模拟器。"""
from __future__ import annotations

import argparse
import hashlib
import itertools
import json
import math
from pathlib import Path


def ring_points(rings, include_center=True):
    """每条环参数依次为点数、半径米数、起始角度。"""
    points = [(0., 0.)] if include_center else []
    for number, radius, phase in rings:
        for k in range(number):
            a = math.radians(phase) + 2 * math.pi * k / number
            points.append((round(radius * math.cos(a), 3),
                           round(radius * math.sin(a), 3)))
    return points


def route_length(points, start=(0., 0.)):
    ans = 0.
    for p in points:
        ans += math.dist(p, start)
        start = p
    return ans


def open_route(points, start=(0., 0.)):
    """多起点近邻路线加开放 2-opt 改进，不宣称整体最优。"""
    points = list(dict.fromkeys(map(tuple, points)))
    if not points:
        return []

    def improve(route):
        while True:
            best, change = -1e-7, None
            for i in range(len(route) - 1):
                a, b = (start if i == 0 else route[i-1]), route[i]
                for j in range(i + 1, len(route)):
                    c = route[j]
                    delta = math.dist(a, c) - math.dist(a, b)
                    if j + 1 < len(route):
                        d = route[j+1]
                        delta += math.dist(b, d) - math.dist(c, d)
                    if delta < best:
                        best, change = delta, (i, j)
            if change is None:
                return route
            i, j = change
            route[i:j+1] = reversed(route[i:j+1])

    candidates = [improve(points[:]), improve(list(reversed(points)))]
    for first in points:
        todo = set(points)
        todo.remove(first)
        route = [first]
        while todo:
            nxt = min(todo, key=lambda p: (math.dist(route[-1], p), p))
            route.append(nxt)
            todo.remove(nxt)
        candidates.append(improve(route))
    return min(candidates, key=lambda r: (route_length(r, start), r))


def _cross(a, b, c):
    return (b[0]-a[0])*(c[1]-a[1]) - (b[1]-a[1])*(c[0]-a[0])


def _hull(indices, points):
```

8 问题三四_连续覆盖构造.py 连续覆盖构造和证书

```
    order = sorted(set(indices), key=lambda i: points[i])
    lower, upper = [], []
    for i in order:
        while len(lower) >= 2 and _cross(points[lower[-2]], points[lower[-1]], points[i]) <= 0:
            lower.pop()
        lower.append(i)
    for i in reversed(order):
        while len(upper) >= 2 and _cross(points[upper[-2]], points[upper[-1]], points[i]) <= 0:
            upper.pop()
        upper.append(i)
    return lower[:-1] + upper[:-1]


def _corners(box):
    x0, y0, x1, y1 = box
    return [(x0,y0), (x1,y0), (x1,y1), (x0,y1)]


def _outside(box, radius):
    x0,y0,x1,y1 = box
    dx = max(x0, -x1, 0)
    dy = max(y0, -y1, 0)
    return dx*dx + dy*dy > radius*radius


def _split(box):
    x0,y0,x1,y1 = box
    xm,ym = (x0+x1)//2, (y0+y1)//2
    return [(x0,y0,xm,ym),(xm,y0,x1,ym),(xm,ym,x1,y1),(x0,ym,xm,y1)]


def _box_witness(box, points, reception):
    corners = _corners(box)
    ids = [i for i,s in enumerate(points)
           if max((s[0]-c[0])**2+(s[1]-c[1])**2 for c in corners) <= reception*reception]
    if len(ids) < 3:
        return None
    hull = _hull(ids, points)
    if len(hull) < 3:
        return None
    # 支撑直线留 2 毫米余量，也覆盖任意
    # 实际测站取整到毫米引入的不超过√2/2 毫米误差。
    if all(_cross(points[i], points[j], c) > 0 and
           _cross(points[i], points[j], c)**2 >
           4*((points[i][0]-points[j][0])**2+(points[i][1]-points[j][1])**2)
           for i,j in zip(hull, hull[1:]+hull[:1]) for c in corners):
        return hull
    return None


def directional_certificate(points, source_radius=1800., reception_radius=999.99,
                            minimum_box_m=1., max_nodes=1000000):
    """返回有限格证书或明确列出未认证格。
    失败只表示未取得此充分证书，不自动构成反例。通过的格严格位于局部凸包内，接收半径略小于 1000 米以保留余量。"""
    pts = [tuple(round(v*1000) for v in p) for p in points]
    radius, reception = round(source_radius*1000), round(reception_radius*1000)
    side = 1
    while side < radius:
        side *= 2
    root = (-side,-side,side,side)
    queue, leaves, unresolved, count = [root], [], [], 0
    min_mm = round(minimum_box_m*1000)
    while queue:
        box = queue.pop()
        count += 1
        if _outside(box,radius):
            leaves.append({'box':box,'outside':True})
            continue
        witness = _box_witness(box, pts, reception)
        if witness is not None:
            leaves.append({'box':box,'hull':witness})
        elif max(box[2]-box[0],box[3]-box[1]) <= min_mm or count >= max_nodes:
            unresolved.append(box)
```

8 问题三四_连续覆盖构造.py 连续覆盖构造和证书

```
        else:
            queue.extend(_split(box))
    return {'schema':'m019-directional-integer-box-v1', 'passed':not unresolved,
            'scale':1000,'source_radius':radius,'reception_radius':reception,
            'points_mm':pts,'root':root,'visited_nodes':count,'leaves':leaves,
            'unresolved':unresolved}


def certify(points, problem='q4', minimum_box_m=16., max_nodes=12000,
            fail_fast=True):
    """为任意实际测站集合快速生成保守在线证书。
    返回是否通过、未认证格及格中心。失败不等于找到真实漏检源，调用方可保留旧后备站。坐标取整到毫米的误差不超过 0.707107 毫米；接收距离留 10 毫米、凸包边留 2 毫米余量，兼顾原坐标。默认最小格边长 16 米，节点预算只限制核验工作量，用尽时返回未认证。"""
    if str(problem).lower() not in ('3','4','q3','q4'):
        raise ValueError('problem must be 3, 4, q3 or q4')
    omni = str(problem).lower() in ('3','q3')
    pts = [tuple(round(v*1000) for v in p) for p in points]
    radius,reception,side = 1800000,999990,2097152
    queue = [(-side,-side,side,side)]
    count, covered, unresolved = 0,0,[]
    minimum = round(minimum_box_m*1000)
    while queue:
        box = queue.pop()
        count += 1
        if _outside(box,radius):
            continue
        if omni:
            cs = _corners(box)
            passed = any(max((s[0]-c[0])**2+(s[1]-c[1])**2 for c in cs)
                         <= reception*reception for s in pts)
        else:
            passed = _box_witness(box,pts,reception) is not None
        if passed:
            covered += 1
        elif max(box[2]-box[0],box[3]-box[1]) <= minimum or count >= max_nodes:
            unresolved.append(box)
            if fail_fast:
                break
        else:
            queue.extend(_split(box))
    return {'passed':not unresolved and not queue,'problem':'q3' if omni else 'q4',
            'visited_nodes':count,'covered_cells':covered,'unresolved':unresolved,
            'gap_points':[((b[0]+b[2])/2000.,(b[1]+b[3])/2000.) for b in unresolved],
            'certificate_type':'integer_box_local_disk' if omni else 'integer_box_local_hull',
            'minimum_box_m':minimum_box_m}


def verify_certificate(certificate):
    """独立遍历分区并核对给定见证。
    不调用凸包生成器或原格检查函数，以整数运算检查凸性、逆时针方向及每个角点的严格内含。"""
    c = certificate
    if not c['passed'] or c['unresolved']:
        return False
    pts, radius, reception = c['points_mm'],c['source_radius'],c['reception_radius']
    rows = {tuple(x['box']):x for x in c['leaves']}
    if len(rows) != len(c['leaves']):
        return False
    root = tuple(c['root'])
    if root[0] > -radius or root[1] > -radius or root[2] < radius or root[3] < radius:
        return False
    stack, used, traversed = [root],set(),0
    while stack:
        box = stack.pop()
        traversed += 1
        if traversed > 4*len(rows)+1:
            return False
        row = rows.get(box)
        if row is None:
            if box[2]-box[0] <= 1 or box[3]-box[1] <= 1:
                return False
            stack.extend(_split(box))
            continue
```

8 问题三四_连续覆盖构造.py 连续覆盖构造和证书

```
        used.add(box)
        if row.get('outside'):
            if not _outside(box,radius):
                return False
            continue
        ids = row['hull']
        if len(ids)<3 or len(set(ids))!=len(ids) or any(i<0 or i>=len(pts) for i in ids):
            return False
        polygon = [pts[i] for i in ids]
        if any(_cross(a,b,p) < 0 for a,b in zip(polygon,polygon[1:]+polygon[:1])
               for p in polygon):
            return False
        for p in polygon:
            for corner in _corners(box):
                if (p[0]-corner[0])**2+(p[1]-corner[1])**2 > reception*reception:
                    return False
        for a,b in zip(polygon,polygon[1:]+polygon[:1]):
            for corner in _corners(box):
                cross = _cross(a,b,corner)
                if cross <= 0 or cross*cross <= 4*((a[0]-b[0])**2+(a[1]-b[1])**2):
                    return False
    return used == set(rows)


def sampled_margin(points, probes):
    """仅作探索：在有限探测点上计算局部凸包的有符号余量。"""
    import numpy as np
    s,p = np.asarray(points,float),np.asarray(probes,float)
    diff = s[None,:,:] - p[:,None,:]
    distance = np.linalg.norm(diff,axis=2)
    angle = np.arctan2(diff[:,:,1],diff[:,:,0])
    angle[distance > 999.99] = np.nan
    angle.sort(axis=1)
    count = np.sum(np.isfinite(angle),axis=1)
    angle[~np.isfinite(angle)] = 1e9
    gap = np.diff(angle,axis=1)
    gap[angle[:,1:] > 1e8] = 0
    maximum = np.max(gap,axis=1)
    last = angle[np.arange(len(p)),np.maximum(count-1,0)]
    maximum = np.maximum(maximum,2*math.pi + angle[:,0]-last)
    maximum[count < 3] = 2*math.pi
    return float(np.min(math.pi-maximum)),p[np.argmin(math.pi-maximum)].tolist()


def disk_probes(spacing=30.):
    import numpy as np
    q = np.arange(-1800,1800.001,spacing)
    x,y = np.meshgrid(q,q)
    inside = x*x+y*y <= 1800**2
    theta = np.arange(0,2*math.pi,.004)
    return np.concatenate([np.column_stack([x[inside],y[inside]]),
                           np.column_stack([1800*np.cos(theta),1800*np.sin(theta)])])


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--points',type=Path,required=True)
    parser.add_argument('--output',type=Path,required=True)
    parser.add_argument('--minimum-box-m',type=float,default=1.)
    args = parser.parse_args()
    raw = json.loads(args.points.read_text(encoding='utf8'))
    points = raw['points'] if isinstance(raw,dict) else raw
    cert = directional_certificate(points,minimum_box_m=args.minimum_box_m)
    cert['independent_verifier_passed'] = verify_certificate(cert)
    route = open_route(points)
    cert['route'] = route
    cert['route_length_m'] = route_length(route)
    cert['source_sha256'] = hashlib.sha256(Path(__file__).read_bytes()).hexdigest()
    args.output.parent.mkdir(parents=True,exist_ok=True)
    args.output.write_text(json.dumps(cert,indent=2),encoding='utf8')
    print(json.dumps({k:cert[k] for k in
['passed','visited_nodes','independent_verifier_passed','route_length_m']}))
    print('unresolved_cells',len(cert['unresolved']),'leaves',len(cert['leaves']))
```

8 问题三四_连续覆盖构造.py 连续覆盖构造和证书

```
if __name__ == '__main__':
    main()
```

9 问题三_主动测点选择.py 带接收保障的主动选点

```
"""仅依观测，为全向源选择具有接收证书的探针。
角度区间是完整响应外包，不是未来观测的抽样；每个区间先扩张角楔再界定后验。只在有限候选中作启发式费用比较，不是全局连续极小极大求解。"""
import math

from 问题一_定位区域与包围圆 import EPS, bearing_planes, clip, diameter, distance, point_region_distance


def _axes(poly):
    _, pair = diameter(poly)
    length = distance(*pair)
    if length <= EPS:
        return (1., 0.), (0., 1.)
    u = ((pair[1][0]-pair[0][0])/length, (pair[1][1]-pair[0][1])/length)
    return u, (-u[1], u[0])


def _box_radius(poly, axes):
    spans = []
    for u in axes:
        values = [p[0]*u[0]+p[1]*u[1] for p in poly]
        spans.append(max(values)-min(values))
    return .5*math.hypot(*spans) + 10*EPS


def bearing_support(poly, point, delta):
    """包含全部可能示向读数的连续角度区间。"""
    if point_region_distance(point, poly) <= 10*EPS:
        return 0., 360.
    angles = sorted(math.degrees(math.atan2(p[1]-point[1], p[0]-point[0])) % 360
                    for p in poly)
    gaps = [((angles[(i+1) % len(angles)]-a) % 360, i)
            for i, a in enumerate(angles)]
    gap, index = max(gaps)
    # 单点只对应一条射线，不覆盖整周角度。
    span = 0. if len(angles) == 1 else 360.-gap
    start = angles[(index+1) % len(angles)]
    return start-delta, start+span+delta


def posterior_radius_bound(poly, point, delta=1.005, bin_width_deg=2., prior_radius=None):
    """为全部未来正常示向及近距响应计算包围半径上界。
    区间内任一示向角楔都被中心示向加区间半宽的角楔包含；与当前域求交后用矩形包围圆界定。各区间与 5 米近距圆的最大值覆盖连续响应空间，旧包围圆可继续收紧上界。无信号不在该界内，接收保证需另查。"""
    if not poly or not 0 < bin_width_deg <= 30 or not 0 < delta < 45:
        raise ValueError('Invalid polygon or angular interval width')
    axes = _axes(poly)
    start, stop = bearing_support(poly, point, delta)
    count = max(1, int(math.ceil((stop-start)/bin_width_deg)))
    width = (stop-start)/count
    bound = min(5., prior_radius) if prior_radius is not None else 5.
    nonempty = 0
    for i in range(count):
        beta = start+(i+.5)*width
        region = list(poly)
        for plane in bearing_planes(point, beta, delta+.5*width+1e-9):
            region = clip(region, plane)
        if not region:
            continue
        nonempty += 1
        a = math.radians(beta)
        rotated = ((math.cos(a), math.sin(a)), (-math.sin(a), math.cos(a)))
        radius = min(_box_radius(region, axes), _box_radius(region, rotated))
        if prior_radius is not None:
            radius = min(radius, prior_radius)
        bound = max(bound, radius)
    return bound, dict(interval_count=count, nonempty_intervals=nonempty,
                       reported_bearing_interval_deg=[start, stop],
                       actual_max_bin_width_deg=width)


def choose_probe(track, position, config):
    """返回候选点和诊断；不适用时返回空点与原因。
```

9 问题三_主动测点选择.py 带接收保障的主动选点

```
    角度分箱默认 2°，剩余不确定性惩罚默认 60 秒。该透明评分只比较候选，不代表预测或保证的完整结束时间。"""
    if config.get('problem_scope') != 'omni':
        return None, {'status': 'requires_omni_scope'}
    if track.cleared or track.near is not None or track.inconsistent or not track.poly or not track.circle:
        return None, {'status': 'track_not_eligible'}
    options = config.get('policy_options', {})
    bin_width = options.get('active_probe_bin_deg', 2.)
    penalty = options.get('active_probe_penalty_s', 60.)
    if (isinstance(bin_width, bool) or not isinstance(bin_width, (int, float)) or
            not math.isfinite(bin_width) or not .25 <= bin_width <= 10):
        raise ValueError('active_probe_bin_deg must be in [0.25,10]')
    if (isinstance(penalty, bool) or not isinstance(penalty, (int, float)) or
            not math.isfinite(penalty) or not 0 <= penalty <= 1000):
        raise ValueError('active_probe_penalty_s must be in [0,1000]')
    center, radius = track.circle
    target_radius = 20.-config['clearance_margin_m']
    if radius <= target_radius:
        return None, {'status': 'already_certified_clear'}
    u, v = _axes(track.poly)
    points = [tuple(position), tuple(center)]
    for offset in (25., 50., 100., 200., 400., 600.):
        points.extend((center[0]+sign*offset*v[0], center[1]+sign*offset*v[1])
                      for sign in (-1, 1))
    # 当多边形主轴
    # 已偏离首测示向时，仍将旧启发式保留在候选中。
    if track.first:
        a = math.radians(track.first[1]+90)
        offset = options.get('refine_offset_m', 100.)
        points.extend((center[0]+sign*offset*math.cos(a), center[1]+sign*offset*math.sin(a))
                      for sign in (-1, 1))
    candidates = []
    rejected = 0
    for point in sorted(set(points)):
        farthest = max(distance(point, p) for p in track.poly)
        if farthest > 1000.-1e-5:
            rejected += 1
            continue
        bound, details = posterior_radius_bound(track.poly, point, config['angle_bound_deg'],
                                                bin_width, radius)
        move = distance(position, point)
        score = move/5.+5.+penalty*max(0., bound/target_radius-1.)
        candidates.append(dict(point=point, farthest_vertex_m=farthest,
                               guaranteed_reception=True, radius_upper_bound_m=bound,
                               one_probe_clear_certificate=bound <= target_radius,
                               movement_m=move, heuristic_score_s=score, **details))
    if not candidates:
        return None, {'status': 'no_reception_certified_candidate', 'rejected_count': rejected}
    best = min(candidates, key=lambda c: (c['heuristic_score_s'], c['radius_upper_bound_m'],
                                          c['movement_m'], c['point']))
    return best['point'], dict(status='selected', method='interval_enclosure_active_probe',
                               candidate_count=len(candidates), rejected_count=rejected,
                               prior_radius_m=radius, penalty_s=penalty,
                               selected=best, candidates=candidates)
```

10 问题三四_巡检点与路线生成.py 基础覆盖与访问次序

```
"""固定覆盖构造与确定性开放路线改进。
测站在观测前确定，不使用隐藏场景信息。保留的 45 点构造以 690 米间距包夹 1800 米圆域，满足 690 小于 1000/√2。"""
import math


def route_length(points, start=(0., 0.)):
    total = 0.
    for point in points:
        total += math.hypot(point[0] - start[0], point[1] - start[1])
        start = point
    return total


def two_opt_open(points):
    """仅接受严格缩短路线的片段翻转，起点固定、终点自由。"""
    route = list(points)
    while True:
        best_delta, best_pair = -1e-8, None
        for i in range(len(route) - 1):
            a = (0., 0.) if i == 0 else route[i - 1]
            b = route[i]
            for j in range(i + 1, len(route)):
                c = route[j]
                delta = math.dist(a, c) - math.dist(a, b)
                if j + 1 < len(route):
                    d = route[j + 1]
                    delta += math.dist(b, d) - math.dist(c, d)
                if delta < best_delta:
                    best_delta, best_pair = delta, (i, j)
        if best_pair is None:
            return route
        i, j = best_pair
        route[i:j + 1] = reversed(route[i:j + 1])


def improved_route(sites):
    # 将原路线保留为候选，保证本次固定测站的名义路长不增。
    candidates = [list(sites), two_opt_open(sites), two_opt_open(list(reversed(sites)))]
    remaining = set(sites)
    position = (0., 0.)
    greedy = []
    while remaining:
        target = min(remaining, key=lambda p: (math.dist(position, p), p))
        remaining.remove(target)
        greedy.append(target)
        position = target
    candidates.append(two_opt_open(greedy))
    return min(candidates, key=lambda r: (route_length(r), r))


def coverage_sites(config):
    design = config.get('coverage_design', 'square64')
    if design == 'triangular':
        from 问题四_三角网格覆盖 import triangular_mesh
        points, _ = triangular_mesh(config['triangular_side_m'], config['triangular_phase'])
        return improved_route(points) if config.get('scan_route') == 'two_opt' else points
    if design in ('omni7', 'omni13', 'omni7_compact'):
        n = 12 if design == 'omni13' else 6
        radius = config['omni_ring_radius_m'] if design == 'omni7_compact' else 1500
        points = [(0., 0.)] + [(radius*math.cos(2*math.pi*k/n), radius*math.sin(2*math.pi*k/n)) for k in
range(n)]
        return improved_route(points) if config.get('scan_route') == 'two_opt' else points
    coords = config['grid_coordinates_m']
    points = [(x, y) for i, y in enumerate(coords)
              for x in (coords if i % 2 == 0 else list(reversed(coords)))]
    if config.get('coverage_design', 'square64') == 'trimmed45':
        points = [p for p in points if not (abs(p[0]) == 2070 and abs(p[1]) == 2070)]
    if config.get('scan_route', 'serpentine') == 'two_opt':
        points = improved_route(points)
    return points
```

11 问题三四_测点与清除规则.py 可选动作的几何与次数约束

```
"""仅依观测的可选调度规则，不依赖模拟器。"""
import math

from 问题三四_巡检点与路线生成 import two_opt_open
from 问题一_定位区域与包围圆 import distance, unit, point_region_distance


def validate_options(config):
    options = config.get('policy_options', {})
    allowed = {'early_detour_m', 'clear_order', 'refine_steps', 'refine_offset_m',
               'opportunistic', 'dynamic_scan', 'safe_clear_shift', 'failed_probe_threshold',
               'refine_policy', 'active_probe_bin_deg', 'active_probe_penalty_s',
               'iterative_probe_rounds', 'pursue_detour_m', 'shared_clear', 'count_stop_unsettled'}
    if not isinstance(options, dict) or set(options) - allowed:
        raise ValueError('Unknown policy options')
    if options and config['strategy'] != 'geometry':
        raise ValueError('Policy options require geometry')
    if type(options.get('shared_clear', False)) is not bool:
        raise ValueError('shared_clear must be boolean')
    limit = options.get('count_stop_unsettled')
    if limit is not None and (type(limit) is not int or not 0 <= limit <= 16):
        raise ValueError('count_stop_unsettled must be zero to sixteen, or null')
    if options.get('refine_policy', 'lateral') not in ('lateral', 'active_interval'):
        raise ValueError('Unknown refinement policy')
    if options.get('refine_policy') == 'active_interval' and config.get('problem_scope') != 'omni':
        raise ValueError('Active interval refinement requires explicit omni scope')
    for key, default, low, high in (('active_probe_bin_deg', 2, .25, 10),
                                    ('active_probe_penalty_s', 60, 0, 1000),
                                    ('pursue_detour_m', 0, 0, 1000)):
        value = options.get(key, default)
        if isinstance(value, bool) or not isinstance(value, (int,float)) or not math.isfinite(value) or not
low <= value <= high:
            raise ValueError('Invalid '+key)
    rounds = options.get('iterative_probe_rounds', 0)
    if type(rounds) is not int or not 0 <= rounds <= 6:
        raise ValueError('Iterative reprobe supports zero to six rounds')
    if options.get('pursue_detour_m', 0) and (config.get('problem_scope') != 'omni' or not
options.get('refine_steps', 0)):
        raise ValueError('Early pursuit requires omni scope and enabled refinement')
    threshold = options.get('failed_probe_threshold', 0)
    if type(threshold) is not int or not 0 <= threshold <= 153:
        raise ValueError('Invalid failed-probe threshold')
    d = options.get('early_detour_m', 0)
    if isinstance(d, bool) or not isinstance(d, (int, float)) or not math.isfinite(d) or not 0 <= d <= 1000:
        raise ValueError('Invalid early detour budget')
    if options.get('clear_order', 'greedy') not in ('greedy', 'two_opt', 'exact12'):
        raise ValueError('Unknown clear order')
    n = options.get('refine_steps', 0)
    if type(n) is not int or not 0 <= n <= 2:
        raise ValueError('Refinement supports zero, one or two probes')
    r = options.get('refine_offset_m', 100)
    if isinstance(r, bool) or not isinstance(r, (int, float)) or not math.isfinite(r) or not 20 <= r <= 500:
        raise ValueError('Invalid refinement offset')
    if type(options.get('safe_clear_shift', False)) is not bool:
        raise ValueError('safe_clear_shift must be boolean')
    if type(options.get('dynamic_scan', False)) is not bool:
        raise ValueError('dynamic_scan must be boolean')
    if type(options.get('opportunistic', False)) is not bool:
        raise ValueError('opportunistic must be boolean')


def settled(track, margin):
    return track.cleared or track.near is not None or bool(
        track.circle is not None and track.circle[1] <= 20-margin)


def ordered_targets(tracks, start, target):
    """近邻初始化后作开放 2-opt，坐标重合时仍保留不同任务身份。"""
    remaining = list(tracks)
    greedy = []
    position = start
    while remaining:
        t = min(remaining, key=lambda t: (distance(position, target(t)), t.channel))
```

11 问题三四_测点与清除规则.py 可选动作的几何与次数约束

```
        remaining.remove(t)
        greedy.append(t)
        position = target(t)
    buckets = {}
    points = []
    for t in greedy:
        p = (target(t)[0]-start[0], target(t)[1]-start[1])
        points.append(p)
        buckets.setdefault(p, []).append(t)
    return [buckets[p].pop(0) for p in two_opt_open(points)]


def refine(session, track, config, events):
    """在估计中心两侧取探针，再保留继承的后备。
    这是名义交会角启发式，不能保证定向源接收；探测前固定中心以约束额外行走。"""
    options = config.get('policy_options', {})
    if not options.get('refine_steps', 0) or settled(track, config['clearance_margin_m']) or not track.circle
or track.inconsistent:
        return
    if options.get('refine_policy') == 'active_interval':
        from 问题三_主动测点选择 import choose_probe
        visited = set()
        for _ in range(options['refine_steps']):
            point, diagnostics = choose_probe(track, session.position, config)
            if point is None or point in visited:
                break
            visited.add(point)
            before = track.circle[1]
            response = session.measure(point, track.channel)
            track.observe(point, response, config['angle_bound_deg'])
            events.append(dict(channel=track.channel, method='active_refinement',
                position=point, result=response['measure_result'], radius_before_m=before,
                radius_after_m=track.circle[1] if track.circle else None,
                selection={k:v for k,v in diagnostics.items() if k != 'candidates'}))
            if settled(track, config['clearance_margin_m']) or track.inconsistent:
                break
        return
    center = track.circle[0]
    u = unit(track.first[1]+90)
    r = options.get('refine_offset_m', 100)
    points = [(center[0]+sign*r*u[0], center[1]+sign*r*u[1]) for sign in (1, -1)]
    points.sort(key=lambda p: (distance(session.position, p), p))
    for p in points[:options['refine_steps']]:
        before = track.circle[1] if track.circle else None
        response = session.measure(p, track.channel)
        track.observe(p, response, config['angle_bound_deg'])
        events.append(dict(channel=track.channel, method='local_refinement',
                           position=p, result=response['measure_result'], radius_before_m=before,
                           radius_after_m=track.circle[1] if track.circle else None))
        if settled(track, config['clearance_margin_m']) or track.inconsistent:
            break


def exact_targets(tracks, start, target):
    """点身份不超过 12 个时精确求名义开放路线，否则用 2-opt 改进。"""
    tracks = sorted(tracks, key=lambda t: t.channel)
    n = len(tracks)
    if n > 12:
        return ordered_targets(tracks, start, target)
    if not n:
        return []
    points = [target(t) for t in tracks]
    dp = {(1 << j, j): (distance(start, points[j]), -1) for j in range(n)}
    for mask in range(1, 1 << n):
        for j in range(n):
            if not mask & (1 << j) or mask == 1 << j:
                continue
            prev = mask ^ (1 << j)
            dp[mask, j] = min((dp[prev, k][0] + distance(points[k], points[j]), k)
                              for k in range(n) if prev & (1 << k))
    mask = (1 << n)-1
    j = min(range(n), key=lambda j: (dp[mask, j][0], j))
    order = []
```

11 问题三四_测点与清除规则.py 可选动作的几何与次数约束

```
    while mask:
        order.append(tracks[j]); old = j
        j = dp[mask, j][1]; mask ^= 1 << old
    return list(reversed(order))


def safe_clear_point(track, position, config):
    """在保证清除圆的内部充分域中选择最近点，不使用隐藏真值。"""
    if track.near is not None:
        center, radius = track.near, 5.
    elif track.circle is not None:
        center, radius = track.circle
    else:
        return track.first[0]
    slack = max(0., 20-config['clearance_margin_m']-radius)
    length = distance(center, position)
    if not config.get('policy_options', {}).get('safe_clear_shift', False) or not length:
        return center
    factor = min(slack, length)/length
    return (center[0]+factor*(position[0]-center[0]), center[1]+factor*(position[1]-center[1]))


def iterative_reprobe(session, track, config, events):
    """清除失败后，交替在原地测向并移到新中心。
    轮数受配置限制，原有有限后备仍保留。无信号不用于排除位置，定向源也如此；本策略希望缩小定位域，但不保证每轮收缩。"""
    rounds = config.get('policy_options', {}).get('iterative_probe_rounds', 0)
    for index in range(rounds):
        if track.cleared or track.inconsistent or not track.poly or not track.circle:
            break
        point = session.position
        before = track.circle[1]
        response = session.measure(point, track.channel)
        track.observe(point, response, config['angle_bound_deg'])
        events.append(dict(channel=track.channel, method='iterative_probe_measure',
            round=index+1, position=point, result=response['measure_result'],
            radius_before_m=before, radius_after_m=track.circle[1] if track.circle else None))
        if response['measure_result'] == 'no_signal' or track.inconsistent or not track.circle:
            break
        target = safe_clear_point(track, point, config)
        if distance(point, target) <= 1e-5 and track.near is None:
            break
        guaranteed = settled(track, config['clearance_margin_m'])
        track.cleared = session.clear(target, track.channel)['clear_result'] == 'success'
        track.inconsistent |= guaranteed and not track.cleared
        events.append(dict(channel=track.channel, method='iterative_probe_clear',
            round=index+1, guaranteed=guaranteed, success=track.cleared,
            radius_m=track.circle[1]))


def opportunistic_clear(session, tracks, config, events, attempted):
    """在成功清除位置试清其他几何上可能同处的已知频道。
    整域未被光学圆包含时属于非保证试探；没有额外移动，每个频道和位置组合只试一次，失败圆不用于收缩凸多边形。成功位置至多
16 个。"""
    if not config.get('policy_options', {}).get('shared_clear', False):
        return
    point = session.position
    for track in sorted(tracks, key=lambda t:t.channel):
        if track.cleared or track.inconsistent or not track.observations:
            continue
        key = (track.channel, point)
        if key in attempted:
            continue
        if track.poly:
            if point_region_distance(point, track.poly) > 20.+1e-5:
                continue
            guaranteed = max(distance(point,p) for p in track.poly) <= 20.-config['clearance_margin_m']
        elif track.near is not None:
            if distance(point,track.near) > 25.+1e-5:
                continue
            guaranteed = distance(point,track.near)+5 <= 20.-config['clearance_margin_m']
        else:
            continue
        attempted.add(key)
```

11 问题三四_测点与清除规则.py 可选动作的几何与次数约束

```
        track.cleared = session.clear(point,track.channel)['clear_result'] == 'success'
        track.inconsistent |= guaranteed and not track.cleared
        events.append(dict(channel=track.channel, method='shared_position_clear',
            position=point, guaranteed=guaranteed, success=track.cleared))
```

12 问题二_安全第二测点.py 第二测点安全候选

```
"""Q2 利用首次接收信息选择全向源第二测站。
此独立选点不改变 Q3/Q4 在线策略，坐标以米计，输入须为正常示向而非近距响应。"""
import math
from 问题一_定位区域与包围圆 import unit

CHOICES = {'legacy': (750.0, 500.0), 'balanced': (710.0, 555.0),
           'accuracy': (840.0, 548.0)}


def reception_candidate_region(position=(0.0, 0.0), bearing=0.0, delta=1.005):
    """四个闭圆盘的交集，对首测局部环扇区信息给出精确条件。
    首距 r 大于 5 米且不超过 1500 米；首次接收给出 R≥r，因此第二距离须不超过 max(1000,r)。只需检查 r=5、1000 及两条角边界。取 5 米闭包不改变对 r>5 的统一保证；未叠加全局源圆域时，该域对具体首测位置充分，但未必最大。"""
    if not all(math.isfinite(v) for v in (*position, bearing, delta)):
        raise ValueError('Coordinates, bearing and delta must be finite')
    if not 0 < delta < 45:
        raise ValueError('This Q2 construction requires 0 < delta < 45 degrees')
    centers=[]
    for r in (5.0, 1000.0):
        for sign in (-1, 1):
            u=unit(bearing+sign*delta)
            centers.append((position[0]+r*u[0], position[1]+r*u[1]))
    return {'disk_centers': centers, 'disk_radius_m': 1000.0,
            'first_distance_closure_m': [5.0, 1500.0],
            'assumption': 'normal first bearing; omnidirectional; fixed unknown reception radius',
            'scope': 'exact local annular-sector domain, sufficient with global source-disk clipping'}


def select_second_point(position=(0.0, 0.0), bearing=0.0,
                        preference='accuracy', side=1, delta=1.005):
    """返回冻结的 Q2 候选及费用和接收信息。
    精度候选以额外行走换较小的最坏直径；均衡候选保留旧路程预算。两者均未证明全局最优，镜像侧的选择不使用真值。"""
    if preference not in CHOICES:
        raise ValueError('preference must be legacy, balanced or accuracy')
    if side not in (-1, 1):
        raise ValueError('side must be -1 or 1')
    region=reception_candidate_region(position,bearing,delta)
    a,b=CHOICES[preference]
    u,v=unit(bearing),unit(bearing+90)
    p=(position[0]+a*u[0]+side*b*v[0],position[1]+a*u[1]+side*b*v[1])
    slack=1000-max(math.dist(p,c) for c in region['disk_centers'])
    if slack < 1e-6:
        raise ValueError('Selected point has no safety margin at this error bound')
    return {'point_m': p, 'local_point_m': [a,side*b], 'preference':preference,
            'move_distance_m': math.hypot(a,b), 'move_time_s': math.hypot(a,b)/5,
            'disk_constraint_slack_m':slack,'guaranteed_omni_reception':True,
            'near_response_action':'clear immediately; a second bearing is unnecessary',
            'global_optimality_proven':False}
```

13 问题三四_搜索定位清除框架.py 有限后备的基础策略

```
"""仅依观测的基线，保留有限几何后备。
策略不导入模拟器，也不接收源位置、类型或真实数量。"""
import math

from 问题三四_巡检点与路线生成 import coverage_sites, improved_route
from 问题一_定位区域与包围圆 import bearing_planes, clip, distance, enclosing_circle, point_region_distance,
unit
from 问题三四_测点与清除规则 import validate_options, settled, ordered_targets, refine, exact_targets,
safe_clear_point, iterative_reprobe, opportunistic_clear


def validate_config(config):
    validate_options(config)
    if config['strategy'] not in ('fallback', 'geometry'):
        raise ValueError('Unknown strategy')
    design = config.get('coverage_design', 'square64')
    expected = {'square64': list(range(-2100, 2101, 600)),
                'trimmed45': list(range(-2070, 2071, 690)), 'omni7': [], 'omni13': [], 'triangular': [],
'omni7_compact': []}
    if design.startswith('omni') and config.get('problem_scope') != 'omni':
        raise ValueError('Sparse coverage requires explicit omni problem scope')
    if design not in expected or config['grid_coordinates_m'] != expected[design]:
        raise ValueError('Use one of the explicitly certified coverage grids')
    if design == 'triangular':
        from 问题四_三角网格覆盖 import triangular_mesh
        triangular_mesh(config.get('triangular_side_m'), config.get('triangular_phase'))
    if design == 'omni7_compact':
        radius = config.get('omni_ring_radius_m')
        if isinstance(radius,bool) or not isinstance(radius,(int,float)) or not 1125 <= radius <= 1500:
            raise ValueError('Certified compact omni ring radius must be in [1125,1500]')
    if config.get('scan_route', 'serpentine') not in ('serpentine', 'two_opt'):
        raise ValueError('Unknown scan route')
    if not 1.005 <= config['angle_bound_deg'] <= 1.01:
        raise ValueError('Angle bound must include rounding and fit fallback rectangle')
    if not 0 < config['clearance_margin_m'] < 1:
        raise ValueError('Use a small positive clearance safety margin')
    if type(config['min_sources']) is not int or not 0 <= config['min_sources'] <= 10:
        raise ValueError('Invalid minimum count')
    if config['transport_timeout_s'] <= 0 or config['exit_reserve_s'] <= 0 or config['transport_retries'] < 0:
        raise ValueError('Invalid execution configuration')


def scan_sites(config):
    return coverage_sites(config)


def local_point(origin, bearing, x, y):
    u, v = unit(bearing), unit(bearing+90)
    return origin[0]+x*u[0]+y*v[0], origin[1]+x*u[1]+y*v[1]


def fallback_sites(origin, bearing):
    return [local_point(origin, bearing, x, y) for i, y in enumerate((-20, 0, 20))
            for x in (range(0, 1501, 30) if i % 2 == 0 else range(1500, -1, -30))]


class Track:
    def __init__(self, channel):
        self.channel = channel
        self.observations = []
        self.first = None
        self.near = None
        self.poly = None
        self.circle = None
        self.inconsistent = False
        self.cleared = False

    def observe(self, position, response, delta):
        result = response['measure_result']
        if result == 'no_signal':
            return
        self.observations.append({'position': position, 'result': result,
                                  'bearing_deg': response.get('svd_deg')})
```

13 问题三四_搜索定位清除框架.py 有限后备的基础策略

```
        if result == 'near':
            self.near = position
            return
        bearing = response['svd_deg']
        if self.first is None:
            self.first = position, bearing
            self.poly = [local_point(position, bearing, x, y)
                         for x, y in ((0, -30), (1500, -30), (1500, 30), (0, 30))]
            for plane in ((1, 0, 1800), (-1, 0, 1800), (0, 1, 1800), (0, -1, 1800)):
                self.poly = clip(self.poly, plane)
        if not self.inconsistent:
            for plane in bearing_planes(position, bearing, delta, max_range=1500):
                self.poly = clip(self.poly, plane)
            if not self.poly:
                self.inconsistent = True
                self.circle = None
            else:
                self.circle = enclosing_circle(self.poly)

    def snapshot(self):
        return {'channel': self.channel, 'observations': self.observations,
                'first_bearing': self.first, 'near_position': self.near,
                'region_vertices': self.poly, 'circle': self.circle,
                'inconsistent': self.inconsistent, 'cleared': self.cleared}


def solve(session, config):
    validate_config(config)
    tracks = {channel: Track(channel) for channel in range(1, 21)}
    counts = {channel: 0 for channel in tracks}
    events = []
    margin = config['clearance_margin_m']
    geometric = config['strategy'] == 'geometry'
    options = config.get('policy_options', {})
    early_attempted = set()
    pursued = set()
    shared_attempted = set()
    count_stop = False
    sites = scan_sites(config)
    for site_index, site in enumerate(sites):
        # 优先检测当前频道，尽可能省去一次换频。
        order = [session.channel] + [ch for ch in tracks if ch != session.channel]
        for ch in order:
            track = tracks[ch]
            is_settled = track.cleared or track.near is not None or (track.first is not None and not
geometric)
            is_settled |= bool(track.circle is not None and track.circle[1] <= 20-margin)
            if is_settled:
                continue
            result = session.measure(site, ch)
            counts[ch] += 1
            track.observe(site, result, config['angle_bound_deg'])
        scan_end_position = session.position
        if options.get('pursue_detour_m', 0) > 0 and site_index + 1 < len(sites):
            next_site = sites[site_index+1]
            eligible = [t for t in tracks.values() if t.first and t.circle and
                        not t.inconsistent and not settled(t, margin) and t.channel not in pursued]
            eligible.sort(key=lambda t: (distance(session.position, t.circle[0])+
                distance(t.circle[0], next_site), t.channel))
            for t in eligible:
                center = t.circle[0]
                detour = distance(session.position, center)+distance(center, next_site)-
distance(session.position, next_site)
                if detour > options['pursue_detour_m']+1e-8:
                    continue
                pursued.add(t.channel)
                refine(session, t, config, events)
                if settled(t, margin) and not t.inconsistent:
                    t.cleared = session.clear(safe_clear_point(t, session.position,
config),t.channel)['clear_result'] == 'success'
                    t.inconsistent |= not t.cleared
                events.append(dict(channel=t.channel, method='early_pursuit',
                    nominal_detour_m=detour, success=t.cleared))
```

13 问题三四_搜索定位清除框架.py 有限后备的基础策略

```
        if options.get('early_detour_m', 0) > 0 and site_index + 1 < len(sites):
            next_site = sites[site_index+1]
            for t in sorted(tracks.values(), key=lambda t: t.channel):
                if t.cleared or t.channel in early_attempted or not settled(t, margin) or t.inconsistent:
                    continue
                p = safe_clear_point(t, session.position, config)
                detour = distance(session.position, p)+distance(p, next_site)-distance(session.position,
next_site)
                if detour <= options['early_detour_m'] + 1e-8:
                    early_attempted.add(t.channel)
                    t.cleared = session.clear(p, t.channel)['clear_result'] == 'success'
                    t.inconsistent |= not t.cleared
                    events.append(dict(channel=t.channel, method='early_guaranteed',
                                       detour_m=detour, success=t.cleared))
                    if t.cleared:
                        opportunistic_clear(session, tracks.values(), config, events, shared_attempted)
        if options.get('count_stop_unsettled') is not None:
            found_so_far = sum(bool(t.observations) for t in tracks.values())
            unsettled_count = sum(bool(t.observations) and not settled(t, margin) for t in tracks.values())
            if found_so_far == 16 and unsettled_count <= options['count_stop_unsettled']:
                count_stop = True
                events.append(dict(method='source_count_discovery_stop', observed_channels=16,
                    unsettled_channels=unsettled_count, visited_scan_sites=site_index+1))
                break
        if options.get('dynamic_scan', False) and session.position != scan_end_position:
            # 只重排尚未访问的后缀，保留全部已认证站点。
            x, y = session.position
            remaining = [(p[0]-x, p[1]-y) for p in sites[site_index+1:]]
            lookup = {(p[0]-x, p[1]-y): p for p in sites[site_index+1:]}
            sites[site_index+1:] = [lookup[p] for p in improved_route(remaining)]
    found = sum(bool(t.observations) for t in tracks.values())
    pending = [t for t in tracks.values() if t.observations and not t.cleared]
    def target(track):
        if track.near is not None:
            return track.near
        if geometric and track.circle is not None:
            return track.circle[0]
        return track.first[0]
    clear_order = ordered_targets(pending, session.position, target) if options.get('clear_order') ==
'two_opt' else None
    if options.get('clear_order') == 'exact12':
        clear_order = exact_targets(pending, session.position, target)
    while pending:
        track = clear_order.pop(0) if clear_order is not None else min(pending, key=lambda t:
(distance(session.position, target(t)), t.channel))
        pending.remove(track)
        if track.cleared:
            continue
        refine(session, track, config, events)
        if track.near is not None:
            track.cleared = session.clear(safe_clear_point(track, session.position, config),
track.channel)['clear_result'] == 'success'
            events.append({'channel': track.channel, 'method': 'near', 'success': track.cleared})
        elif geometric and track.circle is not None:
            center, radius = track.circle
            track.cleared = session.clear(safe_clear_point(track, session.position, config),
track.channel)['clear_result'] == 'success'
            events.append({'channel': track.channel, 'method': 'circle_guaranteed' if radius <= 20-margin else
'circle_probe',
                           'radius_m': radius, 'success': track.cleared})
            if radius <= 20-margin and not track.cleared:
                track.inconsistent = True
        if not track.cleared:
            iterative_reprobe(session, track, config, events)
        threshold = options.get('failed_probe_threshold', 0)
        if threshold and not track.cleared and track.first and track.poly and not track.inconsistent:
            candidates = sum(point_region_distance(p, track.poly) <= 20+1e-5 for p in
fallback_sites(*track.first))
            if candidates >= threshold:
                point = session.position
                response = session.measure(point, track.channel)
                track.observe(point, response, config['angle_bound_deg'])
                events.append(dict(channel=track.channel, method='failed_probe_measure',
```

13 问题三四_搜索定位清除框架.py 有限后备的基础策略

```
candidates=candidates, result=response['measure_result']))
                if settled(track, margin) and not track.inconsistent:
                    track.cleared = session.clear(safe_clear_point(track, session.position, config),
track.channel)['clear_result'] == 'success'
                    track.inconsistent |= not track.cleared
                    events.append(dict(channel=track.channel, method='post_probe_guaranteed',
success=track.cleared))
        if not track.cleared and track.first is not None:
            all_sites = fallback_sites(*track.first)
            if distance(session.position, all_sites[-1]) < distance(session.position, all_sites[0]):
                all_sites.reverse()
            selected = all_sites
            if geometric and track.poly and not track.inconsistent:
                selected = [p for p in all_sites if point_region_distance(p, track.poly) <= 20 + 1e-5]
            tried = set()
            # 保留原蛇形访问次序，删点不会增加该次序的路径长度。
            for phase, points in (('filtered_fallback', selected), ('full_fallback', all_sites)):
                for point in points:
                    if point in tried:
                        continue
                    tried.add(point)
                    if session.clear(point, track.channel)['clear_result'] == 'success':
                        track.cleared = True
                        events.append({'channel': track.channel, 'method': phase, 'attempts': len(tried),
'success': True})
                        break
                if track.cleared:
                    break
                if phase == 'filtered_fallback' and len(selected) < len(all_sites):
                    track.inconsistent = True
                    events.append({'channel': track.channel, 'method': 'region_exhausted', 'success': False})
        if not track.cleared:
            events.append({'channel': track.channel, 'method': 'exhausted', 'success': False})
        else:
            opportunistic_clear(session, tracks.values(), config, events, shared_attempted)
        if options.get('opportunistic', False):
            # 全部未知频道已经完成扫描；此处的
            # 额外测量只精化已知待清目标，不作不存在证据。
            point = session.position
            for other in sorted(pending, key=lambda t: (t.channel != session.channel, t.channel)):
                if settled(other, margin) or other.inconsistent:
                    continue
                response = session.measure(point, other.channel)
                other.observe(point, response, config['angle_bound_deg'])
                events.append(dict(channel=other.channel, method='opportunistic_measure',
                                   result=response['measure_result'], position=point))
    cleared = [ch for ch, track in tracks.items() if track.cleared]
    absent = [ch for ch, track in tracks.items() if not track.observations and (count_stop or counts[ch] ==
len(sites))]
    complete = len(cleared) + len(absent) == 20 and config['min_sources'] <= len(cleared) <= 16
    inconsistent = [ch for ch, track in tracks.items() if track.inconsistent]
    return {'termination': 'coverage_complete' if complete else 'incomplete',
            'coverage_certificate': {'scan_grid': ('trimmed-45-spacing-690'
                if config.get('coverage_design', 'square64') == 'trimmed45' else ('square-64-spacing-600' if
config.get('coverage_design', 'square64') == 'square64' else config['coverage_design'])),
                'coverage_design': config.get('coverage_design', 'square64'),
                'problem_scope': config.get('problem_scope', 'general'), 'site_count': len(sites),
'scan_route': config.get('scan_route', 'serpentine'),
                'scan_sites_m': sites,
                'measurements_per_channel': counts, 'absent_channels': absent,
                **({'absence_basis': 'source_count_upper_bound', 'source_count_upper_bound': 16,
                    'visited_scan_sites': site_index+1} if count_stop else {}),
                'cleared_channels': cleared, 'all_channels_resolved': complete},
            'detected_count': found, 'cleared_count': len(cleared),
            'geometry_inconsistencies': inconsistent, 'events': events,
            'tracks': [track.snapshot() for track in tracks.values() if track.observations]}
```

14 问题四_三角网格覆盖.py 三角格连续覆盖

```
"""仅由已知源圆域生成有限三角格覆盖。
保留与 1800 米闭圆相交的全部闭三角形，边和顶点附近的邻接三角形均保留；顶点提供严格正向投影，距离不超过格边长。本模块不读取真值或观测。"""
import math


def segment_distance_sq(p, a, b):
    dx, dy = b[0]-a[0], b[1]-a[1]
    t = max(0., min(1., ((p[0]-a[0])*dx+(p[1]-a[1])*dy)/(dx*dx+dy*dy)))
    return (p[0]-a[0]-t*dx)**2+(p[1]-a[1]-t*dy)**2


def triangle_distance_sq(p, triangle):
    cross = []
    for a, b in zip(triangle, triangle[1:]+triangle[:1]):
        cross.append((b[0]-a[0])*(p[1]-a[1])-(b[1]-a[1])*(p[0]-a[0]))
    if min(cross) >= -1e-7 or max(cross) <= 1e-7:
        return 0.
    return min(segment_distance_sq(p, a, b)
               for a, b in zip(triangle, triangle[1:]+triangle[:1]))


def triangular_mesh(side=990., phase='vertex'):
    """返回选定顶点和三角形，向外留 10⁻⁵米余量。
    枚举索引范围覆盖源域及完整邻格；距离使用实际浮点坐标检查。"""
    if isinstance(side, bool) or not isinstance(side, (float, int)) or not 891 <= side <= 999:
        raise ValueError('Certified triangular side must be between 891 and 999 m')
    if phase not in ('vertex', 'centroid'):
        raise ValueError('Unknown triangular lattice phase')
    height = math.sqrt(3.)*side/2
    shift = (0., 0.) if phase == 'vertex' else (side/2., height/3.)

    def vertex(i, j):
        return side*(i+j/2.)-shift[0], height*j-shift[1]

    indices = set()
    triangles = []
    for i in range(-6, 7):
        for j in range(-6, 7):
            for ids in (((i,j), (i+1,j), (i,j+1)),
                        ((i+1,j+1), (i,j+1), (i+1,j))):
                triangle = [vertex(*k) for k in ids]
                if triangle_distance_sq((0., 0.), triangle) <= (1800.+1e-5)**2:
                    indices.update(ids)
                    triangles.append(triangle)
    return [vertex(*k) for k in sorted(indices)], triangles
```

15 问题三四_安全定位候选.py 具有接收保障的双侧探针

```
"""由已有正测向构造可核验的双侧探针。
本模块只接收观测产生的多边形，不访问模拟器或源位置、数量、类型、接收半径和朝向。"""
import math


def _local(point, origin, u, v):
    dx, dy = point[0]-origin[0], point[1]-origin[1]
    return dx*u[0]+dy*u[1], dx*v[0]+dy*v[1]


def _cap_vertices(local_polygon, forward):
    """求多边形与前向半平面的交点和顶点，兼顾线段退化。"""
    out = [p for p in local_polygon if p[0] >= forward]
    for p, q in zip(local_polygon, local_polygon[1:]+local_polygon[:1]):
        if (p[0] < forward < q[0]) or (q[0] < forward < p[0]):
            fraction = (forward-p[0])/(q[0]-p[0])
            out.append((forward, p[1]+fraction*(q[1]-p[1])))
    return out


def safe_pair_candidates(track, forward=600.0, lateral=100.0,
                         safety_margin=1e-4):
    """返回两个探针及连续接收证书，或返回空值。
    安全表示两点到整个当前多边形的距离均处于保证接收范围内。"""
    if (not track.first or not track.poly or track.inconsistent or track.cleared
            or track.near is not None):
        return None
    if not (math.isfinite(forward) and forward > 0
            and math.isfinite(lateral) and lateral >= 0
            and math.isfinite(safety_margin) and 0 < safety_margin < 1):
        raise ValueError('Require finite forward > 0, lateral >= 0, and 0 < margin < 1')
    origin, bearing = track.first
    angle = math.radians(bearing)
    u = (math.cos(angle), math.sin(angle))
    v = (-u[1], u[0])
    polygon = [tuple(p) for p in track.poly]
    local = [_local(p, origin, u, v) for p in polygon]
    cap = _cap_vertices(local, forward)
    if not cap:
        return None
    ray_low = forward*min(y/x for x, y in cap)
    ray_high = forward*max(y/x for x, y in cap)
    actual_lateral = max(lateral, abs(ray_low), abs(ray_high))+safety_margin
    points = [(origin[0]+forward*u[0]+sign*actual_lateral*v[0],
               origin[1]+forward*u[1]+sign*actual_lateral*v[1])
              for sign in (-1, 1)]
    maxima = [max(math.hypot(p[0]-q[0], p[1]-q[1]) for q in polygon)
              for p in points]
    if max(maxima) > 1000.0-safety_margin:
        return None
    return dict(kind='two_sided_negative_halfplane', origin=list(origin),
                bearing_deg=bearing, forward_m=forward,
                lateral_m=actual_lateral, points=points,
                source_polygon_before=polygon,
                cap_vertices_local=cap,
                ray_cross_section_y=[ray_low, ray_high],
                max_source_distance_m=maxima,
                safety_margin_m=safety_margin,
                plane=(u[0], u[1], u[0]*origin[0]+u[1]*origin[1]+forward))


def negative_pair_plane(pair, responses):
    """仅在两次实际阴性后返回已认证裁剪半平面。
    须按给定顺序对同一频道检测两个点，其间不能成功清除该源；先保存点对和真实反馈，再更新位置域，便于独立核验。"""
    if pair is None or len(responses) != 2:
        return None
    if any(r.get('measure_result') != 'no_signal' or r.get('accepted') is not True
           for r in responses):
        return None
    return tuple(pair['plane'])
```

16 问题三四_几何求解基线.py 搜索定位与清除调度

```
"""依据实际观测联合搜索与清除，复用冻结几何运算。"""
import math
from functools import lru_cache
from 问题三四_基础参数读取 import baseline_config
from 问题三四_搜索定位清除框架 import Track, fallback_sites
from 问题一_定位区域与包围圆 import distance, unit, point_region_distance, clip, enclosing_circle
from 问题三四_巡检点与路线生成 import coverage_sites, improved_route


DEFAULT = dict(scan_spacing=500., lateral=120., probe_rounds=3,
               joint=True, extra_scan=True, adaptive_cover=False, cover='base',
               prune_min_box=16., pair_probe=False, prune_distance=2000., prospective=False,
               q3_ring_n=6,q3_ring_radius=1125.,
q4_ring_n=14,q4_ring_radius=1855.,q4_inner_n=7,q4_inner_radius=995.,q4_outer_phase=0.)

@lru_cache(maxsize=128)
def validate_fixed_sites(points,problem):
    from 问题三四_连续覆盖构造 import certify
    if problem not in ('q3','q4'):raise ValueError('Expected q3 or q4')
    result=certify(list(points),problem=problem,minimum_box_m=.25,max_nodes=100000,fail_fast=True)
    if not result['passed']:raise ValueError('Initial continuous discovery coverage is not certified')
    return True

class ObservedSession:
    """策略可见字段白名单；真值由运行器另行保存。"""
    __slots__ = ('__measure', '__clear', '__state')
    def __init__(self, session):
        self.__measure = session.measure
        self.__clear = session.clear
        self.__state = lambda: (session.position, session.channel, session.virtual_time)
    @property
    def position(self): return self.__state()[0]
    @property
    def channel(self): return self.__state()[1]
    @property
    def virtual_time(self): return self.__state()[2]
    def measure(self, point, channel): return self.__measure(point, channel)
    def clear(self, point, channel): return self.__clear(point, channel)

def shifted_route(nodes, start):
    # 不同频道的坐标可能相同，任务身份仍分别保存。
    coordinates = [(p[0]-start[0], p[1]-start[1]) for _,p in nodes]
    route = improved_route(coordinates)
    buckets = {}
    for key,p in nodes: buckets.setdefault((p[0]-start[0],p[1]-start[1]), []).append((key,p))
    result = []
    for p in route:
        if buckets[p]: result.append(buckets[p].pop(0))
    return result

class Solver:
    def __init__(self, session, problem, settings=None):
        self.s = session
        self.problem = problem
        self.cfg = dict(DEFAULT, **(settings or {}))
        self.cfg.setdefault('ready_radius',float('inf'))
        self.cfg.setdefault('centroid',False)
        self.cfg.setdefault('inner_first',False)
        self.cfg.setdefault('first_forward',None)
        self.cfg.setdefault('movable_sectors',False)
        self.cfg.setdefault('sector_pull',1.)
        self.cfg.setdefault('skip_impossible',False)
        self.cfg.setdefault('adaptive_pair',False)
        self.cfg.setdefault('adaptive_clear',False)
        self.tracks = {ch:Track(ch) for ch in range(1,21)}
        self.events = []
        self.scanned = []
        self.measured = {ch:[] for ch in self.tracks}
        self.fixed = coverage_sites(baseline_config(problem))
        if problem=='q3' and self.cfg['q3_ring_n']!=6:
            from 问题三四_连续覆盖构造 import ring_points, open_route
            self.fixed=open_route(ring_points([(self.cfg['q3_ring_n'],self.cfg['q3_ring_radius'],0)]))
        if self.cfg['cover'] == 'ring25' and problem=='q4':
```

16 问题三四_几何求解基线.py 搜索定位与清除调度

```
            from 问题三四_连续覆盖构造 import ring_points, open_route
            self.fixed = open_route(ring_points([(8,995,0),(16,1840,0)]))
        if self.cfg['cover'] == 'ring22' and problem=='q4':
            from 问题三四_连续覆盖构造 import ring_points, open_route
            self.fixed =
open_route(ring_points([(self.cfg['q4_inner_n'],self.cfg['q4_inner_radius'],0),(self.cfg['q4_ring_n'],self.cfg
['q4_ring_radius'],self.cfg['q4_outer_phase'])]))
        self.sectors=None
        if problem=='q3' and self.cfg['movable_sectors']:
            from 问题三_全向扇形约束 import make_sectors
            self.sectors=make_sectors(n=self.cfg['q3_ring_n'],radius=self.cfg['q3_ring_radius'])
            self.fixed=[tuple(s['fallback']) for s in self.sectors]
        validate_fixed_sites(tuple([(0.,0.)]+[tuple(p) for p in self.fixed]),problem)
        self.todo = list(enumerate(self.fixed))
        self.count_stop = False
        self.coverage_complete = False

    def observe(self, ch, p):
        response = self.s.measure(p,ch)
        track = self.tracks[ch]
        track.observe(p,response,1.005)
        self.measured[ch].append(tuple(p))
        if track.poly and not track.inconsistent:
            # 外接 32 边形包含完整半径 1800 米源圆域。
            for k in range(32):
                a,b=unit(k*360/32)
                track.poly=clip(track.poly,(a,b,1800.))
            if not track.poly: track.inconsistent=True
            else: track.circle=enclosing_circle(track.poly)
        self.events.append(dict(method='observe',channel=ch,position=p,
            response=response['measure_result'],poly=track.poly))
        if response['measure_result']=='near': self.attempt(track,p,'near')
        return response

    def attempt(self, track, p, method):
        guaranteed=bool((track.near is not None and distance(p,track.near)<=14.99) or
            (track.poly and not track.inconsistent and max(distance(p,q) for q in track.poly)<=19.99))
        ok=self.s.clear(p,track.channel)['clear_result']=='success'
        track.cleared=ok
        if guaranteed and not ok:track.inconsistent=True

self.events.append(dict(method=method,channel=track.channel,position=p,success=ok,guaranteed=guaranteed))
        if method=='near' and not ok:
            raise RuntimeError('Near-confirmed source failed optical clearance; inconsistent feedback')
        return ok

    def scan(self, p, force=False):
        if not force and self.scanned and min(distance(p,q) for q in self.scanned)<self.cfg['scan_spacing']:
            return
        self.scanned.append(tuple(p))
        for ch,t in sorted(self.tracks.items(), key=lambda kv:(kv[0]!=self.s.channel,kv[0])):
            if t.cleared: continue
            if self.cfg['skip_impossible'] and t.poly and not t.inconsistent and
point_region_distance(p,t.poly)>1500.00001:continue
            # 未知频道逐次完成发现扫描；已知目标可从
            # 几何已改变且仍不确定时的新示向获益。
            if t.circle and t.circle[1]<19.99: continue
            if t.observations and min(distance(p,o['position']) for o in t.observations)<60: continue
            self.observe(ch,p)
        if sum(bool(t.observations) for t in self.tracks.values())==16:
            self.count_stop=True
            self.todo=[]
        elif self.sectors:
            from 问题三_全向扇形约束 import safe_sector, sector_certificate
            for i,q in list(self.todo):
                if safe_sector(p,self.sectors[i]):
                    self.todo=[(j,q) for j,q in self.todo if j!=i]

self.events.append(dict(method='sector_covered',certificate=sector_certificate(p,self.sectors[i])))
        elif self.cfg['adaptive_cover']:
            self.prune()

    def prune(self):
```

16 问题三四_几何求解基线.py 搜索定位与清除调度

```
        from 问题三四_连续覆盖构造 import certify
        # 只有证书同时包含实际已检测位置和
        # 全部仍承诺访问的站点时才允许删去未来访问；重复删站保持
        # 此不变量，退出时只剩实际检测位置。
        ordered=sorted(self.todo,key=lambda row:distance(self.s.position,row[1]),reverse=True)
        for i,p in ordered:
            if min(distance(p,q) for q in self.scanned)>self.cfg['prune_distance']:continue
            others=[q for j,q in self.todo if j!=i]
            evidence=certify(self.scanned+others,problem=self.problem,
                minimum_box_m=1. if self.problem=='q3' else self.cfg['prune_min_box'],fail_fast=True)
            if evidence['passed']:
                self.todo=[(j,q) for j,q in self.todo if j!=i]
                self.events.append(dict(method='certified_site_deleted',site_index=i,position=p))

    def target_point(self,t):
        if t.near is not None:return t.near
        if not t.circle:return t.first[0]
        c,r=t.circle
        if self.cfg['centroid'] and t.poly and len(t.poly)>2:
            area=cx=cy=0.
            for a,b in zip(t.poly,t.poly[1:]+t.poly[:1]):
                cross=a[0]*b[1]-b[0]*a[1]
                area+=cross;cx+=(a[0]+b[0])*cross;cy+=(a[1]+b[1])*cross
            if abs(area)>1e-8:c=(cx/(3*area),cy/(3*area))
        if r<200 or len(t.observations)>=2:return c
        origin,angle=t.first
        if self.cfg['first_forward'] is not None:
            u=unit(angle)
            c=(origin[0]+self.cfg['first_forward']*u[0],origin[1]+self.cfg['first_forward']*u[1])
        ux,uy=unit(angle+90)
        amount=self.cfg['lateral']
        candidates=[(c[0]+sign*amount*ux,c[1]+sign*amount*uy) for sign in [-1,1]]
        return min(candidates,key=lambda p:distance(self.s.position,p))

    def resolve(self,t):
        if t.cleared:return
        if t.near is not None:
            self.attempt(t,t.near,'near');return
        p=self.target_point(t)
        if t.circle and (len(t.observations)==1 and t.circle[1]>=200):
            pair=None
            if self.cfg['pair_probe']:
                from 问题三四_安全定位候选 import safe_pair_candidates, negative_pair_plane
                forward=600.
                lateral=self.cfg['lateral']
                if self.cfg['adaptive_pair']:
                    u=unit(t.first[1]);o=t.first[0]
                    xs=[(q[0]-o[0])*u[0]+(q[1]-o[1])*u[1] for q in t.poly]
                    forward=(min(xs)+max(xs))/2
                    lateral=min(120.,max(30.,(max(xs)-min(xs))*.15))
                pair=safe_pair_candidates(t,forward=forward,lateral=lateral)
            if pair:
                responses=[]
                for q in pair['points']:
                    responses.append(self.observe(t.channel,q))
                    if responses[-1]['measure_result']!='no_signal':break
                plane=negative_pair_plane(pair,responses)
                if plane:
                    t.poly=clip(t.poly,plane)
                    if not t.poly:t.inconsistent=True
                    else:t.circle=enclosing_circle(t.poly)
                    self.events.append(dict(method='two_sided_negative_halfplane',channel=t.channel,
                        pair=pair,responses=responses,poly=t.poly))
            else:self.observe(t.channel,p)
            if self.cfg['extra_scan']:self.scan(self.s.position)
        for j in range(self.cfg['probe_rounds']):
            if t.cleared:return
            p=t.circle[0] if t.circle and not t.inconsistent else self.target_point(t)
            if self.attempt(t,p,'center_probe'):
                break
            self.observe(t.channel,p)
            if t.cleared:return
            if self.cfg['adaptive_pair'] and t.circle and t.circle[1]>25 and not t.inconsistent:
```

16 问题三四_几何求解基线.py 搜索定位与清除调度

```
                from 问题三四_安全定位候选 import safe_pair_candidates, negative_pair_plane
                u=unit(t.first[1]);o=t.first[0]
                xs=[(q[0]-o[0])*u[0]+(q[1]-o[1])*u[1] for q in t.poly]
                pair=safe_pair_candidates(t,forward=max(.001,(min(xs)+max(xs))/2),
                    lateral=min(120.,max(30.,(max(xs)-min(xs))*.15)))
                if pair:
                    responses=[]
                    for q in pair['points']:
                        responses.append(self.observe(t.channel,q))
                        if responses[-1]['measure_result']!='no_signal':break
                    plane=negative_pair_plane(pair,responses)
                    if plane:
                        t.poly=clip(t.poly,plane)
                        if not t.poly:t.inconsistent=True
                        else:t.circle=enclosing_circle(t.poly)

self.events.append(dict(method='two_sided_negative_halfplane',channel=t.channel,pair=pair,responses=responses,
poly=t.poly))
                    continue
            if t.circle and distance(t.circle[0],p)<5 and t.circle[1]>20:
                # 测点可能不可接收，或与已有示向形成较差交会角。
                origin,angle=t.first
                ux,uy=unit(angle+90)
                offset=min(self.cfg['lateral'],max(30,t.circle[1]*.5))
                side=1 if j%2==0 else -1
                self.observe(t.channel,(p[0]+side*offset*ux,p[1]+side*offset*uy))
        if not t.cleared:
            if self.cfg['adaptive_clear']:
                from 问题三四_光学清除覆盖 import clear_cover
                cover=clear_cover(t,start=self.s.position)
                if cover:

self.events.append(dict(method='adaptive_clear_cover',channel=t.channel,certificate=cover['certificate'],poly=
t.poly))
                    for p in cover['points']:
                        if self.attempt(t,p,'adaptive_clear_cell'):break
                    if not t.cleared:
                        t.inconsistent=True

self.events.append(dict(method='adaptive_clear_cover_exhausted',channel=t.channel,success=False))
        if not t.cleared:
            points=fallback_sites(*t.first)
            if t.poly and not t.inconsistent:
                filtered=[p for p in points if point_region_distance(p,t.poly)<=20.00001]
            else:filtered=points
            done=set()
            for group in [filtered,points]:
                if distance(self.s.position,group[-1])<distance(self.s.position,group[0]) if group else False:
                    group=list(reversed(group))
                for p in group:
                    if p in done:continue
                    done.add(p)
                    if self.attempt(t,p,'finite_fallback'):break
                if t.cleared:break
        if not t.cleared:raise RuntimeError('Certified fallback exhausted')
        # 同位置光学试清无需移动，也不要求定向无线电接收。
        for other in self.tracks.values():
            if other.cleared or not other.poly:continue
            if point_region_distance(self.s.position,other.poly)<=19.99:
                self.attempt(other,self.s.position,'shared_probe')
        if self.cfg['extra_scan']:self.scan(self.s.position)

    def run(self):
        self.scan((0.,0.),True)
        self.todo=[(i,p) for i,p in self.todo if distance(p,(0.,0.))>1e-7]
        while self.todo or any(t.observations and not t.cleared for t in self.tracks.values()):
            pending=[t for t in self.tracks.values() if t.observations and not t.cleared]
            all_pending=list(pending)
            if self.todo:
                pending=[t for t in pending if t.near is not None or (t.circle and
t.circle[1]<=self.cfg['ready_radius'])]
            active_todo=self.todo
            if self.sectors and all_pending:
```

16 问题三四_几何求解基线.py 搜索定位与清除调度

```
                from 问题三_全向扇形约束 import project_sector
                proposed=[]
                predicted=[self.target_point(t) for t in all_pending]
                for i,p in active_todo:
                    desired=min(predicted,key=lambda q:distance(q,p))
                    pull=self.cfg['sector_pull']
                    desired=(p[0]+pull*(desired[0]-p[0]),p[1]+pull*(desired[1]-p[1]))
                    proposed.append((i,project_sector(desired,self.sectors[i])))
                active_todo=proposed
            if self.problem=='q4' and self.cfg['inner_first']:
                inner=[(i,p) for i,p in active_todo if math.hypot(*p)<1100]
                if inner:active_todo=inner
            prospective_scan=False
            if self.cfg['prospective'] and pending and self.todo:
                from 问题三四_连续覆盖构造 import certify
                predicted=[self.target_point(t) for t in pending]
                active_todo=list(self.todo)
                for i,p in sorted(self.todo,key=lambda row:distance(self.s.position,row[1]),reverse=True):
                    if min(distance(p,q) for q in predicted)>self.cfg['prune_distance']:continue
                    if certify(self.scanned+predicted+[q for j,q in active_todo if j!=i],
                        problem=self.problem,minimum_box_m=1 if self.problem=='q3' else
self.cfg['prune_min_box'],fail_fast=True)['passed']:
                        active_todo=[(j,q) for j,q in active_todo if j!=i]
                        prospective_scan=True
                # 计划测站只帮助选择路线，不能据此确认
                # 频道不存在；原未访问站仍保存在待办集合中。
            nodes=[(('scan',i),p) for i,p in active_todo]
            if self.cfg['joint'] or not nodes:
                nodes += [(('target',t.channel),self.target_point(t)) for t in pending]
            key,p=shifted_route(nodes,self.s.position)[0]
            if key[0]=='scan':
                self.todo=[(i,q) for i,q in self.todo if i!=key[1]]
                self.scan(p,True)
            else:
                if prospective_scan:
                    self.scan(p,True)
                self.resolve(self.tracks[key[1]])
        self.coverage_complete=not self.todo
        cleared=sorted(ch for ch,t in self.tracks.items() if t.cleared)
        absent=sorted(ch for ch,t in self.tracks.items() if not t.observations)
        complete=len(cleared)+len(absent)==20 and 10<=len(cleared)<=16 and not any(t.inconsistent for t in
self.tracks.values())
        return dict(termination='coverage_complete' if complete else
'incomplete',cleared_channels=cleared,absent_channels=absent,
            coverage_evidence=dict(kind='source_count_upper_bound' if self.count_stop else
('continuous_actual_sites' if self.cfg['adaptive_cover'] or self.sectors else 'fixed_sites'),
                problem=self.problem,design=self.cfg['cover'],sites=self.fixed,
                actual_scan_sites=self.scanned,
                measurements_per_channel=self.measured),
            geometry_inconsistencies=[ch for ch,t in self.tracks.items() if t.inconsistent],
            events=self.events,tracks=[t.snapshot() for t in self.tracks.values() if t.observations])

def solve(session, problem, settings=None):
    return Solver(session,problem,settings).run()
```

17 问题三_全向扇形约束.py 环扇区与可移动测站

```
"""仅适用于 Q3 全向源的可移动测站环扇区覆盖。
每个 1000 至 1800 米环扇区以四个端点距离检查认证连续覆盖，角半宽为π/n；几何依据为径向凸性与角度端点极值。"""
import math


DEFAULT_RADII = {6: 1125., 7: 998., 8: 940., 9: 905.,
                 10: 882., 11: 867., 12: 855.}


def minimum_ring_radius(n, margin=0.0):
    if isinstance(n, bool) or not isinstance(n, int) or n < 6:
        raise ValueError('Require an integer ring-point count n >= 6')
    if not math.isfinite(margin) or not 0 <= margin < 1:
        raise ValueError('Require 0 <= finite margin < 1 metre')
    alpha = math.pi/n
    return 1800*math.cos(alpha)-math.sqrt((1000-margin)**2-(1800*math.sin(alpha))**2)


def _point(point):
    if len(point) != 2 or not all(math.isfinite(x) for x in point):
        raise ValueError('Require two finite point coordinates')
    return float(point[0]), float(point[1])


def make_sectors(n=8, radius=None, rotation_deg=0.0, margin=0.01):
    """返回 n 个闭环扇区及已认证后备点。
    每个未知频道须在原点实测以覆盖 1000 米内区域，在每个扇区的安全点也须实际检测，机器狗仅路过不能代替。"""
    required = minimum_ring_radius(n, margin)
    if not math.isfinite(rotation_deg):
        raise ValueError('Require a finite rotation')
    if radius is None:
        radius = DEFAULT_RADII.get(n, math.ceil(required)+1.)
    if not math.isfinite(radius) or radius <= 0:
        raise ValueError('Require a finite positive fallback radius')
    output = []
    for i in range(n):
        beta = rotation_deg+i*360./n
        half = 180./n
        corners = [(r*math.cos(math.radians(beta+side*half)),
                    r*math.sin(math.radians(beta+side*half)))
                   for r in (1000., 1800.) for side in (-1, 1)]
        fallback = (radius*math.cos(math.radians(beta)),
                    radius*math.sin(math.radians(beta)))
        sector = dict(id=i, beta_deg=beta, half_width_deg=half,
                      inner_radius_m=1000., outer_radius_m=1800.,
                      corners=corners, fallback=fallback,
                      margin_m=margin, ring_radius_m=radius,
                      ring_count=n)
        if not safe_sector(fallback, sector):
            raise ValueError('Fallback radius fails the four-endpoint coverage certificate')
        output.append(sector)
    return output


def safe_sector(point, sector):
    """四个端点距离均满足显式余量时才返回真。"""
    point = _point(point)
    limit = 1000.-sector['margin_m']
    return all(math.hypot(point[0]-v[0], point[1]-v[1]) <= limit
               for v in sector['corners'])


def sector_certificate(point, sector):
    point = _point(point)
    distances = [math.hypot(point[0]-v[0], point[1]-v[1])
                 for v in sector['corners']]
    return dict(kind='q3_annular_sector_four_endpoints', sector_id=sector['id'],
                point=point, corners=sector['corners'],
                beta_deg=sector['beta_deg'], half_width_deg=sector['half_width_deg'],
                endpoint_distances_m=distances, max_distance_m=max(distances),
                certified_radius_m=1000.-sector['margin_m'],
                valid=max(distances) <= 1000.-sector['margin_m'])
```

17 问题三_全向扇形约束.py 环扇区与可移动测站

```
def project_sector(desired, sector, max_iterations=256):
    """近似寻找四圆盘交集中的最近点，并保留回退。
    Dykstra 凸投影只负责产生候选；返回前的四项距离检查负责接收保证，迭代次数或收敛本身不作证据。投影失败时返回已验证的固定环点。"""
    desired = _point(desired)
    fallback = _point(sector['fallback'])
    if not safe_sector(fallback, sector):
        raise ValueError('Cannot use an uncertified fallback')
    if safe_sector(desired, sector):
        return desired
    if isinstance(max_iterations, bool) or not isinstance(max_iterations, int) or max_iterations < 0:
        raise ValueError('Require a nonnegative integer iteration count')
    # 额外工作余量用于吸收浮点舍入，并留给
    # 最终独立端点检查使用。
    radius = 1000.-sector['margin_m']-1e-6
    point = desired
    corrections = [(0., 0.) for _ in sector['corners']]
    for _ in range(max_iterations):
        before = point
        for i, center in enumerate(sector['corners']):
            shifted = (point[0]+corrections[i][0], point[1]+corrections[i][1])
            dx, dy = shifted[0]-center[0], shifted[1]-center[1]
            length = math.hypot(dx, dy)
            if length > radius:
                projected = (center[0]+radius*dx/length, center[1]+radius*dy/length)
            else:
                projected = shifted
            corrections[i] = (shifted[0]-projected[0], shifted[1]-projected[1])
            point = projected
        if math.hypot(point[0]-before[0], point[1]-before[1]) < 1e-9:
            break
    return point if safe_sector(point, sector) else fallback
```

18 问题三_阴性观测收缩.py 全向正负观测几何

```
"""只根据合法动作历史使用 Q3 全向阴性接收证据。"""
import math
from pathlib import Path
import sys

ROOT = Path(__file__).resolve().parent
sys.path.insert(0, str(ROOT))
from 问题三四_几何求解基线 import Solver
from 问题三四_测向动作选择 import SenseSolver
from 问题一_定位区域与包围圆 import convex_hull, clip, enclosing_circle, distance


def outward_hull(polygon,padding=1e-6):
    return convex_hull([(p[0]+dx,p[1]+dy) for p in polygon
                        for dx in (-padding,padding) for dy in (-padding,padding)])


def outside_disk_hull(polygon, center, radius=999.9):
    """构造多边形扣除开圆盘后的保守凸包。
    剩余紧集上线性函数的极值可在原多边形边界取得：内部圆弧点仍可沿远离圆心方向微移。因此只需保留外侧顶点和圆与边的交点。排除半径另留 0.1 米余量。"""
    radius_sq = radius*radius
    points = [tuple(p) for p in polygon
              if (p[0]-center[0])**2+(p[1]-center[1])**2 >= radius_sq]
    if len(points) == len(polygon):
        return list(polygon)
    for p, q in zip(polygon, polygon[1:]+polygon[:1]):
        dx, dy = q[0]-p[0], q[1]-p[1]
        sx, sy = p[0]-center[0], p[1]-center[1]
        aa = dx*dx+dy*dy
        if aa == 0:
            continue
        bb, cc = 2*(sx*dx+sy*dy), sx*sx+sy*sy-radius_sq
        discriminant = bb*bb-4*aa*cc
        # 接近相切的数值状态不能作为舍弃支撑点的理由。
        tolerance = 1e-12*max(1., bb*bb, abs(4*aa*cc))
        if discriminant < -tolerance:
            continue
        root = math.sqrt(max(0.,discriminant))
        for fraction in ((-bb-root)/(2*aa),(-bb+root)/(2*aa)):
            if -1e-10 <= fraction <= 1+1e-10:
                fraction = min(1.,max(0.,fraction))
                points.append((p[0]+fraction*dx,p[1]+fraction*dy))
    return convex_hull(points)


class NegativeSolver(SenseSolver):
    def __init__(self, session, problem, settings=None):
        if problem != 'q3':
            raise ValueError('Omni negative-distance evidence is Q3-only')
        super().__init__(session,problem,settings)
        self.negative_points={ch:[] for ch in self.tracks}
        self.cfg.setdefault('negative_disk', True)
        self.cfg.setdefault('radius_consistency', True)
        self.cfg.setdefault('service_clear', False)
        self.cfg.setdefault('replace_scan', False)
        if self.cfg['replace_scan']:
            self.cfg['adaptive_cover']=True

    def observe(self,ch,p):
        response=super().observe(ch,p)
        if response['measure_result']=='no_signal':
            self.negative_points[ch].append(dict(position=tuple(p),measure_index=len(self.measured[ch])))
        track=self.tracks[ch]
        if track.cleared or not track.poly or track.inconsistent:
            return response
        for negative_record in self.negative_points[ch]:
            negative=negative_record['position']
            before=list(track.poly)
            planes=[]
            if self.cfg['radius_consistency']:
                for positive in track.observations:
                    positive=positive['position']
```

18 问题三_阴性观测收缩.py 全向正负观测几何

```
                    # 平移坐标后展开平方距离比较，
                    # 避免直接相减两个较大的平方范数。
                    dx,dy=negative[0]-positive[0],negative[1]-positive[1]
                    plane=(2*dx,2*dy,
                           2*dx*positive[0]+2*dy*positive[1]+dx*dx+dy*dy)
                    track.poly=clip(track.poly,plane)
                    planes.append(plane)
                if track.poly:
                    track.poly=outward_hull(track.poly)
            mid=list(track.poly)
            if self.cfg['negative_disk'] and track.poly:
                track.poly=outside_disk_hull(track.poly,negative)
                if track.poly:
                    track.poly=outward_hull(track.poly)
            if not track.poly:
                track.inconsistent=True
                track.circle=None
                self.events.append(dict(method='q3_negative_region_empty',channel=ch,
                    negative_position=negative,prior_poly=before,poly=[]))
                return response
            if track.poly != before:
                track.circle=enclosing_circle(track.poly)
                self.events.append(dict(method='q3_negative_reception',channel=ch,
                    negative_position=negative,excluded_radius_m=999.9 if self.cfg['negative_disk'] else None,
                    negative_measure_index=negative_record['measure_index'],
                    positives_used=[o['position'] for o in track.observations],
                    outward_padding_m=1e-6 if self.cfg['negative_disk'] else 0.,
                    mid_outward_padding_m=1e-6 if self.cfg['radius_consistency'] else 0.,
                    radius_comparison_planes=planes,prior_poly=before,
                    poly_after_radius_comparison=mid,poly=track.poly))
        return response

    def attempt(self,track,p,method):
        if (self.cfg.get('service_clear',False) and method=='center_probe'
                and track.poly and not track.inconsistent):
            radius=max(distance(p,q) for q in track.poly)
            budget=max(0.,19.99-radius)
            length=distance(self.s.position,p)
            if length>0 and budget>0:
                t=min(1.,budget/length)
                p=(p[0]+t*(self.s.position[0]-p[0]),p[1]+t*(self.s.position[1]-p[1]))
        return super().attempt(track,p,method)

    def resolve(self,track):
        if self.cfg.get('nearest_pair',False):
            import 问题三四_安全定位候选 as localization
            original=localization.safe_pair_candidates
            def nearest(*args,**kwargs):
                pair=original(*args,**kwargs)
                if pair and
distance(self.s.position,pair['points'][1])<distance(self.s.position,pair['points'][0])-1e-9:
                    pair=dict(pair,points=list(reversed(pair['points'])),
                              max_source_distance_m=list(reversed(pair['max_source_distance_m'])))
                return pair
            localization.safe_pair_candidates=nearest
            try:super().resolve(track)
            finally:localization.safe_pair_candidates=original
        else:
            super().resolve(track)
        if (not track.cleared or not self.cfg['replace_scan'] or not self.todo or
                not any(not t.observations for t in self.tracks.values())):
            return
        from 问题三四_连续覆盖构造 import certify
        point=tuple(self.s.position)
        if any(distance(point,q)<1e-7 for q in self.scanned):
            return
        profitable=False
        for identity,site in sorted(self.todo,key=lambda row:distance(point,row[1])):
            proposed=self.scanned+[point]+[q for i,q in self.todo if i!=identity]
            result=certify(proposed,problem='q3',minimum_box_m=1.,max_nodes=12000,fail_fast=True)
            if result['passed']:
                profitable=True
                break
```

18 问题三_阴性观测收缩.py 全向正负观测几何

```
        if not profitable:
            return
        # 在此实际检测每个未知频道；已知频道
        # 无需这次仅用于发现的动作。只有实际检测才能登记覆盖，
        # 仅到达清除终点不算完成检测覆盖。
        self.scanned.append(point)
        unknown=[ch for ch,t in self.tracks.items() if not t.observations]
        for ch in sorted(unknown,key=lambda ch:(ch!=self.s.channel,ch)):
            self.observe(ch,point)
        self.events.append(dict(method='q3_conditional_actual_scan',point=point,
                                channels_measured=unknown))
        if sum(bool(t.observations) for t in self.tracks.values())==16:
            self.count_stop=True
            self.todo=[]
        else:
            self.prune()


def solve(session,problem,settings=None):
    return NegativeSolver(session,problem,settings).run()
```

19 问题三四_测向动作选择.py 到达后按需补测

```
"""在已经到达的停车位置，对有价值的已知频道补测。"""
import math
from 问题三四_几何求解基线 import Solver
from 问题一_定位区域与包围圆 import distance, point_region_distance, unit


class SenseSolver(Solver):
    def useful(self,t):
        p=self.s.position
        if t.cleared or t.inconsistent or not t.poly or not t.circle:return False
        if t.circle[1]<self.cfg.get('sense_radius',80.):return False
        if min(distance(p,q) for q in self.measured[t.channel])<60:return False
        if point_region_distance(p,t.poly)>1500.00001:return False
        if distance(p,t.circle[0])<20:return True
        first,bearing=t.first;u=unit(bearing)
        v=((t.circle[0][0]-p[0])/distance(p,t.circle[0]),(t.circle[0][1]-p[1])/distance(p,t.circle[0]))
        return abs(u[0]*v[1]-u[1]*v[0])>=self.cfg.get('sense_sine',.1)

    def resolve(self,t):
        if self.cfg.get('pre_sense',False) and self.useful(t):
            self.observe(t.channel,self.s.position)
            # 原地补测一次后，交回主循环重新排序。
            # 有用性检查拒绝重复测点，因此不会在原地卡住循环。
            return
        super().resolve(t)
        if self.cfg.get('post_sense',False):
            for ch,other in sorted(self.tracks.items(),key=lambda kv:(kv[0]!=self.s.channel,kv[0])):
                if self.useful(other):self.observe(ch,self.s.position)


def solve(session,problem,settings=None):
    return SenseSolver(session,problem,settings).run()
```

20 问题三四_局部候选测点评估.py 候选探针对选

```
"""M020 的观测定位候选，在 M019 几何与退出证书下比较动作。"""
import math
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
import 问题三四_模型加载 as bootstrap
from 问题三四_几何求解基线 import Solver
from 问题三_阴性观测收缩 import NegativeSolver
from 问题三四_光学清除覆盖 import clear_cover
from 问题一_定位区域与包围圆 import distance, unit, point_region_distance, clip, bearing_planes,
enclosing_circle


def sample_polygon(poly):
    """确定性面积求积只用于候选排名，不用于排除位置。"""
    c = (sum(p[0] for p in poly)/len(poly),sum(p[1] for p in poly)/len(poly))
    out = []
    for a,b in zip(poly,poly[1:]+poly[:1]):
        area = abs((a[0]-c[0])*(b[1]-c[1])-(a[1]-c[1])*(b[0]-c[0]))/2
        if area < 1e-10:continue
        for wc,wa,wb in [(2/3,1/6,1/6),(1/6,2/3,1/6),(1/6,1/6,2/3)]:
            out.append(((wc*c[0]+wa*a[0]+wb*b[0],wc*c[1]+wa*a[1]+wb*b[1]),area/3))
    if not out:return [(p,1/len(poly)) for p in poly]
    total=sum(w for p,w in out)
    return [(p,w/total) for p,w in out]


def receive_probability(origin,probe,source):
    """在首次接收条件下采用均匀朝向先验，仅用于排名。"""
    x=(origin[0]-source[0],origin[1]-source[1]);y=(probe[0]-source[0],probe[1]-source[1])
    norm=math.hypot(*x)*math.hypot(*y)
    if norm<1e-10:return 1.
    return 1-math.acos(max(-1.,min(1.,(x[0]*y[0]+x[1]*y[1])/norm)))/math.pi


class LocalMixin:
    def __init__(self,session,problem,settings=None):
        super().__init__(session,problem,settings)
        self.cfg.setdefault('local_mode','optical')
        self.cfg.setdefault('optical_max_points',6)
        self.cfg.setdefault('optical_min_radius',20.)
        self.cfg.setdefault('pair_cost_weight',1.)

    def choose_pair(self,original,track,**kwargs):
        baseline=original(track,**kwargs)
        if not track.poly or track.inconsistent or track.cleared or not track.first:return baseline
        origin,bearing=track.first;u=unit(bearing)
        xs=[(p[0]-origin[0])*u[0]+(p[1]-origin[1])*u[1] for p in track.poly]
        lo,hi=min(xs),max(xs);span=hi-lo
        candidates=[]
        if baseline:candidates.append(baseline)
        for fraction in (.25,.4,.55,.7,.85):
            for lateral in (30.,60.,100.,180.):
                pair=original(track,forward=max(.001,lo+fraction*span),lateral=lateral)
                if pair:candidates.append(pair)
        if not candidates:return baseline
        samples=sample_polygon(track.poly)
        best=None
        for pair in candidates:
            points=sorted(pair['points'],key=lambda p:distance(self.s.position,p))
            if points!=pair['points']:

pair=dict(pair,points=points,max_source_distance_m=list(reversed(pair['max_source_distance_m'])))
            q=points[0]
            estimate=distance(self.s.position,q)/5+3
            for source,weight in samples:
                poly=list(track.poly)
                angle=math.degrees(math.atan2(source[1]-q[1],source[0]-q[0]))
                for plane in bearing_planes(q,angle,1.005,1500):poly=clip(poly,plane)
                circle=enclosing_circle(poly) if poly else track.circle
```

20 问题三四_局部候选测点评估.py 候选探针对选

```
                r=circle[1]
                positive=(max(0.,distance(q,circle[0])-max(0.,19.99-r))/5+5+
                          self.cfg['pair_cost_weight']*max(0.,r-19.99)*.25)
                probability=1. if self.problem=='q3' else receive_probability(origin,q,source)
                negative=distance(q,points[1])/5+3+distance(points[1],source)/5+5
                estimate+=weight*(probability*positive+(1-probability)*negative)
            key=(estimate,tuple(points))
            if best is None or key<best[0]:best=(key,pair)
        self.events.append(dict(method='m020_pair_ranking',channel=track.channel,
            evaluated_candidates=len(candidates),selected_estimated_cost_s=best[0][0],
            ranking_only=True,source_polygon=track.poly,selected_points=best[1]['points']))
        return best[1]

    def resolve(self,track):
        if (self.cfg['local_mode'] in ('optical','combined') and not track.cleared and
                track.near is None and track.poly and not track.inconsistent and
                track.circle and track.circle[1]>=self.cfg['optical_min_radius']):
            cover=clear_cover(track,start=self.s.position)
            if cover and len(cover['points'])<=self.cfg['optical_max_points']:
                self.events.append(dict(method='adaptive_clear_cover',channel=track.channel,
                    certificate=cover['certificate'],poly=track.poly,m020_early_cover=True))
                for p in cover['points']:
                    if self.attempt(track,p,'adaptive_clear_cell'):break
                if not track.cleared:
                    track.inconsistent=True

self.events.append(dict(method='adaptive_clear_cover_exhausted',channel=track.channel,success=False))
                else:
                    for other in self.tracks.values():
                        if other.cleared or not other.poly:continue
                        if point_region_distance(self.s.position,other.poly)<=19.99:
                            self.attempt(other,self.s.position,'shared_probe')
                    if self.cfg['extra_scan']:self.scan(self.s.position)
        if self.cfg['local_mode'] in ('pair','combined'):
            import 问题三四_安全定位候选 as localization
            original=localization.safe_pair_candidates
            def choose(track,**kwargs):return self.choose_pair(original,track,**kwargs)
            localization.safe_pair_candidates=choose
            try:return super().resolve(track)
            finally:localization.safe_pair_candidates=original
        return super().resolve(track)


class LocalQ3(LocalMixin,NegativeSolver):pass
class LocalQ4(LocalMixin,Solver):pass


def solve(session,problem,settings=None):
    cls=LocalQ3 if problem=='q3' else LocalQ4
    return cls(session,problem,bootstrap.baseline_settings(problem) if settings is None else settings).run()
```

21 问题三四_候选区域概率计算.py 按合法历史计算参考权重

```
"""按观测历史构造排名先验；后验权重不排除几何可行位置。"""
import math
from 问题三四_局部候选测点评估 import LocalQ3, LocalQ4, distance, unit, clip, bearing_planes, enclosing_circle,
sample_polygon, receive_probability
import 问题三四_模型加载 as bootstrap

TAU=2*math.pi


def angle_intervals(source,positive,negative):
    """计算同时满足各接收半平面符号条件的精确方向区间。"""
    constraints=[];cuts={0.,TAU}
    for points,sign in ((positive,1),(negative,-1)):
        for p in points:
            angle=math.atan2(p[1]-source[1],p[0]-source[0])
            constraints.append((angle,sign))
            cuts.add((angle-math.pi/2)%TAU);cuts.add((angle+math.pi/2)%TAU)
    cuts=sorted(cuts);result=[]
    for lo,hi in zip(cuts,cuts[1:]):
        mid=(lo+hi)/2
        if all(sign*math.cos(mid-angle)>=-1e-12 for angle,sign in constraints):result.append((lo,hi))
    return result


def halfplane_mass(intervals,source,probe):
    angle=math.atan2(probe[1]-source[1],probe[0]-source[0])
    cuts=sorted({0.,TAU,(angle-math.pi/2)%TAU,(angle+math.pi/2)%TAU})
    receive=[(lo,hi) for lo,hi in zip(cuts,cuts[1:]) if math.cos((lo+hi)/2-angle)>=0]
    return sum(max(0.,min(hi,b)-max(lo,a)) for lo,hi in intervals for a,b in receive)/TAU


def hypothesis_mass(source,positive,negative,problem,mode='full'):
    """积分计算均匀接收半径与两种源类型各半的参考先验。
    以阴性距离作半径分段，段内有效方向约束固定，积分精度仅受浮点运算限制。该先验用于动作决策，不是对官方生成规律的断言。
"""
    if math.hypot(*source)>1800:return 0.,0.,[]
    lower=max([1000.]+[distance(source,p) for p in positive]);upper=1500.
    if lower>=upper:return 0.,0.,[]
    if problem=='q3':
        upper=min([upper]+[distance(source,p) for p in negative])
        mass=max(0.,upper-lower)/500
        return mass,mass,[]
    if mode=='radius':return (upper-lower)/500,(upper-lower)/500,[]
    distances=[(distance(source,p),p) for p in negative]
    breaks=sorted({lower,upper}.union(d for d,p in distances if lower<d<upper))
    omni=0.;components=[]
    for lo,hi in zip(breaks,breaks[1:]):
        active=[p for d,p in distances if d<(lo+hi)/2]
        weight=(hi-lo)/500
        if not active:omni+=.5*weight
        intervals=angle_intervals(source,positive,active)
        if intervals:components.append((.5*weight,intervals))
    mass=omni+sum(w*sum(hi-lo for lo,hi in intervals)/TAU for w,intervals in components)
    return mass,omni,components


class PosteriorMixin:
    def __init__(self,session,problem,settings=None):
        super().__init__(session,problem,settings)
        self.local_history={ch:[] for ch in self.tracks}
        self.cfg.setdefault('posterior_mode','full')

    def observe(self,ch,p):
        response=super().observe(ch,p)
        self.local_history[ch].append((tuple(p),response['measure_result']))
        return response

    def choose_pair(self,original,track,**kwargs):
        baseline=original(track,**kwargs)
        if self.cfg['posterior_mode']=='nearest':
            if baseline and
distance(self.s.position,baseline['points'][1])<distance(self.s.position,baseline['points'][0])-1e-9:
                return dict(baseline,points=list(reversed(baseline['points'])),
```

21 问题三四_候选区域概率计算.py 按合法历史计算参考权重

```
                            max_source_distance_m=list(reversed(baseline['max_source_distance_m'])))
            return baseline
        if not track.poly or track.inconsistent or track.cleared or not track.first:return baseline
        origin,bearing=track.first;u=unit(bearing)
        xs=[(p[0]-origin[0])*u[0]+(p[1]-origin[1])*u[1] for p in track.poly]
        lo,hi=min(xs),max(xs);span=hi-lo
        candidates=[]
        if baseline:candidates.append(baseline)
        for fraction in (.25,.4,.55,.7,.85):
            for lateral in (30.,60.,100.,180.):
                pair=original(track,forward=max(.001,lo+fraction*span),lateral=lateral)
                if pair:candidates.append(pair)
        if not candidates:return baseline
        positives=[p for p,r in self.local_history[track.channel] if r!='no_signal']
        negatives=[p for p,r in self.local_history[track.channel] if r=='no_signal']
        samples=[]
        for source,area_weight in sample_polygon(track.poly):

mass,omni,components=hypothesis_mass(source,positives,negatives,self.problem,self.cfg['posterior_mode'])
            if mass>0:samples.append((source,area_weight*mass,mass,omni,components))
        if not samples:return super().choose_pair(original,track,**kwargs)
        total=sum(x[1] for x in samples);best=None
        for pair in candidates:
            points=sorted(pair['points'],key=lambda p:distance(self.s.position,p))
            if points!=pair['points']:

pair=dict(pair,points=points,max_source_distance_m=list(reversed(pair['max_source_distance_m'])))
            q=points[0];estimate=distance(self.s.position,q)/5+3
            for source,weight,mass,omni,components in samples:
                poly=list(track.poly)
                angle=math.degrees(math.atan2(source[1]-q[1],source[0]-q[0]))
                for plane in bearing_planes(q,angle,1.005,1500):poly=clip(poly,plane)
                circle=enclosing_circle(poly) if poly else track.circle;r=circle[1]
                positive=max(0.,distance(q,circle[0])-max(0.,19.99-
r))/5+5+self.cfg['pair_cost_weight']*max(0.,r-19.99)*.25
                if self.problem=='q3':probability=1.
                elif self.cfg['posterior_mode']=='radius':probability=receive_probability(origin,q,source)
                else:probability=(omni+sum(w*halfplane_mass(intervals,source,q) for w,intervals in
components))/mass
                negative=distance(q,points[1])/5+3+distance(points[1],source)/5+5
                estimate+=weight/total*(probability*positive+(1-probability)*negative)
            key=(estimate,tuple(points))
            if best is None or key<best[0]:best=(key,pair)
        self.events.append(dict(method='m020_posterior_pair_ranking',channel=track.channel,
            evaluated_candidates=len(candidates),selected_estimated_cost_s=best[0][0],
            ranking_only=True,synthetic_prior='uniform_area_radius_50pct_directional',
            positive_points=positives,negative_points=negatives,source_polygon=track.poly,
            selected_points=best[1]['points'],posterior_mode=self.cfg['posterior_mode']))
        return best[1]


class PosteriorQ3(PosteriorMixin,LocalQ3):pass
class PosteriorQ4(PosteriorMixin,LocalQ4):pass


def solve(session,problem,settings=None):
    cls=PosteriorQ3 if problem=='q3' else PosteriorQ4
    return cls(session,problem,bootstrap.baseline_settings(problem) if settings is None else settings).run()
```

22 问题三四_物理约束调度.py 定向接收信息收益

```
"""将 M020 名义信息评分的天线半角修正为 90°。
早期排名近似曾取 60°，该值未改变真实测量或覆盖证书；修正候选独立保存以支持同场比较。"""
import math
from 问题三四_信息调度基线 import SchedulingMixin, angle_delta
from 问题三四_几何求解基线 import Solver
from 问题三_阴性观测收缩 import NegativeSolver
from 问题三四_模型加载 import baseline_settings
from 问题一_定位区域与包围圆 import distance, clip, enclosing_circle, bearing_planes

class CorrectedMixin(SchedulingMixin):
    def predicted_gain(self,t,p):
        if self.problem=='q3':return super().predicted_gain(t,p)
        if t.cleared or t.inconsistent or not t.circle or not t.poly:return 0.
        center,radius=t.circle
        if radius<40:return 0.
        if self.measured[t.channel] and min(distance(p,q) for q in self.measured[t.channel])<60:return 0.
        length=distance(p,center)
        if not 25.<length<1450.:return 0.
        angle=math.degrees(math.atan2(center[1]-p[1],center[0]-p[0]))%360.
        narrowed=list(t.poly)
        for plane in bearing_planes(p,angle,1.005,max_range=1500.):narrowed=clip(narrowed,plane)
        if not narrowed:return 0.
        gain=max(0.,radius-enclosing_circle(narrowed)[1])
        views=[math.degrees(math.atan2(o['position'][1]-center[1],o['position'][0]-center[0])) for o in
t.observations]
        view=math.degrees(math.atan2(p[1]-center[1],p[0]-center[0]))
        allowed=[a for a in range(0,360,5) if all(abs(angle_delta(a,v))<=90 for v in views)]
        directional=sum(abs(angle_delta(a,view))<=90 for a in allowed)/len(allowed) if allowed else 0.
        return gain*(.5+.5*directional)

class Q3CorrectedSolver(CorrectedMixin,NegativeSolver):pass
class Q4CorrectedSolver(CorrectedMixin,Solver):pass

def solve(session,problem,settings=None):
    return (Q3CorrectedSolver if problem=='q3' else Q4CorrectedSolver)(session,problem,
        settings if settings is not None else baseline_settings(problem)).run()
```

23 问题三四_信息调度基线.py 搜索定位与清除调度

```
"""只依实际观测联合安排原地测量、扫描与延后服务。
几何、有限清除后备及连续覆盖检查沿用原条件；预测信息收益只改变顺序，不删除覆盖任务或可行源位置。"""
import math
from 问题三四_模型加载 import baseline_settings
from 问题三四_几何求解基线 import Solver, shifted_route
from 问题三_阴性观测收缩 import NegativeSolver
from 问题一_定位区域与包围圆 import distance, clip, enclosing_circle, bearing_planes


def angle_delta(a,b):
    return (a-b+180.)%360.-180.


class SchedulingMixin:
    def __init__(self,session,problem,settings=None):
        super().__init__(session,problem,settings)
        self.cfg.setdefault('stationary_gain_m',float('inf'))
        self.cfg.setdefault('defer_factor',0.)
        self.cfg.setdefault('defer_radius_m',80.)
        self.cfg.setdefault('route_mode','two_opt')
        self.cfg.setdefault('service_projection',False)
        self.cfg.setdefault('route_action_points',False)
        self.cfg.setdefault('resolution_recourses',0)
        self.cfg.setdefault('warm_route',False)
        self.cfg.setdefault('warm_optimize',False)
        self.recourses={ch:0 for ch in self.tracks}
        self.route_suffix=[]
        if self.cfg.get('prospective') or self.sectors:
            raise ValueError('Scheduling prototype does not silently handle prospective/sectors')

    def predicted_gain(self,t,p):
        if t.cleared or t.inconsistent or not t.circle or not t.poly:return 0.
        center,radius=t.circle
        if radius<40:return 0.
        if self.measured[t.channel] and min(distance(p,q) for q in self.measured[t.channel])<60:return 0.
        length=distance(p,center)
        if not 25.<length<1450.:return 0.
        # 名义示向仅用于优先级估计，不写作真实观测。
        angle=math.degrees(math.atan2(center[1]-p[1],center[0]-p[0]))%360.
        narrowed=list(t.poly)
        for plane in bearing_planes(p,angle,1.005,max_range=1500.):
            narrowed=clip(narrowed,plane)
        if not narrowed:return 0.
        expected_radius=enclosing_circle(narrowed)[1]
        gain=max(0.,radius-expected_radius)
        if self.problem=='q4':
            # 以中心估计的接收概率仅用于选动作；实际
            # 无信号仍由冻结物理模型决定。
            views=[math.degrees(math.atan2(o['position'][1]-center[1],
                                           o['position'][0]-center[0])) for o in t.observations]
            view=math.degrees(math.atan2(p[1]-center[1],p[0]-center[0]))
            allowed=[a for a in range(0,360,5) if all(abs(angle_delta(a,v))<=60 for v in views)]
            directional=sum(abs(angle_delta(a,view))<=60 for a in allowed)/len(allowed) if allowed else 0.
            gain*=.5+.5*directional
        return gain

    def gather(self):
        threshold=self.cfg['stationary_gain_m']
        if not math.isfinite(threshold):return
        pending=[t for t in self.tracks.values() if t.observations and not t.cleared]
        scored=[(self.predicted_gain(t,self.s.position),t) for t in pending]
        for score,t in sorted(scored,key=lambda row:(-row[0],row[1].channel)):
            if score<threshold:continue
            p=tuple(self.s.position)
            self.events.append(dict(method='stationary_information_choice',channel=t.channel,
                                    point=p,predicted_radius_gain_m=score))
            self.observe(t.channel,p)

    def defer(self,t):
        factor=self.cfg['defer_factor']
        if not factor or not self.todo or not t.circle or t.circle[1]<self.cfg['defer_radius_m']:
            return False
        target=self.target_point(t);origin=self.s.position
```

23 问题三四_信息调度基线.py 搜索定位与清除调度

```
        for _,p in self.todo:
            extra=distance(origin,p)+distance(p,target)-distance(origin,target)
            gain=self.predicted_gain(t,p)
            if gain>60. and extra<factor*gain:
                return True
        return False

    def attempt(self,track,p,method):
        if self.cfg['service_projection'] and method=='center_probe' and track.poly and not
track.inconsistent:
            radius=max(distance(p,q) for q in track.poly)
            budget=max(0.,19.99-radius)
            length=distance(self.s.position,p)
            if budget and length:
                fraction=min(1.,budget/length)
                p=(p[0]+fraction*(self.s.position[0]-p[0]),p[1]+fraction*(self.s.position[1]-p[1]))
        return super().attempt(track,p,method)

    def resolve(self,t):
        """定位更新代价较大时返回联合调度。
        每源中断次数有界，原定位器继续作为后备，避免破坏有限完成条件。"""
        cap=self.cfg['resolution_recourses']
        if (not cap or t.cleared or t.near is not None or not t.circle or t.inconsistent
                or t.circle[1]<80. or self.recourses[t.channel]>=cap):
            return super().resolve(t)
        if self.cfg.get('pre_sense') and hasattr(self,'useful') and self.useful(t):
            self.observe(t.channel,self.s.position);return
        self.recourses[t.channel]+=1
        if len(t.observations)==1 and t.circle[1]>=200.:
            from 问题三四_安全定位候选 import safe_pair_candidates, negative_pair_plane
            from 问题一_定位区域与包围圆 import unit
            pair=None
            if self.cfg['pair_probe']:
                forward=600.;lateral=self.cfg['lateral']
                if self.cfg['adaptive_pair']:
                    u=unit(t.first[1]);o=t.first[0]
                    xs=[(q[0]-o[0])*u[0]+(q[1]-o[1])*u[1] for q in t.poly]
                    forward=(min(xs)+max(xs))/2
                    lateral=min(120.,max(30.,(max(xs)-min(xs))*.15))
                pair=safe_pair_candidates(t,forward=forward,lateral=lateral)
            if pair:
                if self.cfg.get('nearest_pair') and
distance(self.s.position,pair['points'][1])<distance(self.s.position,pair['points'][0])-1e-9:

pair=dict(pair,points=list(reversed(pair['points'])),max_source_distance_m=list(reversed(pair['max_source_dist
ance_m'])))
                responses=[]
                for q in pair['points']:
                    responses.append(self.observe(t.channel,q))
                    if responses[-1]['measure_result']!='no_signal':break
                plane=negative_pair_plane(pair,responses)
                if plane:
                    t.poly=clip(t.poly,plane)
                    if not t.poly:t.inconsistent=True
                    else:t.circle=enclosing_circle(t.poly)

self.events.append(dict(method='two_sided_negative_halfplane',channel=t.channel,pair=pair,responses=responses,
poly=t.poly))
            else:self.observe(t.channel,self.target_point(t))
        else:
            p=t.circle[0]
            if not self.attempt(t,p,'center_probe'):
                self.observe(t.channel,p)

self.events.append(dict(method='localization_recourse',channel=t.channel,recourse=self.recourses[t.channel]))
        if t.cleared:
            # 即使近距
            # 反馈已清除本轮定位目标，仍保留 Q3 带证书的条件替站。
            return super().resolve(t)

    def action_points(self,t):
        """预测定位器首个付费停车点及源服务点。
        路线目标与定位器实际首先使用的双侧点或圆心可能不同；预测只服务排序，真实动作仍由定位器提交。"""
```

23 问题三四_信息调度基线.py 搜索定位与清除调度

```
        if t.near is not None:return t.near,t.near
        entry=self.target_point(t)
        destination=t.circle[0] if t.circle and not t.inconsistent else entry
        if t.circle and not t.inconsistent:
            if len(t.observations)==1 and t.circle[1]>=200:
                if self.cfg['pair_probe']:
                    from 问题三四_安全定位候选 import safe_pair_candidates
                    from 问题一_定位区域与包围圆 import unit
                    forward=600.;lateral=self.cfg['lateral']
                    if self.cfg['adaptive_pair']:
                        u=unit(t.first[1]);o=t.first[0]
                        xs=[(q[0]-o[0])*u[0]+(q[1]-o[1])*u[1] for q in t.poly]
                        forward=(min(xs)+max(xs))/2
                        lateral=min(120.,max(30.,(max(xs)-min(xs))*.15))
                    pair=safe_pair_candidates(t,forward=forward,lateral=lateral)
                    if pair:
                        entry=pair['points'][0]
                        if self.cfg.get('nearest_pair'):
                            entry=min(pair['points'],key=lambda q:distance(self.s.position,q))
            else:entry=destination
        return entry,destination

    def action_route(self,nodes):
        endpoints=[]
        for key,p in nodes:
            entry,end=self.action_points(self.tracks[key[1]]) if key[0]=='target' else (p,p)
            endpoints.append((entry,end))
        origin=self.s.position
        def cost(order):
            total=0.;previous=origin
            for i in order:
                entry,end=endpoints[i];total+=distance(previous,entry);previous=end
            return total
        remaining=list(range(len(nodes)));order=[];previous=origin
        while remaining:
            i=min(remaining,key=lambda i:distance(previous,endpoints[i][0]))
            order.append(i);remaining.remove(i);previous=endpoints[i][1]
        best=cost(order)
        # 开放有向路线的 2-opt 比较中，任务内部从进入到离开的费用与
        # 排列位置无关，因此不加入这项顺序比较。
        changed=True
        while changed:
            changed=False
            for i in range(len(order)-1):
                for j in range(i+1,len(order)):
                    candidate=order[:i]+list(reversed(order[i:j+1]))+order[j+1:]
                    value=cost(candidate)
                    if value<best-1e-7:
                        order,best=candidate,value;changed=True
        return [nodes[i] for i in order]

    def select_route(self,nodes):
        fresh=shifted_route(nodes,self.s.position)
        if not self.cfg['warm_route']:return fresh
        by_key={key:p for key,p in nodes}
        previous=[(key,by_key[key]) for key in self.route_suffix if key in by_key]
        included={key for key,p in previous}
        origin=self.s.position
        def cost(route):
            return sum(distance(p,q) for p,q in zip([origin]+[p for _,p in route[:-1]],[p for _,p in route]))
        for key,p in nodes:
            if key in included:continue
            trials=[previous[:i]+[(key,p)]+previous[i:] for i in range(len(previous)+1)]
            previous=min(trials,key=cost);included.add(key)
        chosen=min((fresh,previous),key=cost)
        if self.cfg['warm_optimize']:
            best=cost(chosen);changed=True
            while changed:
                changed=False
                for i in range(len(chosen)-1):
                    for j in range(i+1,len(chosen)):
                        candidate=chosen[:i]+list(reversed(chosen[i:j+1]))+chosen[j+1:]
                        value=cost(candidate)
```

23 问题三四_信息调度基线.py 搜索定位与清除调度

```
                        if value<best-1e-7:
                            chosen,best=candidate,value;changed=True
        self.route_suffix=[key for key,p in chosen]
        return chosen

    def run(self):
        self.scan((0.,0.),True)
        self.todo=[(i,p) for i,p in self.todo if distance(p,(0.,0.))>1e-7]
        while self.todo or any(t.observations and not t.cleared for t in self.tracks.values()):
            self.gather()
            pending=[t for t in self.tracks.values() if t.observations and not t.cleared]
            if self.todo:
                pending=[t for t in pending if (t.near is not None or
                    (t.circle and t.circle[1]<=self.cfg['ready_radius'])) and not self.defer(t)]
            active_todo=self.todo
            if self.problem=='q4' and self.cfg['inner_first']:
                inner=[(i,p) for i,p in active_todo if math.hypot(*p)<1100]
                if inner:active_todo=inner
            nodes=[(('scan',i),p) for i,p in active_todo]
            if self.cfg['joint'] or not nodes:
                nodes += [(('target',t.channel),self.target_point(t)) for t in pending]
            if not nodes:break
            if self.cfg['route_action_points']:
                key,p=self.action_route(nodes)[0]
            elif self.cfg['route_mode']=='nearest':
                key,p=min(nodes,key=lambda row:distance(self.s.position,row[1]))
            else:
                key,p=self.select_route(nodes)[0]
            if key[0]=='scan':
                self.todo=[(i,q) for i,q in self.todo if i!=key[1]]
                self.scan(p,True)
            else:self.resolve(self.tracks[key[1]])
        self.coverage_complete=not self.todo
        cleared=sorted(ch for ch,t in self.tracks.items() if t.cleared)
        absent=sorted(ch for ch,t in self.tracks.items() if not t.observations)
        complete=not self.todo and len(cleared)+len(absent)==20 and 10<=len(cleared)<=16 and not
any(t.inconsistent for t in self.tracks.values())
        return dict(termination='coverage_complete' if complete else
'incomplete',cleared_channels=cleared,absent_channels=absent,
            coverage_evidence=dict(kind='source_count_upper_bound' if self.count_stop else
('continuous_actual_sites' if self.cfg['adaptive_cover'] else 'fixed_sites'),
                problem=self.problem,design=self.cfg['cover'],sites=self.fixed,actual_scan_sites=self.scanned,
                measurements_per_channel=self.measured),
            geometry_inconsistencies=[ch for ch,t in self.tracks.items() if t.inconsistent],
            events=self.events,tracks=[t.snapshot() for t in self.tracks.values() if t.observations])


class Q3SchedulingSolver(SchedulingMixin,NegativeSolver):pass
class Q4SchedulingSolver(SchedulingMixin,Solver):pass


def solve(session,problem,settings=None):
    cls=Q3SchedulingSolver if problem=='q3' else Q4SchedulingSolver
    return cls(session,problem,settings if settings is not None else baseline_settings(problem)).run()
```

24 问题三四_开放巡检路线优化.py 开放路线优化

```
"""确定性开放路径优化；规模较小的固定点集使用 Held-Karp 精确求解。"""
import math

def matrix(points,start):
    all_points=list(points)+[start]
    return [[math.dist(p,q) for q in all_points] for p in all_points]
def cost(order,d):
    return sum(d[a][b] for a,b in zip([len(d)-1]+list(order[:-1]),order))
def two_opt(order,d):
    order=list(order);n=len(order);origin=len(d)-1
    while True:
        best=-1e-8;pair=None
        for i in range(n-1):
            a=origin if i==0 else order[i-1];b=order[i]
            for j in range(i+1,n):
                c=order[j];delta=d[a][c]-d[a][b]
                if j+1<n:delta+=d[b][order[j+1]]-d[c][order[j+1]]
                if delta<best:best,pair=delta,(i,j)
        if pair is None:return order
        i,j=pair;order[i:j+1]=reversed(order[i:j+1])
def exact(points,start):
    n=len(points);d=matrix(points,start)
    if not n:return [],0.
    dp={(1<<i,i):(d[n][i],None) for i in range(n)}
    for mask in range(1,1<<n):
        for last in range(n):
            if (mask,last) not in dp:continue
            value,_=dp[mask,last]
            for nxt in range(n):
                if mask>>nxt&1:continue
                key=(mask|1<<nxt,nxt);candidate=value+d[last][nxt]
                if key not in dp or candidate<dp[key][0]-1e-10:dp[key]=(candidate,last)
    full=(1<<n)-1;last=min(range(n),key=lambda i:dp[full,i][0]);best=dp[full,last][0]
    order=[];mask=full
    while last is not None:
        order.append(last);previous=dp[mask,last][1];mask^=1<<last;last=previous
    return list(reversed(order)),best
def optimize(points,start,initial,exact_limit=10,relocate=True):
    n=len(points);d=matrix(points,start)
    if n<=exact_limit:
        order,value=exact(points,start)
        return order,dict(method='held_karp_fixed_points_float',nodes=n,length_m=value,
                          guarantee='Fixed predicted coordinates only; not online total-cost optimality')
    candidates=[two_opt(initial,d),two_opt(list(reversed(initial)),d)]
    for first in range(n):
        remaining=set(range(n));remaining.remove(first);order=[first]
        while remaining:
            nxt=min(remaining,key=lambda j:(d[order[-1]][j],j));remaining.remove(nxt);order.append(nxt)
        candidates.append(two_opt(order,d))
    best=min(candidates,key=lambda order:(cost(order,d),order));value=cost(best,d)
    if relocate:
        while True:
            change=None;new_value=value
            for width in (1,2,3):
                for i in range(n-width+1):
                    block=best[i:i+width];rest=best[:i]+best[i+width:]
                    for j in range(len(rest)+1):
                        if j==i:continue
                        trial=rest[:j]+block+rest[j:];v=cost(trial,d)
                        if v<new_value-1e-7:new_value,change=v,trial
            if change is None:break
            best=two_opt(change,d);value=cost(best,d)
    return best,dict(method='all_first_nearest_two_opt_relocation',nodes=n,length_m=value,
                     guarantee='Improves retained route on these predicted points; no global optimality
claim')
```

25 数据/参数/问题三_算法参数.json 第三问固定参数


```
{
  "scan_spacing": 500.0,
  "lateral": 60,
  "probe_rounds": 3,
  "joint": true,
  "extra_scan": false,
  "adaptive_cover": true,
  "cover": "ring22",
  "prune_min_box": 16.0,
  "pair_probe": true,
  "prune_distance": 2000.0,
  "prospective": false,
  "q3_ring_n": 6,
  "q3_ring_radius": 1125.0,
  "q4_ring_n": 12,
  "q4_ring_radius": 1867,
  "q4_inner_n": 8,
  "q4_inner_radius": 999,
  "q4_outer_phase": 15,
  "ready_radius": 10000,
  "centroid": true,
  "skip_impossible": true,
  "adaptive_pair": true,
  "adaptive_clear": true,
  "negative_disk": true,
  "radius_consistency": true,
  "pre_sense": true,
  "nearest_pair": true,
  "service_clear": true,
  "replace_scan": true,
  "inner_first": false,
  "first_forward": null,
  "movable_sectors": false,
  "sector_pull": 1.0,
  "post_sense": false,
  "sense_radius": 80.0,
  "sense_sine": 0.1,
  "local_mode": "pair",
  "posterior_mode": "full",
  "posterior_target": false,
  "posterior_optical": true,
  "optical_gain_threshold": 0
}
```

26 数据/参数/问题四_算法参数.json 第四问固定参数

```
{
  "cover": "ring22",
  "ready_radius": 400,
  "lateral": 300,
  "pair_probe": true,
  "extra_scan": false,
  "q4_inner_n": 8,
  "q4_inner_radius": 999,
  "q4_ring_n": 12,
  "q4_ring_radius": 1867,
  "q4_outer_phase": 15,
  "skip_impossible": true,
  "adaptive_pair": true,
  "adaptive_clear": true,
  "local_mode": "pair",
  "posterior_mode": "full",
  "posterior_target": false,
  "posterior_optical": true,
  "optical_gain_threshold": 0,
  "warm_route": true,
  "warm_optimize": true,
  "stationary_gain_m": 120,
  "scan_gain_m": 30,
  "scan_count_stop": true,
  "intercept_gain_m": 120,
  "intercept_efficiency": true,
  "intercept_other_only": true
}
```